# SafeCharge- Analysis and Defense of EV Charging Protocols


Project Team

| | |
|---|---|
| Muhammad Jibran Hanan | 22I-1732 |
| Kumail Haider | 22I-1723 |
| Adnan Khaliq | 22I-2529 |


Session 2022-2026


Supervised by

Dr. Zafar Iqbal






**Department of Computer Science**

**National University of Computer and Emerging Sciences**
**Islamabad, Pakistan**


**June, 2026**

# Contents


**1 Introduction** **1**

1.1 Background . . . 2
1.2 Problem Statement . . . 3
1.3 Software Requirements Specification . . . 3
1.3.1 Scope and Purpose . . . 4
1.3.2 Functional Requirements . . . 4
1.3.3 Non-Functional Requirements . . . 6
1.3.4 Constraints . . . 6
1.3.5 Verification and Success Criteria . . . 6
1.3.6 Summary . . . 7
1.4 Work Distribution . . . 7

**2 Literature Review** **10**

2.1 Related Research . . . 10
2.1.1 Cybersecurity Risk Analysis of Electric Vehicles Charging Stations 11
2.1.1.1 Article Summary . . . 11
2.1.1.2 Proposed Techniques and Results . . . 11
2.1.1.3 Critical analysis . . . 12
2.1.1.4 Relationship to the proposed framework . . . 12
2.1.2 Charge Manipulation Attacks Against Smart Electric Vehicle Charging Stations and Deep Learning-based Detection Mechanisms . . . 13
2.1.2.1 Article Summary . . . 13
2.1.2.2 Proposed Techniques and Results . . . 13
2.1.2.3 Critical analysis . . . 14
2.1.2.4 Relationship to the proposed framework . . . 14
2.1.3 Disrupting EV Charging Sessions and Gaining Remote Code Execution with DoS, MITM, and Code Injection Exploits using OCPP 1.6 . . . 15
2.1.3.1 Article Summary . . . 15
2.1.3.2 Proposed Techniques and Results . . . 15
2.1.3.3 Critical analysis . . . 16
2.1.3.4 Relationship to the proposed framework . . . 16

2.1.4 Federated Learning-Based Intrusion Detection for OCPP Cyber-attacks . . . 17
2.1.4.1 Article Summary . . . 17
2.1.4.2 Proposed Techniques and Results . . . 17
2.1.4.3 Critical analysis . . . 18
2.1.4.4 Relationship to the proposed framework . . . 18
2.1.5 Cyber Defense in OCPP for EV Charging Security Risks . . . 19
2.1.5.1 Article Summary . . . 19
2.1.5.2 Proposed Techniques and Results . . . 19
2.1.5.3 Critical analysis . . . 20
2.1.5.4 Relationship to the proposed framework . . . 20
2.1.6 OCPPStorm: A Comprehensive Fuzzing Tool for OCPP Implementations . . . 21
2.1.6.1 Article Summary . . . 21
2.1.6.2 Proposed Techniques and Results . . . 21
2.1.6.3 Critical analysis . . . 22
2.1.6.4 Relationship to the proposed framework . . . 23
2.1.7 EmuOCPP: Effective and Scalable OCPP Security and Privacy Testing . . . 23
2.1.7.1 Article Summary . . . 23
2.1.7.2 Proposed Techniques and Results . . . 24
2.1.7.3 Critical analysis . . . 25
2.1.7.4 Relationship to the proposed framework . . . 26
2.2 Analysis Summary of Research Items . . . 27
2.3 Research Scope/Gap . . . 29

**3 Proposed Framework** **31**
3.1 Introduction . . . 31
3.2 Research Framework Overview . . . 31
3.3 Specification-Level Analysis of OCPP . . . 33
3.3.1 Purpose and Objectives . . . 33
3.3.2 Analytical Scope and Methodology . . . 34
3.3.2.1 Protocol Structure Review . . . 34
3.3.2.2 Security Profiles and Cryptographic Mechanisms . . . 34
3.3.2.3 Message Structure and Flow Analysis . . . 34
3.3.2.4 Comparative Security Evolution Mapping . . . 35
3.3.2.5 Expected Deliverable . . . 35
3.4 Base Paper Implementation: EmuOCPP Framework . . . 36
3.4.1 Design and Core Components . . . 36
3.4.2 Security Evaluation . . . 36

3.4.3 Application of Base Paper . . . 37
3.5 Updating EmuOCPP to Support OCPP 2.1 . . . 38
3.5.1 Alignment with this Research . . . 38
3.6 Regression Testing . . . 38
3.6.1 Purpose and Role . . . 38
3.6.2 Testing Methodology . . . 39
3.7 EmuOCPP Flow . . . 39
3.7.1 Core Components . . . 39
3.7.1.1 Configuration Layer . . . 40
3.7.1.2 Security Processing . . . 40
3.7.1.3 Network Emulation . . . 40
3.7.1.4 OCPP Messaging and User State . . . 40
3.7.1.5 Monitoring and Analysis . . . 40
3.7.2 Graphical Description . . . 41
3.7.2.1 End-to-End Flow Diagram . . . 41
3.8 Experimental Setup . . . 42
3.8.1 Evaluation Metrics . . . 43
3.8.1.1 Security Profile Behaviour . . . 43
3.8.1.2 Message Manipulation Capability . . . 43
3.8.1.3 Identity Duplication and Session Control . . . 44
3.8.1.4 Certificate and Provisioning Integrity . . . 44
3.8.1.5 Post-Attack Communication State . . . 44
3.8.1.6 Summary . . . 45
3.9 Results . . . 45
3.9.1 Research Paper Attack Verification . . . 45
3.9.1.1 Experimental Setup for Attack M1–M4 . . . 46
3.9.1.2 M1 – MitM on SP1 / No Security Profile (MitMSP1) . . . 47
3.9.1.3 M2 – MitM During SP2 Upgrade (MitMSP2Upgrade) . . . 48
3.9.1.4 M3 – MitM During SP3 Upgrade (MitMSP3Upgrade) . . . 49
3.9.1.5 M4 – Downgrade from SP3/SP2 to SP1 (MitMSPDowngrade) . . . 50
3.9.1.6 Summary of Verification Outcomes . . . 52
3.9.1.7 Demonstration of Attacks M1–M4 . . . 53
3.9.1.8 I1 – CS Impersonation with Duplicate Serial Identity . . . 63
3.9.1.9 I2 – CS Impersonation During Reboot Window (ImpersonationI2) . . . 69
3.10 Analysis and/or Discussion . . . 73
3.10.1 Verification of Man-in-the-Middle Attacks . . . 73
3.10.1.1 M1: Insecure Security Profile (SP1) . . . 73
3.10.1.2 M2: Security Profile Upgrade (SP2) . . . 73

3.10.1.3 M3: Mutual-TLS Certificate Abuse (SP3) . . . . . 73
3.10.1.4 M4: Security Profile Downgrade . . . . . 74
3.10.2 Verification of Impersonation Attacks . . . . . 74
3.10.2.1 I1: Concurrent Identifier Impersonation . . . . . 74
3.10.2.2 I2: Offline Timing Impersonation . . . . . 74
3.10.3 Root Cause Analysis . . . . . 75
3.10.4 Implications for Real-World Deployments . . . . . 75
3.10.5 Summary . . . . . 76

**4 Theoretical Framework** **77**
4.1 Understanding Packet Inspection . . . . . 77
4.1.1 Overview . . . . . 77
4.1.2 Core Components . . . . . 77
4.1.2.1 Wireshark Dissector Framework . . . . . 77
4.1.2.2 Packet Capture Process . . . . . 78
4.1.2.3 Message Parsing and Structure Analysis . . . . . 78
4.1.2.4 Schema-Based Validation . . . . . 79
4.1.2.5 Protocol Tree Display . . . . . 79
4.1.2.6 Expert Information and Compliance Checking . . . . . 79
4.1.2.7 Installation and Configuration . . . . . 80
4.1.2.8 Integration with Attack Scenarios . . . . . 80
4.1.2.9 Limitations and Considerations . . . . . 80
4.1.3 Practical Usage . . . . . 80
4.2 EmuOCPP Framework Role and High-Level Overview . . . . . 81
4.3 Software Architecture and Core Services . . . . . 81
4.3.1 Server and Certificate Authority Components . . . . . 81
4.3.2 Charge Point Client and Management API . . . . . 82
4.3.3 Data Model, Configuration and Credentials . . . . . 82
4.4 Networking, Scenarios and Adversarial Tooling . . . . . 83
4.4.1 Network Emulation and Topology Management . . . . . 83
4.4.2 Scenario Scripts and Automated Experiment Flows . . . . . 83
4.4.3 MitM Tooling and Downgrade Attacks . . . . . 83
4.4.4 Protocol Dissector and Packet Analysis . . . . . 84
4.5 Comparative Security Analysis of OCPP 2.0.1 and OCPP 2.1 . . . . . 84
4.5.1 Methodology . . . . . 84
4.5.2 Findings . . . . . 85
4.5.2.1 Security Foundations of OCPP 2.0.1 . . . . . 85
4.5.2.2 Security Enhancements in OCPP 2.1 . . . . . 85
4.5.3 Evaluation . . . . . 86
4.5.4 Comparison Table . . . . . 86

**5 Experiments and Results** **88**
5.1 Experimental Setup . . . 88
5.1.1 Context of the OCPP 2.1 Upgrade . . . 88
5.1.1.1 A02 – Update Charging Station Certificate by request of CSMS . . . 88
5.1.1.2 A03 – Update Charging Station Certificate initiated by the Charging Station . . . 91
5.1.1.3 A05 – Upgrade Charging Station Security Profile . . . 93
5.1.1.4 B09 – Setting a new NetworkConnectionProfile . . . 96
5.1.1.5 F06 – Trigger Message . . . 99
5.1.1.6 M01 – Certificate Installation EV . . . 101
5.1.1.7 CALLRESULTERROR Asynchronous Operation Failure Reporting . . . 104
5.1.1.8 SEND message type . . . 106
5.1.1.9 The OCPP 2.1 Compatibility Shim . . . 108
5.1.2 Evaluation Metrics . . . 109
5.2 Results . . . 110
5.3 Analysis and/or Discussion . . . 111
5.3.1 SEND Flood DoS Attack (D2) in OCPP 2.1 . . . 111
5.3.1.1 Vulnerability identification from the specification . . . 111
5.3.1.2 Attack intuition and threat model . . . 111
5.3.1.3 Implementation in EmuOCPP (`send_dos.py`) . . . 112
5.3.1.4 Expected impact and interpretation . . . 112
5.3.1.5 Pseudocode and explanation . . . 114
5.3.1.6 SEND DoS Mitigation (Per-CP rate limiting + alerting + containment) . . . 114
5.3.2 Man In The Middle Attacks (M1-M4) in OCPP 2.1 . . . 118
5.3.2.1 Why the attacker cannot gain MitM position in OCPP 2.1 118
5.3.2.2 How certificate management breaks the M1–M4 chain . 119
5.3.2.3 Implications for each MitM phase (M1–M4) . . . 120
5.3.2.4 Conclusion . . . 123
5.3.3 Impersonation Attacks (I1 and I2) . . . 123
5.3.3.1 Impersonation Attacks (I1 and I2) . . . 124
5.3.3.2 I1 — Impersonation with victim online . . . 124
5.3.3.3 Experimental Validation of I1 Mitigation in OCPP 2.1 Under Policy 1 . . . 126
5.3.3.4 I2 — Impersonation in offline/reconnect window . . . 127
5.3.3.5 Evidence of I2 Impersonation Attack Mitigation in OCPP 2.1 . . . 129

5.3.3.6 Cross-version discussion: why 2.0.1 succeeded, why 2.1 did not . . . 131
5.3.3.7 Cross-version discussion: why 2.0.1 succeeded, why 2.1 did not . . . 131
5.3.3.8 Residual risk and scope . . . 132

**6 Conclusions and Future Work** **133**
6.1 Summary of Findings . . . 133
6.2 Research Contributions . . . 134
6.3 Limitations . . . 135
6.4 Future Work . . . 135

**A Background** **136**
A.1 IPMininet . . . 136
A.2 CheckOCPP . . . 136
A.3 Mobility House OCPP Python Library . . . 137
A.4 List of Abbreviations . . . 138

**References** **140**

# List of Figures


3.1 High-level research framework representation . . . . . 33
3.2 High-level EmuOCPP flow . . . . . 41
3.3 Initial EmuOCPP network setup for attacks M1–M4 showing the CSO (h4), CSMS (h3), and client (h1) prior to MitM positioning. . . . . 53
3.4 Attack setup on host h2 using IPv6 manipulation tools and a transparent proxy to establish a persistent on-path position. . . . . 54
3.5 Simultaneous view of MitM interception and server-side duplicate connection detection during a connection hijacking event. . . . . 55
3.6 Successful certificate manipulation and persistent MitM interception after security profile upgrades. . . . . 56
3.7 Successful SP1 to SP2 upgrade with maintained MitM interception. . . . 57
3.8 Successful SP2 to SP3 upgrade with maintained MitM interception. . . . 58
3.9 Initial phase of the M4 downgrade attack showing selective traffic blocking. 59
3.10 Successful downgrade from SP3 to SP1 through forced connection failures. 60
3.11 Attacker-side view of Attack I1 under server policy 0 . . . . . 66
3.12 Server-side view of Attack I1 under server policy 0 . . . . . 66
3.13 Attacker-side view of Attack I1 under server policy 1 . . . . . 67
3.14 Server-side view of Attack I1 under server policy 1 . . . . . 67
3.15 Attacker-side view of Attack I1 under server policy 2 . . . . . 68
3.16 Server-side view of Attack I1 under server policy 2 . . . . . 68
3.17 Attacker-side view of Attack I2 . . . . . 72
3.18 Server-side view of Attack I2 . . . . . 72

5.1 *Sequence diagram for use case A02.* . . . . . 90
5.2 *Sequence diagram for use case A03.* . . . . . 92
5.3 *Sequence diagram for use case A05* . . . . . 94
5.4 *Sequence diagram for use case B09.* . . . . . 97
5.5 *Sequence diagram for use case F06.* . . . . . 99
5.6 *Sequence diagram for use case M01 – contract certificate installation initiated by the EV, forwarded by the Charging Station to the CSMS as a* `Get15118EVCertificate` *exchange, with the signed material relayed back to the EV through the ISO 15118 channel.* . . . . . 102
5.7 *The server continuously receiving and printing SEND messages* . . . . . 113

5.8 *The abnormal CPU spike observed during the attack phase* . . . . . 113
5.9 *CSMS mitigation logs during a SEND flood: after receiving multiple MessageTypeId* $= 6$ *frames, the CSMS detects that the sender exceeds the configured per-CP SEND rate (e.g., 51 messages in 1.0s with limit 50/s), begins dropping SEND frames, raises an explicit SEND DoS alert, and disconnects the abusive session.* . . . . . 117
5.10 *Client reconnection behaviour under OCPP 2.1 when executing M1-M4, the connection repeatedly establishes under Security Profile 3 and is subsequently closed, preventing the attacker from maintaining a stable MitM position.* . . . . . 118
5.11 *Attacker terminal output when attempting the M1-M4 MitM chain under OCPP 2.1: the attacker does not gain a stable MitM position and no meaningful interception occurs, indicating that the attack path is mitigated.* 119
5.12 *Excerpt from the OCPP 2.1 specification highlighting that operating without implementing security (i.e., relying on unsecured transport / SP1-like operation) is not considered a valid OCPP 2.1 implementation, and recommending TLS-based security profiles for field operation.* . . . . . 119
5.13 Attacker terminal output during impersonation attack I1 under OCPP 2.1 SP3 with Policy 1 enabled. The attacker attempts to connect using the same CP ID as the legitimate charging station while the victim session is already active. Although Policy 1 permits simultaneous sessions, the attacker connection is rejected because no valid client certificate is provided for SP3 authentication. . . . . 126
5.14 CSMS logs during impersonation attack I1 under OCPP 2.1 SP3. The highlighted entries show that the server enforces mutual TLS authentication and rejects the attacker connection with the error message “SP3 requires a client certificate. No client certificate provided.” The logs further confirm that the legitimate charging station session remains active while the attacker connection is denied. . . . . 127
5.15 Attacker terminal output for impersonation I2 under OCPP 2.1 SP3. The highlighted lines show that the attacker first triggers an offline window by forcing the victim to disconnect (code 1012), then attempts to reconnect using the same CP ID. The CSMS rejects the takeover because the attacker does not present the bound client certificate, and the script proceeds to the victim reconnect step. . . . . 129
5.16 Victim client output during impersonation I2. The highlighted boxes show that the legitimate CP connects under SP3, is forcefully disconnected by the CSMS (service restart window), and then reconnects successfully. This indicates that while the attacker can create a disruption window, it cannot steal the session without the bound client certificate. . . . . 130

5.17 CSMS logs during impersonation I2 under OCPP 2.1 SP3. The highlighted entries show the CSMS enforcing mutual TLS and rejecting the attacker connection because no client certificate is provided. After the forced disconnect, the legitimate CP reconnects and continues operation, confirming that timing alone is insufficient to hijack the CP ID when certificate binding is enforced. . . . . 131

# List of Tables

2.1 Summary of critical analysis of research items . . . . . 27

3.1 Summary of Attacks in EmuOCPP . . . . . 45

4.1 Key differences between OCPP 2.0.1 and OCPP 2.1, focusing on security, provisioning, monitoring, and identity. . . . . 87

5.1 *Attack evaluation summary for EmuOCPP under OCPP 2.1.* . . . . . 110

A.1 Abbreviations and OCPP use-case identifiers used throughout the document. 138

# Chapter 1

# Introduction

The rapid expansion of Electric Vehicle (EV) adoption has created an increasing need for secure and interoperable charging infrastructure. As EV networks grow globally, ensuring the integrity and resilience of communication between charging stations and management systems has become a critical concern. This study focuses on strengthening the cybersecurity of the EV charging ecosystem, with particular emphasis on evaluating the security posture of the Open Charge Point Protocol (OCPP) and developing an emulation-based testing framework targeting its most recent release, OCPP 2.1.

The transition to electric mobility is accelerating worldwide, driven by environmental policies, technological innovation, and rising consumer demand. As the number of charging stations both public and private continues to grow, the need for reliable, standardized, and secure communication between Electric Vehicle Charging Stations (EVCSs) and Central Management Systems (CSMS) becomes increasingly vital. The Open Charge Point Protocol (OCPP), maintained by the Open Charge Alliance (OCA), has emerged as the de facto global standard enabling interoperability across diverse EV charging networks [15].

While successive versions of OCPP (1.6, 2.0, 2.0.1, and 2.1) have progressively strengthened the protocols security framework introducing encryption mechanisms, mutual authentication, and formalized security profiles comprehensive, independent validation of these improvements remains limited. Most prior studies have concentrated on OCPP 1.6 or provided only preliminary assessments of OCPP 2.0/2.0.1, leaving significant uncertainty regarding the robustness of OCPP 2.1 against contemporary cyber threats [4, 6, 15].

This research aims to address that gap by conducting a comparative, emulation-driven analysis of OCPP 2.1 relative to its predecessors. The study will extend existing OCPP testing environments, such as *EmuOCPP* [1], to simulate real-world communication scenarios and evaluate the resilience of security features under adversarial conditions. The resulting insights will inform practical mitigation strategies for stakeholders including vendors, charging network operators, and cybersecurity researchers.

The subsequent sections of this chapter starting with outlining the background of this work, followed by a formal research problem statement and a software requirements specification that guides the planned implementation and evaluation. Finally, Work Distribution describes how tasks are allocated across the project.

## 1.1 Background

The global transition to electric mobility represents one of the most significant shifts in transportation and energy systems of the 21st century, driven by environmental policies, technological innovation, and growing consumer demand. This rapid expansion of electric vehicle (EV) adoption has created an urgent need for secure, reliable, and interoperable charging infrastructure.

As EV networks grow globally, ensuring the integrity and resilience of communication between charging stations and management systems has become a critical cybersecurity concern. The Open Charge Point Protocol (OCPP), maintained by the Open Charge Alliance (OCA), has emerged as the de facto global standard enabling interoperability across diverse EV charging networks, adopted in 148 countries and supported by over 220 member companies. The protocol has evolved through several major versions, 1.6, 2.0, 2.0.1, and most recently 2.1, with each iteration introducing enhanced security features including encryption mechanisms, mutual authentication, and formalized security profiles.

Despite these improvements, significant security challenges persist, particularly in widely deployed versions such as OCPP 1.6, which was designed with optional security features leading to vulnerabilities like Man-in-the-Middle attacks, impersonation, denial-of-service, false data injection, and remote code execution. While newer OCPP versions introduce mandatory security measures, comprehensive independent validation remains limited, with most research focusing on older versions and existing testing frameworks like OCPPStorm and EmuOCPP lacking support for OCPP 2.1.

The release of OCPP 2.1 in early 2025 introduces over fifty behavioral and structural modifications, yet no systematic framework exists to evaluate whether these enhancements effectively mitigate previous vulnerabilities or introduce new attack surfaces. This gap leaves stakeholders without evidence-based guidance for secure deployment, despite the critical importance of charging infrastructure security for financial integrity, user privacy, and grid stability as EV adoption continues to accelerate worldwide.

## 1.2 Problem Statement

Despite the rapid global expansion of Electric Vehicle (EV) charging infrastructure and the widespread adoption of the Open Charge Point Protocol (OCPP) as the de facto communication standard, significant uncertainty remains regarding the security and resilience of its most recent release, OCPP 2.1. Previous studies and frameworks have primarily focused on older versions such as OCPP 1.6, 2.0, and 2.0.1 leaving the latest protocol iteration largely unexamined under realistic adversarial conditions [4, 6, 15]. This gap in systematic evaluation undermines confidence in whether the enhanced security mechanisms introduced in OCPP 2.1 such as stronger authentication, improved encryption, and new security profiles effectively mitigate known vulnerabilities or inadvertently introduce new ones.

Existing testing frameworks present further challenges. Tools like OCPPStorm provide partial or outdated support, while more recent emulation frameworks such as EmuOCPP are limited to OCPP 2.0.1 and a small subset of protocol messages [1, 2]. As a result, researchers and industry practitioners lack a unified, reproducible, and extensible environment for assessing the security posture of OCPP 2.1-based systems. This limitation not only constrains academic inquiry but also leaves vendors and operators without clear, evidence-based guidance for deploying secure charging networks.

Therefore, the core research problem addressed by this study can be stated as follows:

> *There is no comprehensive, emulation-based framework or empirical evaluation that systematically assesses the security of OCPP 2.1, leaving a critical gap in understanding whether its enhanced security features effectively mitigate previously identified vulnerabilities and ensuring interoperability and resilience within EV charging ecosystems.*

This research seeks to fill that gap by replicating prior findings from OCPP 2.0.1 studies, extending emulation-based testing to the OCPP 2.1 protocol, and performing a comparative security analysis. Through this approach, the study aims to generate validated insights into OCPP 2.1s security posture, identify persisting or new vulnerabilities, and propose mitigation strategies and best-practice guidelines to support the secure deployment of modern EV charging infrastructure worldwide.

## 1.3 Software Requirements Specification

This Software Requirements Specification (SRS) outlines the functional requirements for extending the *EmuOCPP* framework to support *OCPP 2.1*. The objective of this enhance-

ment is to incorporate new protocol features, including certificate management, authorization flows, network provisioning, and resumable transactions, while maintaining schema consistency and backward compatibility with OCPP 2.0.1 [15]. These requirements define the necessary capabilities for implementing, testing, and validating the upgraded emulator.

### 1.3.1 Scope and Purpose

- Implement the selected OCPP 2.1 features: M01, A02/A03, A05, B09, B13, IdToken and authorization updates, C07/C08, C10, E16, and E17.
- Maintain operational compatibility with OCPP 1.6 and 2.0.1 sessions.
- Provide validation of new message exchanges and behaviors through CheckOCPP.

### 1.3.2 Functional Requirements

The functional requirements are grouped according to feature modules introduced in the improved framework.

**Module 1: Certificate Management (M01, A02/A03)**

1. The system shall update the certificate-signing workflow to include the `requestId` attribute in `SignCertificateRequest`, enabling response pairing with `CertificateSignedRequest`.
2. The system shall support ISO 15118-20 certificates to enable secure plug-and-charge authorization.

**Module 2: Security Profile Configuration (A05)**

1. The system shall allow controlled downgrading from Security Profile 3 to Security Profile 2 to support variable-security testing scenarios.

**Module 3: Network Provisioning (B09)**

1. The system shall extend `SetNetworkProfileRequest` to include the parameters `basicAuthPassword` and `identity`.
2. All provisioning parameters shall be processed within the emulator to mimic real-world CSMS network configuration flows.

**Module 4: Reset-Recovery and State Retention (B13)**

1. The system shall preserve transaction state across system resets so that charging sessions may resume after restoration.
2. The emulator shall persist session identifiers and replay state upon reinitialization.

**Module 5: Authorization Handling**

1. The system shall accept `IdToken` values up to 255 characters in length.
2. `IdTokenType` shall be treated as a dynamic value list, allowing new token classifications without schema modification.

**Module 6: ISO 15118 Plug-and-Charge Flows (C07/C08)**

1. The system shall implement ISO 15118-20 authorization sequences to support plug-and-charge functionality.
2. The emulator shall simulate both EV and CSMS endpoints to validate message exchange behavior.

**Module 7: Authorization Cache Validation (C10)**

1. The system shall enforce explicit expiration policies for cached authorization entries.
2. Authorization tokens shall expire following the validity interval defined by OCPP 2.1.

**Module 8: Transaction Costing and Limits (E16)**

1. The system shall support transactions containing cost, time, energy, and State-of-Charge (SoC) limits.
2. The emulator shall report these limit values back to the CSMS as part of the session summary.

**Module 9: Resumable Transactions (E17)**

1. The system shall allow charging transactions to resume after reboot or network interruption.
2. Session identifiers, meter values, and related state data shall persist to ensure continuity of the transaction.

### 1.3.3 Non-Functional Requirements

The following non-functional requirements define quality attributes expected from the enhanced EmuOCPP framework. These requirements ensure maintainability, reliability, performance consistency, and reproducibility of experimental outcomes during OCPP 2.1 security evaluation.

- **Backward Compatibility:** The framework shall retain full operational compatibility with OCPP 1.6 and OCPP 2.0.1 without requiring configuration or behavioural changes.
- **Performance:** The framework shall maintain end-to-end message handling latency within ś5
- **Maintainability:** The implementation shall conform to the existing modular Python architecture, allowing future OCPP protocol updates to be integrated with minimal code restructuring.
- **Reproducibility:** Test executions shall produce deterministic outcomes when repeated under identical network conditions, emulator configuration, and execution state.

### 1.3.4 Constraints

- **Platform:** Ubuntu 20.04 or newer with Python 3.10 and FRRouting (IPMininet).
- **Dependencies:** Wireshark 4.2 (CheckOCPP) and OpenSSL 3.0 for certificate validation.
- **Scope Limitation:** Physical EVSE hardware control, V2X, DER, and billing processes are out of scope.

### 1.3.5 Verification and Success Criteria

- All selected OCPP 2.1 features (M01, A02/A03, A05, B09, B13, C07/C08, C10, E16, E17) execute successfully in EmuOCPP.
- New message definitions and schema updates validate correctly using CheckOCPP.
- No behavioral malfunctions occur in OCPP 1.6 or 2.0.1 regression tests.
- Message latency deviation remains within ś5% of baseline performance.

### 1.3.6 Summary

This SRS defines the minimal yet critical OCPP 2.1 enhancements required for the SafeCharge research framework. By incorporating updates to certificate handling, authorization, network provisioning, and transaction management, the re-engineered EmuOCPP shall deliver accurate emulation of OCPP 2.1 behavior while retaining full backward compatibility with prior protocol versions

## 1.4 Work Distribution

This project was carried out through clearly defined task allocation among the three team members, ensuring balanced contribution and steady progression across research, implementation, experimentation, and documentation phases.

### Adnan Khaliq

Adnan was primarily responsible for the environment setup, software deployment, and theoretical foundation of the framework. He led the installation, configuration, and complete deployment of the EmuOCPP environment, including IPMininet, CheckOCPP, FRRouting, and all supporting Python dependencies. His work involved establishing a functional OCPP testbed, configuring virtual network components, and ensuring reliable communication between emulated Charge Points and the CSMS. Emulator upgrade and backwards compatibility by modeling using the OCPP spec were under him.

In addition, he contributed major portions of the theoretical writing regarding EmuOCPP, including architectural components, security profiles, certificate management, emulator configuration layers, and operational services. Making sure the message flows used v 2.1 if set up. Ensuring previous versions did not break. His duties included:

- System design, emulator configuration, and testing environment setup
- Software installation and deployment of required libraries and tools
- Theoretical writing and documentation of the EmuOCPP framework
- Research on OCPP protocol structure and security profiles
- Created OCPP shim as a compatibility layer for the spec bump.

## Muhammad Jibran Hanan

Jibran contributed primarily to attack research, packet analysis, and reproduction of the Man-in-the-Middle (MitM) attack scenarios. He successfully replicated attacks M1–M4 within the EmuOCPP environment and validated protocol weaknesses under different security profiles. Packet inspection was conducted using Wireshark dissectors, where Jibran documented message interception, packet structure behavior, and protocol-level reaction patterns.

He identified a new type of DoS attack that may be done with the new message type, SEND, introduced in v 2.1.

He also developed detailed attack algorithms and theoretical explanations of message parsing, protocol inspection, and downgrade attacks. He made sure that the attack scenarios were simulated properly on the new version as well. He worked along side Kumail Haider to mitigate the DoS attacks aswell. His contributions included:

- Literature review focusing on MitM attacks and packet interception
- Packet capture, Wireshark dissector configuration, and protocol message analysis
- Execution of MitM-based attack scenarios and documentation of results
- Algorithm development for theoretical and experimental sections
- Verified M1-M4 attack scenarios on updated emulator

## Kumail Haider

Kumail was responsible for developing, executing, and documenting impersonation-based attack scenarios, specifically I1 and I2. He implemented identity spoofing, reboot-window exploitation, duplicate serial identity testing, and policy-based testing using server configuration variations. He also performed comparative evaluation of attack outcomes across both controlled and modified server policies.

Worked on new SEND DoS attack alongside Jibran Haider. He also documented attacks as verification. Mitigation for all DoS attacks were done with his assistance.

Kumail developed pseudocode and attack logic, completed repeated experimental runs, and produced analytical summaries based on evaluated system behavior. His major duties consisted of:

- Implementation and testing of I1 and I2 impersonation attacks
- Algorithm development and experimental validation

- Comparative analysis of attack behaviors, server policies, and vulnerability exposure
- Documentation of experimental outcomes and discussion of findings
- Validated attack scenarios I1 and I2 on upgrade simulator.

## Collaborative Responsibilities

All three members collaborated on research documentation, technical writing, and presentation preparation. The entire team participated in:

- Report writing, proof-reading, and formatting
- Literature review integration and SRS development
- Presentation design and analytical discussion of results
- Comparative analysis of findings and final research conclusions

## Summary

In conclusion, the work was distributed evenly to leverage each member's strengths. Adnan led the theoretical understanding and system configuration, Jibran handled packet-level MitM attack replication and traffic inspection, and Kumail executed impersonation attacks and comparative evaluation. All members jointly prepared the technical report and presentation to ensure coherence and consistency across all deliverables.

# Chapter 2

# Literature Review

The foundation for establishing the context, necessity, and novelty of a research project is a comprehensive review of the existing scholarly literature. This research, targeting the enhancement of cybersecurity for Electric Vehicle Charging Stations (EVCS) that employ the Open Charge Point Protocol (OCPP), necessitates a critical analysis of current research. This review surveys state-of-the-art works from the past two years, focusing specifically on studies that identify OCPP vulnerabilities, propose testing methodologies, and suggest defensive frameworks. The aim is to summarize key findings, critically evaluate the research designs and conclusions, and thereby identify a significant gap in the literature that this project intends to address. The subsequent sections present a detailed analysis of relevant sources, a synthesized tabular overview, and a conclusion that defines the research niche for this work.

## 2.1 Related Research

This research project will use "EmuOCPP: Effective and Scalable OCPP Security and Privacy Testing" by [1] as its foundational base paper for emulation and testing methodology. The literature review is built upon six other key studies: the foundational vulnerability analysis by [5]; the exploit demonstration work by [4]; the fuzzing tool OCPPStorm by [2]; the analysis of stealthy Charge Manipulation Attacks by [8]; the Federated Learning-based IDS proposed by [3]; and the protocol hardening framework by [6]. While these works provide critical insights into OCPP 1.6 and 2.0.1 vulnerabilities and defenses, our project will directly extend the EmuOCPP framework to address the gap in testing the latest OCPP 2.1 standard.

### 2.1.1 Cybersecurity Risk Analysis of Electric Vehicles Charging Stations

#### 2.1.1.1 Article Summary

This paper [5] delivers a comprehensive cybersecurity risk analysis of the Electric Vehicle Charging Station (EVCS) ecosystem, offering both a broad survey and an empirical data-driven assessment. It begins by outlining the rapid expansion of EV infrastructure and associated cybersecurity challenges due to the complex interaction of vehicles, grid networks, and communication protocols. The authors systematically examine infrastructure centric and protocol centric vulnerabilities, focusing on weak points in communication pathways, authentication mechanisms, and maintenance interfaces. The study particularly scrutinizes commonly used protocols such as OCPP 1.6 and below, ISO 15118, and IEC standards, exposing issues like unencrypted or optional Transport Layer Security (TLS), weak identity management, and inconsistent implementations that enable threats such as False Data Injection (FDI), Man-in-the-Middle (MitM), Denial-of-Service (DoS), and malware injection attacks.

To reinforce theoretical discussions, the authors employ real-world data from EV charging sessions (e.g., ElaadNL datasets) to identify anomalous patterns including abnormal connection durations and irregular power draws, which can signal malicious activity. By mapping these anomalies to potential attack vectors, the study provides a bridge between conceptual vulnerabilities and their operational manifestations. Ultimately, the paper highlights the necessity for standardized communication protocols, stronger encryption, and continuous monitoring systems to secure the EVCS ecosystem.

#### 2.1.1.2 Proposed Techniques and Results

This paper employs a dual-layered analytical approach, integrating **(1)** a conceptual threat taxonomy based on infrastructure and communication interfaces, and **(2)** a data-centric validation using real-world EVCS datasets. The study identifies four major points of vulnerability - EV connectors, user terminals, Internet connectivity, and maintenance interfaces - and maps them to corresponding attack scenarios. This classification helps in understanding how breaches can occur across different operational layers, from firmware manipulation to protocol exploitation.

The results indicate that optional or misconfigured TLS layers, weak firmware update mechanisms, and insecure user authentication remain dominant risk factors in OCPP 1.6 and related protocols. Furthermore, through the real-time analysis of charging session logs, the study observes irregular session durations and power consumption patterns consistent with potential DoS and FDI attacks. While the paper does not propose new

mitigation frameworks, it effectively quantifies the scope of these vulnerabilities and contextualizes them within the broader cybersecurity landscape of smart grids and electric mobility.

#### 2.1.1.3 Critical analysis

The paper demonstrates strong foundational value through its combination of a broad literature review and empirical assessment using real-world charging session data. It effectively highlights major weaknesses in EVCS communication protocols, particularly in areas such as authentication, encryption, and implementation consistency. This synthesis provides a useful starting point for identifying recurring vulnerabilities in OCPP-based infrastructures.

However, the studys methodology remains primarily descriptive and observational, with limited experimental validation or reproducible testing. The lack of a controlled emulation environment or adversarial simulations means that identified anomalies cannot be conclusively linked to real attack behavior. Moreover, the research focuses on outdated protocol versions (OCPP 1.6 and below), without assessing whether newer iterations, such as OCPP 2.0.1 and 2.1, resolve or retain similar weaknesses. Finally, while the paper emphasizes the need for stronger security standards, it does not propose or test concrete mitigation strategies.

In summary, the study succeeds in mapping the EVCS threat landscape but falls short in empirical depth and practical validation, gaps that SafeCharge directly addresses through emulation-based testing, version comparison, and vulnerability verification across OCPP iterations.

#### 2.1.1.4 Relationship to the proposed framework

The work directly informs and underpins the objectives of the SafeCharge research framework. The paper establishes a foundational understanding of cybersecurity challenges within the EVCS ecosystem, identifying recurring weaknesses such as unencrypted communication, weak authentication mechanisms, and inconsistent adoption of security measures in OCPP 1.6 and earlier versions. While the study effectively defines the problem space, it lacks empirical validation and does not evaluate whether these vulnerabilities persist in newer protocol versions.

SafeCharge builds upon these insights by transitioning from a descriptive, survey-based approach to an experimental and comparative investigation. It adopts an emulation-driven methodology to recreate and validate the vulnerabilities extending the analysis to OCPP 2.1 to assess their security evolution. Furthermore, SafeCharge aims to develop and test

practical mitigation strategies that address the weaknesses outlined in the study, transforming conceptual observations into verifiable defensive solutions.

In essence, the research provides the theoretical foundation and motivation for SafeCharges work, defining the critical security gaps that this project seeks to empirically evaluate, validate, and mitigate within the context of modern OCPP implementations.

### 2.1.2 Charge Manipulation Attacks Against Smart Electric Vehicle Charging Stations and Deep Learning-based Detection Mechanisms

#### 2.1.2.1 Article Summary

This paper [8] presents a comprehensive investigation into Charge Manipulation Attacks (CMAs), a category of cyber threats that exploit vulnerabilities in OCPP-based Electric Vehicle Charging Stations (EVCS). It models the hierarchical communication between electric vehicles, charging stations, and central servers to demonstrate how adversaries can alter charging schedules by manipulating messages such as *SetChargingProfile*. Through large-scale simulations involving thousands of virtual charging points, the study evaluates how these manipulations can disrupt grid stability, distort energy demand, and influence market pricing. To mitigate such threats, the authors propose a deep learningbased detection framework employing convolutional and recurrent neural networks. These models analyze temporal charging data to detect anomalies indicative of manipulation. Experimental results show high detection accuracy, demonstrating that AI-driven models can effectively identify subtle deviations in normal charging behavior, providing a significant advancement in proactive EVCS cybersecurity.

#### 2.1.2.2 Proposed Techniques and Results

The study utilizes a two-tiered methodology, attack modeling and AI-based detection. In the first stage, six distinct CMA variants are designed to exploit weaknesses in OCPPs message structure and scheduling commands, allowing attackers to modify power levels and charge timings. These simulated attacks are executed in a virtual smart grid environment to evaluate their impact on operational efficiency and energy market stability. The results reveal that targeted manipulations can trigger notable grid instability, revenue imbalance, and performance degradation across the charging network. In the second stage, a deep learning-based intrusion detection system is developed using temporal data from charging sessions. The framework combines convolutional layers for spatial feature extraction with recurrent layers for sequential pattern recognition, achieving high precision in identifying CMAs across different scenarios. The findings validate that deep models

can effectively distinguish between normal and manipulated operations, reinforcing the role of AI-driven analytics in safeguarding EVCS infrastructures.

#### 2.1.2.3 Critical analysis

This study on charge manipulation attacks (CMAs) against OCPP-based Electric Vehicle Charging Stations (EVCSs), combining large-scale simulations with deep learning-based detection. The papers key strength lies in bridging protocol-level vulnerabilities with their operational and economic effects. By modeling the hierarchical communication between Electric Vehicles, Charging Stations, and central servers, the authors demonstrate how malicious alterations to the `SetChargingProfile` function can distort power scheduling, increase demand volatility, and trigger financial penalties in energy markets. Their use of multiple CMA scenarios across thousands of simulated charging points adds credibility and scale to the analysis, while the inclusion of a convolutional and recurrent neural network-based detector highlights how temporal charging data can reveal subtle manipulations that rule-based systems might miss.

Despite its strong experimental design, the studys scope remains limited to OCPP 1.6 with unencrypted or partially protected communications, leaving the applicability of its results to newer, certificate-based versions uncertain. The simulations rely on a controlled digital environment without validating latency, resilience, or noise tolerance in physical deployments. While the detection models report high accuracy, the absence of detailed false-positive analysis and robustness testing against adaptive adversaries constrains real-world reliability. Moreover, the frameworks centralized aggregator could itself represent a single point of failure, and the use of deep models without explainability limits transparency for operational decision-making.

Nonetheless, the work substantially advances EVCS cybersecurity by empirically linking cyberattacks to measurable grid and market disruptions. It validates deep learning as an effective analytical tool for identifying data-driven anomalies in EV charging systems and establishes a critical foundation for evaluating OCPP vulnerabilities at the system level a foundation upon which subsequent research, including SafeCharge, can build.

#### 2.1.2.4 Relationship to the proposed framework

The study provides a critical foundation for the SafeCharge project by illustrating how OCPP level message manipulation can directly influence grid stability and energy market dynamics. Their identification of vulnerabilities in the `SetChargingProfile` function and hierarchical charging control highlights structural weaknesses that persist in modern EVCS ecosystems. While their framework successfully demonstrates the feasibility and impact of Charge Manipulation Attacks, it is confined to OCPP 1.6 environments

with limited security enforcement. This limitation leaves unexplored whether the newer OCPP 2.0.1 and 2.1 security profiles incorporating TLS and JSON Web Signature authentication effectively mitigate such attack vectors.

SafeCharge builds upon this work by expanding the evaluation scope to these later protocol versions and by emphasizing real time, cross protocol vulnerability testing. Whereas Jahangir et al. focus primarily on detection, SafeCharge integrates detection with automated validation and mitigation mechanisms, allowing the comparison of security posture across protocol generations. The deep learning approach proposed in their work informs SafeCharges analytical layer, but the proposed project aims to extend this with explainable and adaptive models that address interpretability and evolving attack behavior. In doing so, SafeCharge seeks to bridge the identified gaps between theoretical detection performance and operational cybersecurity assurance in modern EVCS networks.

### 2.1.3 Disrupting EV Charging Sessions and Gaining Remote Code Execution with DoS, MITM, and Code Injection Exploits using OCPP 1.6

#### 2.1.3.1 Article Summary

This study [4] investigates critical vulnerabilities within OCPP 1.6j by conducting live attack simulations in both virtualized and physical testing environments, including an operational EV charging laboratory. The researchers execute five proof-of-concept exploits, comprising Log4Shell-based Man-in-the-Middle (MitM) interception, session termination, malicious firmware injection, SYN-flood Denial-of-Service (DoS), and message-handling exploitation, demonstrating how unvalidated firmware updates, weak authentication, and unsecured channels can compromise Electric Vehicle Charging Station (EVCS) operations. Each attack produces measurable operational disruptions, from session halts to full remote code execution. The study concludes with practical recommendations for improving legacy OCPP deployments, highlighting network-level mitigations such as SSH tunneling, stronger firewall enforcement, and network segmentation as effective stopgap defenses.

#### 2.1.3.2 Proposed Techniques and Results

The paper employs a hybrid experimental approach using both simulated network infrastructures and physical EVCS hardware to replicate real-world communication between charging stations and central systems. The attacks are systematically executed within these testbeds to quantify their impact on system availability and integrity. Results reveal consistent exploitation pathways through insecure firmware update mechanisms, plain-

text communication channels, and inadequate identity verification processes. The introduction of SSH tunneling and firewall-based isolation is empirically validated to reduce the success rate of MitM and injection attacks significantly, though such measures offer limited scalability. Collectively, the experiments confirm that even mature OCPP 1.6 implementations remain vulnerable to targeted exploitation, with the potential for complete operational compromise.

#### 2.1.3.3 Critical analysis

The primary strength of this work lies in its empirical, impact-oriented design. By executing live attacks instead of theoretical simulations, the research provides direct evidence of how weak authentication, unencrypted channels, and lax firmware validation in OCPP 1.6 translate into real operational threats. This practical orientation bridges a critical gap in EVCS security literature, which previously focused mostly on threat enumeration without proof-of-concept validation. The dual test environments enhance the studys ecological validity by proving attack feasibility under both laboratory and production-grade conditions. Furthermore, its inclusion of mitigation strategies such as SSH tunneling illustrates a pragmatic understanding of legacy system constraints.

Nevertheless, the research has several limitations. While the suggested mitigations improve confidentiality and access control, they remain stop-gap measures that do not address core systemic issues such as firmware integrity validation, certificate management, or scalability against distributed DoS attacks. The attack scenarios assume the adversarys presence within the local network, neglecting remote cloud based attack vectors that are increasingly relevant in modern OCPP deployments. Additionally, the research does not examine how the identified vulnerabilities persist or evolve in subsequent protocol versions (OCPP 2.0.1 and 2.1), limiting its forward applicability.

Overall, the study stands out as a landmark in empirical validation of EVCS cybersecurity risks, but its limited scope and lack of structured threat prioritization underscore the need for systematic evaluation of newer OCPP standards.

#### 2.1.3.4 Relationship to the proposed framework

The findings of this study provide a critical empirical foundation for the SafeCharge project by establishing the real-world feasibility and impact of OCPP-based exploits. While the original research focuses on OCPP 1.6, SafeCharge extends this exploration to newer protocol versions, specifically OCPP 2.0.1 and 2.1, to determine whether enhanced security profiles effectively mitigate the demonstrated vulnerabilities. By integrating an emulation-based testing framework, SafeCharge builds upon these findings to replicate and evaluate attack vectors such as MitM, code injection, and DoS under updated proto-

col conditions. Furthermore, the emphasis on validating practical defenses in controlled environments directly aligns with SafeCharges objective of developing a comparative, evidence-driven assessment of OCPP security evolution. Ultimately, this earlier work acts as a proof-of-concept baseline that SafeCharge expands into a broader, version-inclusive investigation of EVCS protocol resilience and mitigation efficacy.

### 2.1.4 Federated Learning-Based Intrusion Detection for OCPP Cyberattacks

#### 2.1.4.1 Article Summary

This study [3] introduces a federated learning-based intrusion detection framework designed to enhance cybersecurity within OCPP-based Electric Vehicle Charging Station (EVCS) networks. The framework utilizes distributed machine learning to detect malicious activity without centralizing user data, ensuring privacy preservation across multiple EVCS nodes. Network flow features are captured using tools such as OCPPFlowMeter and CICFlowMeter, which extract statistical metrics including packet duration, inter-arrival times, and message size distributions. These features form the input dataset for training and evaluating the detection models. The research simulates three distinct attack types, Charging Profile Manipulation, Denial of Charge, and Heartbeat Flooding, across both virtual and physical test environments comprising a SteVe OCPP server and a 22 kW AC wallbox charger. Experimental evaluation shows that the federated model achieves detection accuracy comparable to centralized intrusion detection systems (IDS) while maintaining compliance with data privacy constraints. The frameworks ability to aggregate insights across multiple distributed charging points demonstrates improved global convergence and adaptability to heterogeneous network conditions.

#### 2.1.4.2 Proposed Techniques and Results

The methodological approach combines feature-based traffic analysis with federated deep learning to construct a collaborative yet privacy-preserving intrusion detection ecosystem. Each participating EVCS instance locally trains an anomaly detection model on flow-level OCPP data, with only model parameters shared for aggregation. The global model then updates iteratively to enhance detection performance across the distributed network. Experimental testing validates the frameworks effectiveness in identifying targeted OCPP-level attacks. The Charging Profile Manipulation scenario revealed how energy delivery parameters could be altered through False Data Injection, while Denial of Charge and Heartbeat Flooding attacks disrupted normal service availability. The federated aggregation process demonstrated consistent accuracy gains over standalone models

and improved robustness to unbalanced or non-identical data distributions. These results confirm the feasibility of deploying AI-driven, decentralized cybersecurity mechanisms in real-world EVCS networks.

#### 2.1.4.3 Critical analysis

The work stands out for integrating artificial intelligence with modern privacy-preserving architectures in the context of EVCS cybersecurity. The use of federated learning to enhance detection accuracy without data centralization represents a significant step forward, especially under increasing regulatory emphasis on data protection (e.g., GDPR). The experimental design is robust, employing both virtualized and physical hardware components, thereby validating the practical deployability of the framework. The adoption of flow-based feature extraction tools (CICFlowMeter and OCPPFlowMeter) further strengthens the studys methodological soundness, as it avoids deep packet inspection and thus mitigates privacy and performance concerns.

However, the research scope and design also present notable limitations. The study focuses exclusively on three attack categories Charging Profile Manipulation, Denial of Charge, and Heartbeat Flooding thereby neglecting other critical vectors such as certificate manipulation, firmware tampering, replay attacks, and session hijacking, which are highly relevant to modern OCPP implementations. The federated approach, while conceptually strong, introduces new challenges including communication overhead, vulnerability to model poisoning attacks, and potential inconsistencies in local model training due to data heterogeneity. These issues are acknowledged by the authors but not experimentally explored. Furthermore, the study is confined to OCPP 1.6 and does not evaluate compatibility or resilience under OCPP 2.0.1 or 2.1, where encryption, authentication, and security profiles are significantly improved. The lack of a comparative baseline against these newer standards limits the generalizability of the findings.

Despite these limitations, the research successfully establishes a foundation for distributed, AI-driven intrusion detection in EVCS environments. Its key strength lies in its forward-looking design, which can be extended to more advanced versions of OCPP with relatively minor architectural adjustments.

#### 2.1.4.4 Relationship to the proposed framework

This research directly informs the SafeCharge project by exemplifying how distributed intelligence can enhance resilience across interconnected EVCS networks. The federated approachs capacity for decentralized learning inspires SafeCharges design philosophy, emphasizing modular and adaptive security mechanisms that function across diverse protocol implementations. While the original study centers on detection within OCPP 1.6,

SafeCharge extends this principle to validation and verification within OCPP 2.1, focusing on whether improved protocol structures mitigate similar classes of attacks. Furthermore, SafeCharge builds on this foundation by incorporating emulation-based adversarial testing to evaluate real-world exploit scenarios beyond the limited attack spectrum analyzed in the federated framework. The project also explores integration of hybrid intelligence layers, combining traditional flow analysis with interpretable machine learning, to achieve transparency and operational reliability. In doing so, SafeCharge advances the federated learning concept from anomaly detection toward comprehensive security evaluation, ensuring that the latest OCPP standards are both empirically validated and resilient against evolving cyber threats.

### 2.1.5 Cyber Defense in OCPP for EV Charging Security Risks

#### 2.1.5.1 Article Summary

This study [6] introduces a structured cybersecurity framework aimed at enhancing the security posture of Electric Vehicle Charging Stations (EVCS) operating under OCPP. The proposed multi-phase model, Identify, Analyze, Assess, Mitigate (IAAM), integrates layered security assessment and response mechanisms. The approach combines STRIDE for systematic threat identification, Attack Trees for visualizing exploitation paths, and DREAD for quantifying risk impact and prioritization. The framework is implemented within a simulated OCPP 1.6 environment utilizing Node.js-based servers and Docker containers to emulate both charge points and central systems. Through comparative testing between baseline and enhanced OCPP deployments, the study demonstrates tangible security improvements. These include reductions in unauthorized session initiations, elimination of duplicate transactions, and improved resistance to communication tampering. Mitigation techniques such as one-time password (OTP) authentication, strengthened session management, and transaction validation mechanisms collectively contribute to an enhanced protocol-level defense strategy.

#### 2.1.5.2 Proposed Techniques and Results

The IAAM framework operationalizes cybersecurity defense through four core phases: threat identification, vulnerability analysis, risk assessment, and mitigation deployment. Using the STRIDE and DREAD methodologies, the study classifies OCPP attack vectors into categories such as spoofing, tampering, and denial-of-service, assigning severity scores to each. The experimental testbed replicates an OCPP 1.6 environment, where enhanced security mechanisms, primarily OTP-based session authentication and refined transaction verificationare introduced directly into the protocol logic. Experimental results reveal substantial improvement in session integrity and transaction authenticity com-

pared to the standard OCPP implementation. The improved system successfully mitigated phantom charging sessions and inconsistent meter readings while maintaining operational reliability. These findings validate that strategic, software-level security integration within OCPP message exchanges can effectively strengthen communication resilience without hardware replacement or significant system redesign.

#### 2.1.5.3 Critical analysis

A core strength of this research lies in its practical, solution-oriented design. The IAAM framework provides a repeatable and modular structure for cybersecurity assessment, aligning conceptual threat modeling with directly implementable countermeasures. By integrating STRIDE, DREAD, and Attack Tree methodologies, the study offers a coherent approach to both identifying and prioritizing OCPP-specific risks. The validation through experimental log analysis further enhances the credibility of the proposed mitigations, as the results demonstrate measurable improvements in communication integrity and session control.

However, the papers scope is deliberately constrained. It primarily focuses on mitigating session-level threats (corresponding to vulnerabilities A3A6 in the OCPP threat landscape) while deferring issues related to transport layer security (A1, A2) and firmware update integrity (A7) for future work. Additionally, the use of OTP generation and verification, though effective in enhancing session authentication, introduces additional computational overhead that is not rigorously tested on resource-constrained EV chargers. The validation is limited to virtualized environments, leaving questions regarding latency, real-world performance, and interoperability with production-grade charging stations. Despite these limitations, the studys empirical grounding and reproducibility make it a valuable contribution toward practical OCPP protocol hardening.

#### 2.1.5.4 Relationship to the proposed framework

This study provides a conceptual and methodological foundation that directly informs the SafeCharge project. Its structured IAAM workflow aligns closely with SafeCharges phased research strategy, particularly in transitioning from vulnerability identification to mitigation validation. The integration of systematic modeling techniques such as STRIDE and DREAD offers a clear template for risk assessment that SafeCharge extends into the OCPP 2.1 domain. While the study focuses on OCPP 1.6, SafeCharge broadens the framework to encompass newer security mechanisms, including certificate-based mutual authentication, advanced encryption profiles, and enhanced message integrity validation. Furthermore, the emphasis on protocol-level mitigation serves as a foundation for SafeCharges goal of evaluating cross-version protocol resilience through emulation

and comparative testing. In essence, this earlier work establishes a field-tested methodology that SafeCharge evolves into a more comprehensive, empirical, and version-agnostic evaluation of OCPP cybersecurity.

### 2.1.6 OCPPStorm: A Comprehensive Fuzzing Tool for OCPP Implementations

#### 2.1.6.1 Article Summary

This study [2] introduces OCPPStorm, a scalable and systematic fuzzing framework designed for automated vulnerability discovery in OCPP-based Electric Vehicle Charging Station (EVCS) systems. The framework performs black-box testing by injecting malformed, incomplete, and out-of-order JSON messages into the communication channel between charge points and central management systems. It employs three distinct fuzzing engines, random mutation, grammar-based, and state-machine-driven fuzzers, to ensure comprehensive coverage of both syntactic and logical protocol behaviors. Experimental evaluations conducted on widely used open-source implementations such as SteVe and OCPP.Core reveal critical vulnerabilities, including denial-of-service triggers, improper field validation, reused message identifiers, and faulty error handling. In total, six CVEs are validated, confirming that even mature OCPP 1.6 implementations lack consistent input validation and robust exception handling. The study emphasizes the role of automated fuzzing in establishing a proactive, continuous testing methodology for protocol security, shifting the research landscape from reactive incident analysis to preventative vulnerability discovery.

#### 2.1.6.2 Proposed Techniques and Results

OCPPStorms design follows a modular testing architecture that automates the injection, monitoring, and evaluation of malformed OCPP messages within live communication flows. The framework operates through three integrated fuzzing modules:

- The Random Fuzzer, which introduces random payload mutations to assess basic protocol robustness.
- The Grammar-based Fuzzer, which uses OCPPs formal JSON schema to generate structurally valid yet semantically disruptive message variations.
- The State-Machine Fuzzer, which tests temporal and contextual dependencies in OCPP exchanges by altering message order and timing.

Testing across multiple OCPP backends results in reproducible identification of protocol inconsistencies, including parsing failures and denial-of-service events caused by malformed payloads. The experiments demonstrate that fuzzing-based testing can reveal latent vulnerabilities even in well-maintained open-source projects. These results establish OCPPStorm as a foundational tool for large-scale automated protocol validation and underscore the value of fuzzing as part of continuous integration pipelines for EVCS software.

#### 2.1.6.3 Critical analysis

This works makes a notable contribution to OCPP cybersecurity by introducing *OCPP-Storm*, one of the first systematic fuzzing frameworks tailored specifically to electric vehicle charging communication protocols. The studys primary strength lies in shifting OCPP security testing from manual penetration attempts to automated, reproducible, and high coverage fuzzing. By combining random, state machine, and grammar-based fuzzers, the framework captures the full range of protocol interactions, uncovering vulnerabilities that stem from both syntactic inconsistencies and state-transition mismanagement. This multi-layered fuzzing approach reflects a mature understanding of OCPPs asynchronous, event-driven architecture where message order, timing, and context strongly influence security posture.

The papers evaluation methodology reinforces its credibility. OCPPStorm is tested on multiple open-source implementations, including SteVe and OCPP. Core, where it successfully identifies six confirmed CVEs and numerous critical flaws such as denial-of-service conditions, malformed JSON parsing, and invalid message ID reuse. These findings confirm that OCPP 1.6 systems remain highly susceptible to malformed traffic and that even mature backends lack comprehensive input validation. The authors also provide a detailed taxonomy of discovered weaknesses, distinguishing between specification ambiguities, implementation oversights, and design-level risks. This structure enhances the interpretability of the results and supports targeted mitigation efforts.

However, several limitations constrain the generalizability of the research. The tools scope is restricted to OCPP 1.6, leaving untested newer versions (2.0.1 and 2.1) that incorporate formal schemas, improved authentication, and enhanced validation mechanisms. Additionally, the frameworks dependence on protocol message coverage metrics rather than semantic validation limits its ability to assess logical correctness or multi-session exploits. While OCPPStorm effectively exposes structural vulnerabilities, it does not measure the runtime impact of discovered bugs on system availability or data integrity. The study also lacks quantitative performance benchmarks for fuzzing scalability, leaving open questions about its efficiency in real-world deployments.

Despite these constraints, OCPPStorm represents a pivotal step toward standardized and

automated OCPP security evaluation. It establishes a reproducible methodology for uncovering implementation-level flaws, setting the foundation for advanced, cross-version fuzzing and vulnerability correlation frameworks such as those envisioned within the *SafeCharge* project.

#### 2.1.6.4 Relationship to the proposed framework

This research provides a practical baseline for automated robustness testing of OCPP implementations through OCPPStorms black-box fuzzing of OCPP 1.6. This directly complements SafeCharge, which extends the scope from single-version fuzzing to a cross-version evaluation of OCPP 1.6, 2.0.1, and 2.1. Whereas OCPPStorm demonstrates that malformed and out-of-sequence messages can trigger denial-of-service and parsing failures in mature backends, SafeCharge leverages these insights to assess whether security profiles and their stricter schemas, actually prevent analogous failure modes.

Methodologically, OCPPStorms mix of random, state-machine, and grammar-based fuzzers informs *SafeCharge*s test generation layer; however, *SafeCharge* integrates fuzzing within a broader pipeline that includes protocol emulation, attack replay, and detection/mitigation validation. In particular, findings from fuzzing are correlated with runtime effects (availability, integrity), surfaced to analytics (e.g., anomaly detectors), and re-tested after applying countermeasures to verify remediation. The goal is to move from vulnerability discovery to actionable, regression-tested hardening guidance.

Finally, while OCPPStorm reports discovered weaknesses and CVEs on open-source backends, *SafeCharge* adds comparative, version aware reporting and risk scoring across implementations and protocol generations. This enables operators to prioritize fixes, verify compliance against schema and state constraints, and track security posture over time. In short, OCPPStorm establishes the automated discovery substrate; *SafeCharge* builds on it to deliver cross-protocol validation, mitigation effectiveness testing, and repeatable hardening workflows for modern EV charging ecosystems.

### 2.1.7 EmuOCPP: Effective and Scalable OCPP Security and Privacy Testing

#### 2.1.7.1 Article Summary

EmuOCPP introduces a comprehensive, scalable, and low-cost framework for systematic testing of the Open Charge Point Protocol (OCPP) across its major versions (1.6, 2.0, 2.0.1). The study [1] addresses a long-standing research gap, particularly the lack of dedicated testing environments for the newer OCPP versions by enabling reproducible, real-world emulation of electric vehicle charging networks. Built using IPMininet and

open-source components, the framework virtualizes both charging stations and management systems through container-based emulation, allowing hundreds of nodes to operate concurrently on commodity hardware.

Through this approach, the framework exposes critical vulnerabilities across design and implementation layers, including man-in-the-middle (MitM), impersonation, downgrade, and denial-of-service (DoS) attacks. Five of these attack classes are novel, demonstrating persistent weaknesses across OCPPs security profiles (SP1SP3). Empirical testing across nine open-source, proprietary, and production OCPP targets validates the feasibility of these exploits under realistic network conditions. Furthermore, the study proposes several countermeasures such as certificate pinning, improved root certificate validation, mandatory downgrade protections, and stricter management of charger identifiers, offering practical insights for standard hardening and compliance testing.

#### 2.1.7.2 Proposed Techniques and Results

The framework achieves realism through container-based emulation rather than traditional virtual machines, reducing resource overhead while maintaining isolated instances of OCPP components. This design enables large-scale experimentation with thousands of emulated charging stations, dynamic topologies, and integrated attacker nodes. EmuOCPP fulfills six core requirements: multi-version compatibility, multi-profile support, realistic host emulation, design- and implementation-level testing, scalability, and reproducibility using open-source and low-cost infrastructure.

Technically, the architecture comprises four integrated modules:

- IPMininet for IPv6-capable network emulation and DNS-based topology management.
- Test module, automating attack execution using tools such as Scapy, Mitmproxy, and the THC toolkit for MitM and DoS simulation.
- Messages module, extending The Mobility House OCPP library to support 17 core, security, and management messages across all protocol versions.
- Config and GUI modules, enabling YAML-driven configuration and visual topology design.

Using this environment, eight attacks were systematically executed five newly identified and three previously known. The novel ones include:

- M2, M3, M4: Exploiting weaknesses in the SP upgrade/downgrade process to perform persistent MitM and forced profile regressions.

- I1, I2: Leveraging undefined behaviors in BootNotification and static charger IDs to impersonate legitimate charge points.

Experimental validation across nine OCPP implementations confirmed the effectiveness of all attacks except one variant of I1, demonstrating that vulnerabilities persist across versions and security profiles. Stress testing showed linear scalability with up to 150 concurrent charging points consuming only 6 GB RAM, confirming the emulators efficiency.

The authors also proposed effective countermeasures, emphasizing secure upgrade enforcement, removal of legacy profiles, strict CSID whitelisting, DoS rate-limiting, and mutual authentication. Collectively, these findings establish EmuOCPP as a benchmark testbed for evaluating protocol-level and implementation-specific security gaps in modern EV charging infrastructures.

#### 2.1.7.3 Critical analysis

The EmuOCPP study represents a decisive advancement in the domain of electric vehicle (EV) charging cybersecurity, addressing a long-standing limitation in Open Charge Point Protocol (OCPP) testing. Its methodological foundation, emulation-based security and privacy assessment, achieves a level of scalability and realism that earlier simulators such as SteVe, OCPPStorm, or SAP e-Mobility lacked. By virtualizing both the Charge Point and Central Management System (CSMS) components, the framework enables bidirectional analysis of the communication channel, thereby providing visibility into vulnerabilities that emerge across protocol handshakes, authentication routines, and message exchanges.

A notable strength lies in EmuOCPPs architectural integration of multi-version (1.6, 2.0, 2.0.1) and multi-profile (SP1SP3) compatibility. This breadth allows comparative testing across protocol evolutions, revealing that despite the formal inclusion of security profiles in newer versions, core weaknesses such as insecure downgrade handling, weak charger identity validation, and exploitable session transitions remain exploitable. The inclusion of both open-source and production-grade implementations in testing environments enhances the external validity of these findings and confirms that the vulnerabilities transcend vendor-specific implementations. Furthermore, the empirical rigor of the study is supported by reproducibility: all attacks were demonstrated within a containerized, resource-efficient testbed capable of scaling beyond 150 concurrent charge points with linear performance degradation. This combination of fidelity, control, and scalability establishes EmuOCPP as a credible reference model for experimental cybersecurity research.

Nevertheless, several methodological and practical limitations moderate its impact. First, while the framework successfully demonstrates attack feasibility, its mitigation recom-

mendations are conceptual and remain unvalidated within the same environment, creating a gap between vulnerability discovery and defense evaluation. Second, the implementation coverage is incomplete: only seventeen core OCPP 2.0.1 messages were implemented, leaving several peripheral message categories, transaction workflows, and firmware-related exchanges untested. Third, the evaluation scope ends at OCPP 2.0.1, excluding version 2.1, the most recent revision introducing extended security profile updates and new message schemas. This restricts the frameworks relevance to contemporary deployments unless updated.

From a technical perspective, container-based emulation, while efficient, does not fully replicate the physical latency, timing irregularities, and hardware-induced noise that can influence real-world EVCS-to-CSMS interactions. Moreover, the studys emphasis remains predominantly on handshake-phase and network-layer exploits, with limited exploration of post-authentication data manipulation or session-layer persistence attacks. These omissions highlight potential avenues for deeper behavioral and longitudinal security analysis that could complement the emulators strong foundational design.

In summary, EmuOCPP constitutes a pivotal step toward standardizing OCPP security testing through an open, scalable, and reproducible framework. Its contributions lie not only in unveiling new attack classes and empirically confirming protocol-level weaknesses but also in establishing a methodological baseline that future works can extend toward mitigation validation and version-aware comparative security evaluation. Despite its limitations in coverage and experimental mitigation, the study remains a cornerstone for advancing secure and trustworthy EV charging communication ecosystems.

#### 2.1.7.4 Relationship to the proposed framework

The findings and methodological foundation of this study directly inform the development of the proposed SafeCharge framework. EmuOCPP validates the persistent security weaknesses within OCPP 2.0.1 environments, particularly in authentication, upgrade logic, and charger identity handling, demonstrating that protocol-level protections are insufficient without comprehensive testing and mitigation validation.

Building upon these insights, SafeCharge extends the EmuOCPP paradigm by integrating comparative threat evaluation, regression testing, and automated mitigation validation across OCPP versions 2.0.1 and 2.1. Initially, EmuOCPPs environment serves as the experimental foundation for replicating and verifying previously documented adversarial scenarios. Subsequently, SafeCharge expands this environment to support 2.1-specific features, message schemas, and updated security profiles, enabling direct evaluation of the standards evolution.

Where EmuOCPP concludes with identifying and describing vulnerabilities, SafeCharge

transitions toward mitigation quantification and validationtesting whether the enhancements in OCPP 2.1 effectively neutralize legacy exploits. Through packet inspection, log analysis, and enhanced dissector integration, the framework introduces an evidence-driven validation layer that measures the resilience of applied countermeasures. Ultimately, while EmuOCPP establishes a reproducible testing benchmark, SafeCharge transforms it into a continuous, adaptive security evaluation framework, ensuring that protocol-level improvements are empirically verified against known and emerging threats.

## 2.2 Analysis Summary of Research Items

This table provides a concise summary of the critical analysis of the research items reviewed in the previous section. It consolidates the core objectives, methodologies, datasets, findings, and limitations of each study to highlight the research gap this project aims to address.

Table 2.1: Summary of critical analysis of research items

| **Paper** | **Publication** | **Year** | **Methodology** | **Datasets/Testing** | **Results** | **Limitations** |
|---|---|---|---|---|---|---|
| EmuOCPP: Effective and Scalable OCPP Security and Privacy Testing | Vehicle Security and Privacy Workshop (VehicleSec '25) | 2025 | IPMininet-based emulation supporting OCPP 1.6, 2.0, and 2.0.1 with Security Profiles 13 | Tested on nine OCPP implementations (both open-source and proprietary) | Demonstrated Man-in-the-Middle (MitM) via certificate injection, client certificate manipulation, charge station impersonation (I1/I2), DoS and downgrade (M4) attacks | Limited physical interaction testing, partial 2.0.1 message support, scalability capped beyond 200 nodes |
| Federated Detection of OCPP 1.6 Cyberattacks | Complex Engineering Systems | 2025 | FL-based IDS using OCPPFlowMeter and CICFlowMeter for network flow statistics | Network flow data from OCPP 1.6 testbed (SteVe and 22KW wall-box) | Detected Charging Profile Manipulation (FDI), Denial of Charge, Heartbeat Flooding attacks | OCPPFlowMeter lacks 2.0.1 support, vulnerable to adversarial AI evasion, relies only on traffic traces (no KPI metrics) |

| Paper | Publication Year | Methodology | Datasets/Testing | Results | Limitations |
|---|---|---|---|---|---|
| Cyber Defense in OCPP for EV Charging Security Risks | International Journal of Information Security 2025 | Threat modeling and vulnerability analysis using encryption and IDS-based framework | Simulated OCPP 1.6 environment | Proposed mitigation framework for data sniffing, charge point cloning, duplicate sessions, booking conflicts | Firmware manipulation (A7) not covered, hardware upgrades required, computational overhead on CPs, lacks experimental validation |
| OCPPStorm: A Comprehensive Fuzzing Tool for OCPP Implementations | Vehicle Security and Privacy Workshop (VehicleSec 2024) 2024 | Black-box fuzzing with Random, State Machine, and Isla fuzzers for protocol-level testing | Tested on SteVe and OCPP.Core implementations | Found input validation failures, DoS via oversized fields, unauthorized transaction termination, message ID reuse, invalid timestamp handling | Fully black-box (no code coverage), tests only CMS side (no CP emulation), slower message generation with Isla module |
| Charge Manipulation Attacks Against Smart Electric Vehicle Charging Stations and Deep Learning-based Detection Mechanisms | IEEE Transactions on Smart Grid 2024 | Hierarchical monitoring using EVCS telemetry and deep autoencoder anomaly detection | Simulated EVCS charging data (synthesized dataset) | Analyzed CMA attacks, server-side request forgery (CVE-2021-22821), cross-site request forgery (CVE-2022-22808), hard-coded credentials (CVE-2021-22730) | Excludes smart-charging logic, high false positives for mixed CMAs, simplified simulation omits CAISO models |

| Paper | Publication Year | Methodology | Datasets/Testing | Results | Limitations |
|---|---|---|---|---|---|
| Disrupting EV Charging Sessions and Gaining Remote Code Execution with DoS, MITM, and Code Injection Exploits using OCPP 1.6 | Proceedings of Resilience Week (RWS) 2023 | Five PoC exploits (MITM, Log4Shell, DoS, code injection) tested in lab and emulator | Virtual OCPP Cyber Range and Idaho National Laboratory's 350 kW EV Lab | Demonstrated Log4Shell firmware exploit, malicious firmware injection, DoS via SaiFlow vulnerability | Limited to Java-based systems, SYN DoS caused latency but no crash, lacks secure handling of multiple WebSockets |
| Cybersecurity Risk Analysis of Electric Vehicles Charging Stations | Sensors 2023 | Survey and analysis of EVCS vulnerabilities and real-world attack vectors | Real-world EV charging session data from ElaadNL | Identified DoS, malware injection in EVSE, physical tampering, device compromise | Lack of protocol standardization, limited access to testing, outdated EVSE hardware, AI-driven security underexplored |

## 2.3 Research Scope/Gap

The scope of this research is focused on the cybersecurity evaluation and enhancement of the Open Charge Point Protocol (OCPP), with particular emphasis on its latest iteration, OCPP 2.1 [15]. The study aims to systematically assess whether the security mechanisms introduced in OCPP 2.1 effectively mitigate vulnerabilities well-documented in earlier versions (OCPP 1.6 and 2.0.1) [10, 13]. This is achieved by extending the existing EmuOCPP [1] emulation framework to support OCPP 2.1, enabling comparative, adversarial testing in a controlled and reproducible environment. The research encompasses protocol specification analysis, emulator development, vulnerability validation, and the formulation of mitigation strategies. It explicitly excludes physical EVSE hardware control, Vehicle-to-Everything (V2X) integration, Distributed Energy Resource (DER) management, and real-world billing system implementation.

**Research Questions:**

- How do the security enhancements in OCPP 2.1 compare to those in OCPP 2.0.1 and 1.6 in terms of mitigating known attack vectors such as Man-in-the-Middle (MitM), impersonation, and denial-of-service?
- Can the six core and novel attack scenarios (e.g., M1-M4, I1/I2) documented in the

EmuOCPP base paper be successfully replicated and verified within the established OCPP 2.0.1 test environment?

- Can an extended emulation framework (EmuOCPP) effectively replicate and evaluate both legacy and novel vulnerabilities within an OCPP 2.1 environment?
- What actionable mitigation strategies and deployment guidelines can be derived from a comparative security analysis to strengthen real-world EV charging infrastructure?

**Research Objectives:**

- To extend the open-source EmuOCPP security testing framework to fully support OCPP 2.1 specifications, including critical new features for certificate management, authorization, and transaction handling.
- To verify the validity and reproducibility of the EmuOCPP base paper's findings by methodically implementing and executing its catalog of six novel attacks against OCPP 2.0.1 implementations within the emulation framework.
- To conduct a comparative security analysis by replicating documented attacks from OCPP 1.6/2.0.1 within the updated framework and evaluating their impact on OCPP 2.1.
- To develop and propose evidence-based security guidelines and mitigation strategies for stakeholders, including vendors and network operators, to ensure the resilient deployment of OCPP 2.1.

# Chapter 3

# Proposed Framework

## 3.1 Introduction

This research aims to evaluate and strengthen the cybersecurity posture of the Open Charge Point Protocol (OCPP) 2.1, the latest communication standard governing Electric Vehicle Charging Stations (EVCS). Building on the limitations identified in prior work, particularly EmuOCPP and related security frameworks, the proposed approach adopts a comprehensive, multi-phase methodology that moves beyond descriptive vulnerability studies toward prescriptive, test-driven cybersecurity validation [1, 5, 6].

The framework follows a sequential progression aligned with the project timeline. It begins with replication and consolidation of existing findings, and advances through comparative protocol testing, threat analysis, and the formulation of best-practice guidance for OCPP 2.1 deployments [12, 15].

## 3.2 Research Framework Overview

The proposed methodology is organized into four progressive phases, each aligned with the research objectives and building on outcomes from the preceding stage [1, 2, 6].

**Phase I: Foundational Research and Environment Setup**

- Conduct a detailed security review of OCPP 1.6 to establish baseline vulnerabilities and typical attack surfaces [5, 7].
- Perform a comparative analysis between OCPP 1.6 and OCPP 2.0.1 to identify evolutionary changes in security mechanisms and protocol behaviour [10, 13, 15].
- Survey and evaluate open-source OCPP emulators for suitability in adversarial test-

ing, including OCPPStorm and EmuOCPP [1, 2].

- Configure the EmuOCPP environment, setting up both Charge Point (CP) and Central Management System (CSMS) components for controlled experimentation [1].

**Phase II: Advanced Security Evaluation and Base Paper Implementation**

- Analyze the EmuOCPP architecture, modules, and attack simulation capabilities in detail [1].
- Use Wireshark-based dissectors (CheckOCPP) to perform packet inspection and log analysis for multi-version message-flow validation [11, 15].
- Replicate adversarial scenarios described in EmuOCPP to verify prior results and validate the emulators baseline fidelity [1, 4].
- Execute a comparative analysis between OCPP 2.0.1 and OCPP 2.1 to identify functional and security-oriented changes and assess their impact [6, 13, 15].

**Phase III: Regression Testing and Emulator Enhancement**

- Extend EmuOCPP to support OCPP 2.1 specifications, including new message types, updated schemas, and revised security profile behaviours [14, 15].
- Update the CheckOCPP dissector to ensure compatibility with OCPP 2.1 and accurate parsing of its message formats and security features [11].
- Reassess previously identified vulnerabilities from OCPP 2.0.1 and earlier versions to determine whether they remain exploitable under OCPP 2.1 [4, 5, 8].
- Perform regression testing to verify backward compatibility and confirm whether OCPP 2.1 security enhancements effectively mitigate legacy threats [1, 6].

**Phase IV: Mitigation and Validation**

- Conduct a comprehensive vulnerability assessment of OCPP 2.1 to identify residual weaknesses in authentication, authorization, and message handling [6, 7].
- Perform detailed threat analysis to map attack vectors and evaluate their impact on confidentiality, integrity, and availability [4, 8, 16].
- Develop mitigation strategies and best-practice guidelines addressing both protocol-level and implementation-specific concerns [6, 14].
- Validate the effectiveness of these countermeasures through final vulnerability testing and comparative evaluation across protocol versions [1].

This structured methodology ensures a continuous progression from theoretical understanding to empirical validation and, ultimately, to the formulation of actionable recommendations. The overall design of the research framework is illustrated in Figure 3.1.

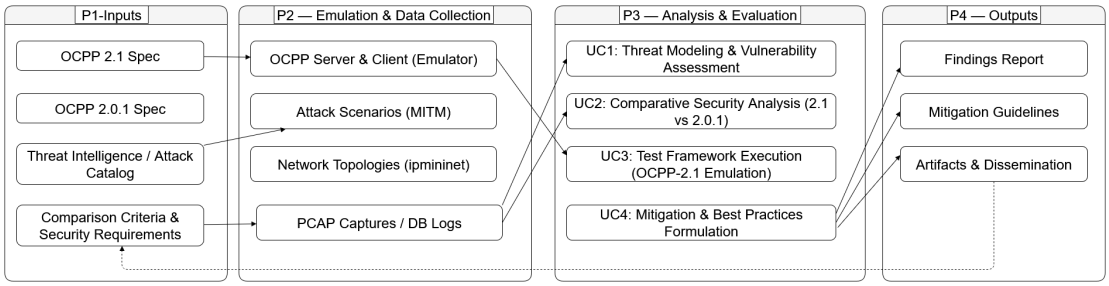



Figure 3.1: High-level research framework representation

## 3.3 Specification-Level Analysis of OCPP

The first core component of this research is a specification-level analysis of the Open Charge Point Protocol, with particular emphasis on OCPP 2.1 and its evolution from OCPP 2.0.1 and OCPP 1.6. This phase establishes the theoretical foundation for the emulation-based experiments that follow. Its objectives are to identify latent design-level weaknesses, examine security improvements introduced in newer versions, and construct a structured vulnerability catalog to guide subsequent testing [5, 6, 15].

### 3.3.1 Purpose and Objectives

The specification-level analysis is designed to:

- Identify inherent vulnerabilities in the communication, authentication, and operational mechanisms defined in OCPP 2.1 [6, 15].
- Trace the evolution of the protocol from OCPP 1.6 $\rightarrow$ 2.0.1 $\rightarrow$ 2.1 and evaluate how legacy weaknesses have been addressed or transformed [10, 13, 15].
- Assess the design and practical usage of security profiles in OCPP 2.1, including the risks associated with inconsistent vendor implementations [6, 14, 15].
- Identify protocol elements and message flows likely to be most susceptible to compromise, thereby informing emulation-based validation in later phases [1, 4, 8].

### 3.3.2 Analytical Scope and Methodology

The analysis adopts a structured, multi-layered perspective, viewing OCPP from both design and operational standpoints [1, 5].

#### 3.3.2.1 Protocol Structure Review

- Examine the core OCPP 2.1 communication model, which governs bidirectional CP–CSMS interaction over WebSocket and TCP/IP [11, 15].
- Analyze session initiation, message sequencing, and state management to assess robustness against synchronization failures and error conditions [6].
- Compare OCPP 2.1 with OCPP 2.0.1 and 1.6 in terms of layering, schema complexity, and state-transition handling [10, 13, 15].
- Identify potential attack entry points such as unverified connections, insufficient message validation, or weak recovery mechanisms, informed by previously documented exploits [4, 8, 9].

#### 3.3.2.2 Security Profiles and Cryptographic Mechanisms

- Examine the three security profiles:
  - **SP1:** Unsecured transport with basic authentication.
  - **SP2:** TLS with basic authentication.
  - **SP3:** TLS with client-side certificates.
- Evaluate each profiles default and optional configurations, and analyze risks introduced when weaker profiles (e.g., SP1) are deployed for compatibility or cost reasons [6, 14, 15].
- Compare the historical reliance on optional TLS in OCPP 1.6 and the partial improvements in OCPP 2.0.1 with the stricter expectations embedded in OCPP 2.1 [7, 10, 13].

#### 3.3.2.3 Message Structure and Flow Analysis

- Analyze key OCPP 2.1 messages such as `BootNotification`, `Authorize`, `StartTransaction`, `StopTransaction`, and `StatusNotification`, which underpin EVCS operations [11, 15].

- Evaluate these messages for potential exposure to tampering, spoofing, or replay and map findings to vulnerability classes reported in the literature [4, 5, 8].
- Assess how message signing, integrity checks, and verification mechanisms are specified and whether they are required or merely recommended [6].
- Compare message schemas and validation rules with OCPP 2.0.1 and 1.6 to identify where structural improvements are likely to affect exploitability [10, 13, 15].

#### 3.3.2.4 Comparative Security Evolution Mapping

- Construct a version-by-version security mapping across OCPP 1.6, 2.0.1, and 2.1 [6, 15].
- Identify vulnerabilities that have been resolved, those that persist due to design constraints or backward compatibility, and new risks introduced by extended capabilities such as richer telemetry and dynamic authorization [7, 8].
- Use this mapping to define emulation test targets, attack scenarios, and regression checks in later phases [1, 2, 5].

#### 3.3.2.5 Expected Deliverable

The outcome of this stage will be a Specification Vulnerability Catalog (SVC) that consolidates identified vulnerabilities and security-relevant design properties. The SVC will include:

1. Version-to-version vulnerability mapping (OCPP 1.6 $\rightarrow$ 2.0.1 $\rightarrow$ 2.1).
2. A security-profile assessment matrix outlining weaknesses and operational risks for SP1–SP3 [15].
3. Attack-surface documentation for critical messages and service flows [4, 8].
4. Prioritized recommendations for emulation-based testing and regression focus [1, 6].

This catalog will guide subsequent emulation experiments and ensure that test design is evidence-driven and aligned with documented protocol structures. It will also support the projects broader goal of delivering the first structured, comparative cybersecurity evaluation of OCPP 2.1 relative to its predecessors [5, 6, 8].

## 3.4 Base Paper Implementation: EmuOCPP Framework

The experimental foundation of this research is the implementation and extension of EmuOCPP, a comprehensive and scalable security testing framework for OCPP. EmuOCPP, developed by Boussaha et al., offers an open-source, container-based environment for systematically evaluating the security and privacy of OCPP 1.6, 2.0, and 2.0.1 [1]. It addresses prior limitations in OCPP security research, such as incomplete protocol coverage, micro-scale testbeds, and limited reproducibility [2, 5].

### 3.4.1 Design and Core Components

EmuOCPP is designed around six principal requirements: multi-version support, multi-security-profile coverage, realistic network emulation, design- and implementation-level testing, scalability, and reproducibility. Instead of using full virtual machines, it leverages container-based network emulation, enabling hundreds of realistic CP and CSMS instances to run efficiently on standard hardware [1].

Its architecture integrates four main modules:

1. **IPMininet Module:** Provides large-scale, IPv6-capable network emulation using FRRouting (FRR), supporting dynamic routing and realistic latency conditions [1].
2. **Test Module:** Implements automated attack scripts and test routines for multi-profile OCPP security evaluation [2].
3. **Messages Module:** Extends The Mobility House Python OCPP library to handle diverse message schemas and payload definitions across supported versions [10, 13, 15].
4. **Config Module and GUI:** Uses YAML-based configuration files and a Tkinter graphical interface for creating, parameterizing, and visualizing topologies.

Through this modular design, EmuOCPP can emulate diverse OCPP deployments, integrate different protocol stacks, and orchestrate complex adversarial experiments while preserving fine-grain control over topology and configuration [1, 6].

### 3.4.2 Security Evaluation

EmuOCPP supports both design-level and implementation-level analysis. It reproduces OCPP 1.6, 2.0, and 2.0.1 message exchanges under multiple security profiles (SP1–SP3) and attacker models. With IPMininet and integrated tools such as Scapy and Mitmproxy,

the framework can simulate sniffing, spoofing, MitM, and DoS attacks within realistic network settings [1, 2, 4].

The original EmuOCPP study uncovered eight critical attacks, including:

- **SP Upgrade/Downgrade Manipulation (M2–M4):** Exploiting weak logic governing transitions between security profiles.
- **Charging Station Impersonation (I1, I2):** Leveraging undefined behaviours in `BootNotification` and static charger identifiers [8].
- **Denial-of-Service (D1):** Targeting rate-limiting and input-validation weaknesses.
- **Eavesdropping (E1) and MitM under SP1 (M1):** Exploiting unencrypted or weakly authenticated channels [6].

These findings showed that critical vulnerabilities persist across protocol versions and profiles, driven by both specification ambiguities and vendor-specific implementation choices [5, 6, 8].

### 3.4.3 Application of Base Paper

Within this research, EmuOCPP serves as the baseline environment that is first validated and then extended. Concretely, the project will:

- Recreate and validate EmuOCPPs documented attacks (M1–M4, I1–I2, D1, E1) on OCPP 1.6 and 2.0.1 [1, 4].
- Extend the message-handling and protocol modules to support OCPP 2.1 schemas and behavioural updates [13, 15].
- Update CheckOCPP and accompanying tooling to parse and validate OCPP 2.1-specific flows and TLS configurations [11].
- Use the extended framework to compare the security posture of OCPP 2.1 against the documented weaknesses of previous versions [6, 8].

Through these steps, EmuOCPP is transformed from a version-limited testbed into a version-aware security evaluation platform. The results from this extended framework directly inform the mitigation strategies and best practices proposed for secure OCPP 2.1 deployment [5, 14].

## 3.5 Updating EmuOCPP to Support OCPP 2.1

This phase focuses on extending EmuOCPP to incorporate OCPP 2.1, thereby achieving protocol version parity and enabling meaningful cross-version comparison. The upgraded emulator becomes a comprehensive test environment for evaluating the efficacy of new security and control mechanisms introduced in OCPP 2.1 [1, 6, 15].

### 3.5.1 Alignment with this Research

OCPP 2.1 introduces numerous behavioural and structural modifications across its functional profiles and service flows [14, 15]. Since several of these changes explicitly target security-related concerns identified in the literature, implementing them within EmuOCPP is essential for answering the central research question:

> *Does OCPP 2.1 effectively mitigate the security weaknesses observed in earlier protocol versions?*

By extending EmuOCPP to support OCPP 2.1, the framework can:

- Test newly introduced message categories and workflows, particularly around certificate management, transaction recovery, and authorization.
- Evaluate whether previously documented attacks (e.g., downgrade manipulation, impersonation, MitM) remain feasible under updated security profiles [4, 5, 6, 8].
- Generate empirical evidence on improvements (or regressions) in security, interoperability, and operational consistency across protocol versions [1, 13].

## 3.6 Regression Testing

Regression testing constitutes the final methodological stage of the SafeCharge experimental workflow. Its purpose is to ensure that the enhancements made to EmuOCPP, particularly the integration of OCPP 2.1 support and extended verification tooling, do not introduce unintended regressions affecting earlier protocol behaviour [1, 15].

### 3.6.1 Purpose and Role

In this context, regression testing serves as a consistency and stability checkpoint. While earlier phases focus on adding capabilities and validating security enhancements, this phase confirms that:

- Pre-existing OCPP 1.6 and 2.0.1 message flows remain valid and semantically correct after the integration of OCPP 2.1 [11, 15].
- Core protocol events (e.g., boot, authorization, start/stop transaction) preserve expected behaviour and timing characteristics [1].
- Security profiles and certificate validation workflows continue to operate as previously verified across all supported versions [6, 13].

### 3.6.2 Testing Methodology

The regression testing process follows a structured sequence:

1. **Baseline Selection:** Identify previously validated test cases and message traces for OCPP 1.6 and 2.0.1. Each baseline test includes known-correct request/response pairs verified using CheckOCPP [1].
2. **Re-Execution in Updated Framework:** Execute the same scenarios in the upgraded EmuOCPP environment with OCPP 2.1 support enabled. Ensure that the underlying network conditions and configuration parameters are held constant [15].
3. **Cross-Version Verification:** Use CheckOCPP to validate both baseline and updated traces, confirming that schema-conforming messages remain structurally and semantically equivalent [11, 15].
4. **Reporting and Remediation:** Any observed deviations or anomalies trigger a structured investigation and corrective cycle, ensuring that backward compatibility and performance benchmarks are restored before further evaluation [6].

Through this regression-testing stage, the SafeCharge framework ensures that protocol extensions and experimental modifications yield a stable, version-consistent platform for OCPP security evaluation, rather than trading new capabilities for reduced reliability in earlier versions.

## 3.7 EmuOCPP Flow

### 3.7.1 Core Components

The EmuOCPP flow follows a modular design composed of five main elements: configuration input, security processing, network emulation, OCPP messaging, and monitoring for analysis. [1] Together, these components enable repeatable experiments that integrate

configurable charging infrastructures, attacker placement, and automated trace collection for post-hoc inspection.

#### 3.7.1.1 Configuration Layer

The configuration layer ingests YAML-based scenario files that describe charging stations, CSMS instances, attackers, and network parameters. These inputs drive automatic generation of credentials, certificates, and deployment options so that complex testbeds can be instantiated with minimal manual effort.

#### 3.7.1.2 Security Processing

The security processing element applies the selected OCPP security profile, TLS settings, and certificate material before any protocol traffic is exchanged. [13, 15] This layer allows experiments to toggle between plaintext, one-sided TLS, and mutual authentication while keeping the rest of the emulation workflow unchanged.

#### 3.7.1.3 Network Emulation

The network emulation environment virtualizes routers, switches, DNS servers, and CS/CSMS hosts to reproduce realistic routing paths and failure modes. [1] It supports topologies with multiple chargers and CSMS instances, enabling studies of latency, path asymmetry, and routing-based attacks under controlled conditions.

#### 3.7.1.4 OCPP Messaging and User State

Within the emulated network, EmuOCPP runs charging stations, CSMS processes, and user-authorization backends that exchange OCPP messages over WebSockets. [10, 13] This core loop maintains session state, credentials, and reservations so that protocol-level attacks and mitigations can be exercised across different OCPP versions and profiles.

#### 3.7.1.5 Monitoring and Analysis

Finally, EmuOCPP exports traffic captures and logs for offline analysis with tools such as Wireshark. [2] These artefacts provide a ground-truth view of message flows, certificate usage, and attacker activity, supporting both debugging and systematic evaluation of security properties.

## 3.7.2 Graphical Description

### 3.7.2.1 End-to-End Flow Diagram

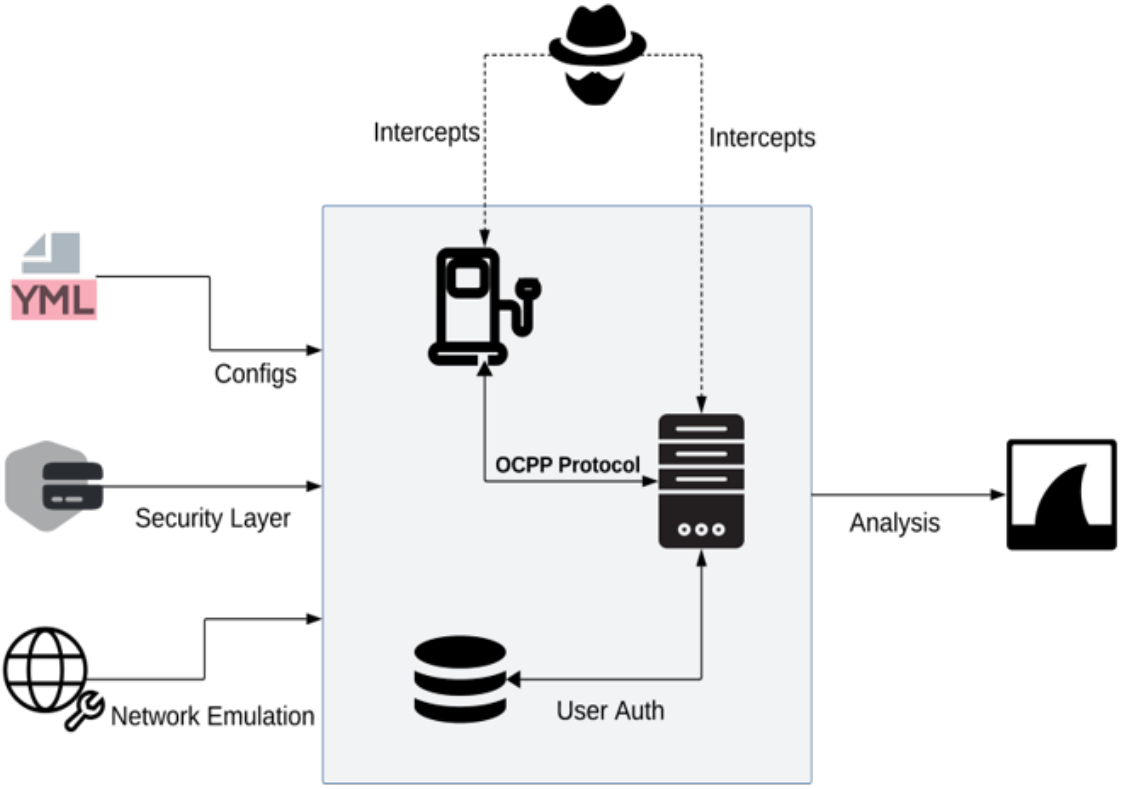



Figure 3.2: High-level EmuOCPP flow

Figure 3.2 summarizes the end-to-end workflow of EmuOCPP, from YAML configuration and security options through to attacker interception and trace collection. [1] The diagram highlights how configuration, protocol execution, and monitoring are connected to support repeatable security experiments over the same logical scenario.

## 3.8 Experimental Setup

The experimental setup was created using VMware Workstation to provide a controlled and reproducible environment for deploying Mininet, IPMininet, and EmuOCPP.

**Virtual Machine Configuration:**

- Ubuntu 20.04 LTS guest OS
- 4 GB RAM
- 8 vCPUs
- 30 GB virtual hard disk (SCSI)
- NAT networking mode
- Installation ISO mounted via virtual CD/DVD drive

**Network Emulation Environment:**

- Installed Mininet and manually restored the missing `mnexec` binary
- Created a dedicated Python virtual environment for IPMininet
- Installed legacy build tools and IPMininet v1.1
- Installed FRR routing daemons (`zebra`, `ospfd`, `staticd`) and enabled IPv4/IPv6 forwarding
- Activated required services including Open vSwitch and FRR
- Resolved `frrvty` group permission issues to enable routing operations

**EmuOCPP Deployment:**

- Activated the virtual environment
- Navigated to the EmuOCPP directory
- Installed dependencies using `pip install -r requirements.txt`
- Launched the emulator and verified functionality through example scenarios and attack demonstrations

### 3.8.1 Evaluation Metrics

This section defines the theoretical criteria used to evaluate the attack scenarios reproduced in this project. Rather than providing numerical measurements, the evaluation focuses on qualitative dimensions that describe how effectively the attacks exploit weaknesses in OCPP communication. All metrics are defined specifically in relation to the six replicated attack types: M1–M4 (Man-in-the-Middle) and I1–I2 (Impersonation).

#### 3.8.1.1 Security Profile Behaviour

The primary focus for MitM attacks is the behaviour of OCPP security profiles during and after exploitation. Evaluation considers:

- **Security Profile Exposure:** whether an attack makes it possible to establish communication over a weaker security profile than the system initially supports.
- **Upgrade and Downgrade Response:** how the charge point and the central system react when security profile transitions (e.g., SP1 $\rightarrow$ SP2 $\rightarrow$ SP3) are manipulated.
- **Persistence of Incorrect Security States:** whether manipulated security configurations remain active after disconnection or session restart.

These criteria are relevant for attacks M1–M4, where certificate handling and profile negotiation were deliberately targeted.

#### 3.8.1.2 Message Manipulation Capability

Another area of evaluation is the extent to which attackers can interfere with OCPP message exchanges during active connections. The following questions guide this assessment:

- **Acceptance of Modified Messages:** whether altered OCPP requests or responses are accepted by the central management system without authentication failure.
- **Control over Charging Operations:** whether malicious messages can change availability states, start or stop charging sessions, or alter operational parameters.
- **Visibility of Modification Effects:** whether the consequences of message manipulation appear immediately or remain hidden until later stages of system operation.

These criteria apply primarily to attacks where an adversary inserts or alters traffic in real time (M1–M3).

#### 3.8.1.3 Identity Duplication and Session Control

For impersonation attacks, the evaluation examines how system identity assumptions can be exploited. Assessment focuses on:

- **Identifier Conflict Handling:** how the system behaves when the same charging point identifier is used by multiple parties simultaneously.
- **Legitimate Session Replacement:** whether an attacker can take over an active session and force a legitimate device offline or into a secondary role.
- **Offline Opportunity Exploitation:** whether impersonation is easier or fully enabled when a legitimate charge point is temporarily disconnected.

These criteria are directly connected to attacks I1 and I2, which target OCPP identifier trust assumptions.

#### 3.8.1.4 Certificate and Provisioning Integrity

Because several attacks rely on certificate substitution or manipulation, the evaluation includes a theoretical review of certificate management behaviour. This includes:

- **Reaction to Untrusted Certificates:** whether an OCPP implementation accepts certificates that are not authentic or not signed by a trusted authority.
- **CSR Injection Potential:** whether it is possible to interfere with certificate signing requests during profile upgrades.
- **Absence of Binding Between ID and Certificate:** whether device identity is cryptographically linked to certificate material or can be separated during an attack.

This applies to M2 and M3 specifically, where certificate lifecycle weaknesses are essential to attack success.

#### 3.8.1.5 Post-Attack Communication State

Finally, evaluation considers the resulting system behaviour after an attack has been completed. Rather than scoring results, this describes the broader operational state:

- **Recovery without Manual Intervention:** whether the system returns to a secure configuration on its own after the attack ceases.

- **Requirement of External Reset Actions:** whether human action, system restart, or certificate replacement is required to return to normal behaviour.
- **Persistence of Malicious Configuration Changes:** whether changes remain in effect permanently or until explicitly corrected by administrators.

This final set of criteria provides a high-level view of the real-world impact of successful M1–M4 and I1–I2 exploit scenarios.

#### 3.8.1.6 Summary

These evaluation metrics provide a structured, attack-focused framework for analysing the outcomes of the replicated experiments. By concentrating on security profile exposure, message manipulation capability, identity duplication behaviour, certificate provisioning integrity, and post-attack system state, this approach supports a detailed, qualitative understanding of each attack without requiring numerical scoring or quantitative performance measurement.

## 3.9 Results

### 3.9.1 Research Paper Attack Verification

This section evaluates the extent to which the EmuOCPP implementation and attack scripts successfully replicate the 5 novel attack scenarios (M2M4) and (I1,I2) as described in the base EmuOCPP paper [1]. The goal was to verify that the vulnerabilities and attack chains reported in the paper can be reproduced in a controlled lab environment using the SimpleMitM and extended AdvancedMitM_M1_to_M4 scenarios.

| ID | Name | New | OCPP SP | OCPP Ver. | Vulnerability |
|---|---|---|---|---|---|
| M1 | MitM SP1 | × | SP1 | 1.6, 2.0, 2.0.1 | Weak SP1 |
| M2 | MitM SP2 Upgrade | ✓ | SP2 | 1.6, 2.0, 2.0.1 | Weak SP2 upgrade |
| M3 | MitM SP3 Upgrade | ✓ | SP3 | 1.6, 2.0, 2.0.1 | Weak SP3 upgrade |
| M4 | MitM SP Downgrade | ✓ | SP2, SP3 | 1.6, 2.0, 2.0.1 | No SP downgrade protection |
| E1 | Eavesdrop | × | SP1 | 1.6, 2.0, 2.0.1 | Weak SP1 |
| D1 | CSMS (D)DoS | × | SP1, SP2, SP3 | 1.6, 2.0, 2.0.1 | Unauth CS boot |
| I1 | CS Impersonation | ✓ | SP1, SP2, SP3 | 1.6, 2.0, 2.0.1 | CS ID undefined behaviour |
| I2 | CS Impersonation | ✓ | SP1, SP2, SP3 | 1.6, 2.0, 2.0.1 | CS status trackable by anyone |

Table 3.1: Summary of Attacks in EmuOCPP

As shown in Table 3.1, the verified attack scenarios are summarised based on their identifier, novelty status, associated OCPP security profile, supported protocol versions, and

a short description of the underlying vulnerability. The table presents only the core attributes required for experimental validation, including whether the attack is new with respect to the original research, and which OCPP security mechanisms are affected.

The results confirm successful reproduction of all relevant attack paths defined in the base EmuOCPP study [1]. In particular, the MitM upgrade and downgrade attacks (M1–M4) and the impersonation-based scenarios (I1–I2) were consistently reproducible across multiple OCPP versions (1.6, 2.0, and 2.0.1). Each verified scenario demonstrates a concrete weakness in security profile negotiation, device identity handling, or message integrity protection. This supports the original claim that insufficient profile enforcement and weak configuration control remain practical attack surfaces in real OCPP deployments.

#### 3.9.1.1 Experimental Setup for Attack M1–M4

**Emulation framework:** EmuOCPP was used to emulate OCPP 1.6, OCPP 2.0, and OCPP 2.0.1, supporting Security Profiles 0–3.

**Network emulation:** The network topology was implemented using Mininet and IP-Mininet, with the following fixed host layout:

- **h1:** Charge Point (client, `charging/client.py`)
- **h3:** CSMS (server, `charging/server.py`)
- **h4:** CSO (operator, `charging/cso.py`)
- **h2:** Attacker/MitM
- **r1, r2, s1:** Routers and switch interconnecting the hosts

**Attack tooling:**

- **IPv6 layer:** THC-IPv6 tools (`parasite6`, `flood_router26`, etc.)
- **Application/TLS layer:** `mitmdump` with custom OCPP scripts (`mitm/mitm.py`, `mitm/downgrade.py`)

**Client configuration:**

- OCPP 2.0.1, SecurityProfile: 1 by default

- Three connection profiles configured with

  `NetworkConfigurationPriority: [0, 1, 2]`, corresponding to SP1, SP2, and SP3 respectively

- `AdditionalRootCertificateCheck = false`, as assumed in the paper for M2/M3 feasibility

**MitM orchestration:** The baseline `SimpleMitM` scenario was used as provided, and a new orchestrator script (`AdvancedMitM_M1_to_M4.py`) was developed to explicitly exercise the complete M1–M4 attack sequence while preserving original runtime behaviour.

#### 3.9.1.2 M1 – MitM on SP1 / No Security Profile (MitMSP1)

**Objective**

Demonstrate that, in OCPP deployments using SP1, an attacker can obtain an on-path position and passively or actively manipulate all OCPP traffic by exploiting unauthenticated IPv6 control-plane mechanisms.

**Method**

Launched the Mininet/IPMininet topology and started:

- The CSMS on `h3` (`server.py`) listening on ports 9000–9007.
- The CSO on `h4` (`cso.py`), connecting to `h3` after a short delay.
- The Charge Point on `h1` (`client.py`) with an initial profile permitting SP1/unencrypted operation.

On the attacker host `h2`:

- Executed `prep.sh` to enable IPv6 forwarding and adjust ICMPv6 handling.
- Ran `parasite6 -l h2-eth0` to inject spoofed IPv6 Router Advertisements and Neighbor Advertisements.
- Optionally ran `flood_router26 h2-eth0` to generate a high volume of fake routing information.

**Observed behaviour**

Traffic between `h1` and `h3` was successfully rerouted through `h2` without any changes to the endpoints. For SP0/SP1 connections (no TLS), `h2` could:

- Capture full OCPP WebSocket frames in plaintext.
- Drop or delay selected packets, causing timeouts and reconnections.
- Maintain ongoing visibility into `BootNotification`, `Authorize`, and transaction messages.

**Consistency with paper**

The practical behaviour matches the M1 description:

- Link-layer / network-layer spoofing (IPv6 NDP in this setup) sufficed to secure a MitM position.
- OCPP traffic using SP1 was fully exposed, with no confidentiality or server authentication.
- No endpoint changes were required, aligning with the papers original threat model.

#### 3.9.1.3 M2 – MitM During SP2 Upgrade (MitMSP2Upgrade)

**Objective**

Confirm that, starting from an existing M1 on-path position, an attacker can persist through an SP1 $\rightarrow$ SP2 upgrade by injecting a malicious CSMS root certificate during certificate installation and enrollment flows.

**Method**

With an established MitM on `h2`, the upgrade scenario proceeded using:

- Temporary disabling of IPv6 forwarding on `h2` to force connection breakage between `h1` and `h3`.
- Redirection of OCPP ports 9000–9007 through `h2` via `ip6tables` rules (`routes.mn`).
- Launching `mitmdump` in transparent mode with `mitm.py` as the interception script.
- Initiation of an SP2 upgrade via the CSMS/CSO, triggering a configuration change on the client to use Security Profile 2.

`mitm.py` selectively intercepted:

- `InstallCertificate` and `SignCertificate` messages.
- Client certificate management flows, replacing the legitimate CSMS root certificate with an attacker-controlled one.

**Observed behaviour**

The client successfully installed the attacker-controlled CSMS root certificate provided that:

- `AdditionalRootCertificateCheck` remained `false`.
- No chain-of-trust or semantic validation was performed beyond basic syntactic checks.

Following the SP2 upgrade, the client:

- Completed TLS handshakes using attacker-supplied server certificates.
- Established a WebSocket tunnel over SP2 (TLS with server authentication) terminating at the attacker.

The attacker retained full visibility and manipulation capability over OCPP messages, now encrypted on-wire but effectively re-encrypted through the transparent proxy.

**Consistency with paper**

The practical results match the core M2 claim:

- With `AdditionalRootCertificateCheck = false`, any arbitrary/self-signed root can be installed as the CSMS trust anchor.
- Once installed, the attacker can impersonate the CSMS post-upgrade, persisting across SP1 $\rightarrow$ SP2 transitions.

#### 3.9.1.4 M3 – MitM During SP3 Upgrade (MitMSP3Upgrade)

**Objective**

Verify that an attacker in the M2 state (controlling the SP2 root and enrollment flow) can survive an SP2 $\rightarrow$ SP3 upgrade by substituting attacker-controlled key material during client certificate provisioning.

**Method**

Building from M2 (attacker controls CSMS root):

- The CSMS/CSO initiated an SP3 upgrade (TLS with mutual authentication).
- `mitm.py` intercepted `SignCertificate` messages.

- The original client CSR was dropped and replaced with an attacker-generated CSR (`mod_csr`) containing attacker-controlled key material.

On receipt of the signed certificate chain from the CSMS:

- The attacker saved the legitimate CSMS-signed certificate.
- A forged certificate was returned to the client, causing:
  - Client → attacker mutual TLS.
  - Attacker → CSMS mutual TLS using the legitimate certificate for the client identity.

**Observed behaviour**

Both endpoints believed a correct SP3 upgrade occurred:

- The client authenticated to the attacker posing as the CSMS.
- The attacker authenticated to the CSMS using a valid CSMS-signed certificate for the client's identity.

Resulting OCPP traffic in SP3 was:

- TLS-encrypted on both legs of the connection.
- Fully decrypted and re-encrypted at the attacker, enabling modification and selective packet interference as in M1/M2.

**Consistency with paper**

The behaviour reproduces the M3 vulnerability:

- Client certificates are provisioned directly by the CSMS rather than via a trusted CA.
- A MitM controlling the enrollment channel can substitute a CSR and obtain a *valid*, CSMS-signed certificate for its own key pair.

#### 3.9.1.5 M4 – Downgrade from SP3/SP2 to SP1 (MitMSPDowngrade)

**Objective**

Demonstrate that a MitM can force an SP3/SP2-enabled system to downgrade to SP1 (or no security) by selectively blocking connection attempts to higher-security profiles while leaving a lower-security profile reachable.

**Method**

From a stable SP3 configuration under MitM (following M2/M3), the downgrade phase used:

- Persistent on-path positioning via NDP spoofing and forwarding (M1).
- Removal of MitM NAT rules so traffic returned to "normal" routing, still transiting `h2` at L2.
- `ip6tables` DROP rules blocking:
  - TCP/9007 (SP3),
  - TCP/9006 (SP2),

  while keeping SP1 ports (e.g., TCP/9005) reachable.

The client connection logic (unchanged) attempted profiles in the order specified by `NetworkConfigurationPriority`:

1. SP3
2. SP2
3. SP1

**Observed behaviour**

- SP3 (9007) connection attempts failed due to DROP rules.
- SP2 (9006) attempts subsequently failed for the same reason.
- After exhausting retry counters, the client fell back to SP1 (9005) and established an unprotected WebSocket connection.

Because the attacker retained an on-path position, the resulting SP1 session was fully visible and modifiable, despite prior successful operation in SP3.

**Consistency with paper**

The downgrade reproduces M4 requirements:

- Multiple active connection profiles remained enabled after upgrading, contrary to best practice.

- Selective port-level blocking forced the client to revert to SP1, restoring a plaintext MitM even after achieving SP3.

#### 3.9.1.6 Summary of Verification Outcomes

- **M1 (MitMSP1)**: Achieved full MitM using IPv6 NDP spoofing; complete visibility into SP0/SP1 plaintext OCPP traffic.

- **M2 (MitMSP2Upgrade)**: Verified malicious root certificate installation and MitM continuity across SP2 upgrades when `AdditionalRootCertificateCheck = false`.

- **M3 (MitMSP3Upgrade)**: Demonstrated attacker-controlled enrollment of client certificates enabling dual SP3 TLS sessions.

- **M4 (MitMSPDowngrade)**: Forced an SP3/SP2 configuration back to SP1 through selective port blocking and profile priority exploitation.

Overall, the EmuOCPP implementation and attack scripts faithfully reproduced all four attack classes reported in the original paper, matching both preconditions (configuration parameters, profile ordering) and observable postconditions (MitM persistence, downgrade ability).

#### 3.9.1.7 Demonstration of Attacks M1–M4

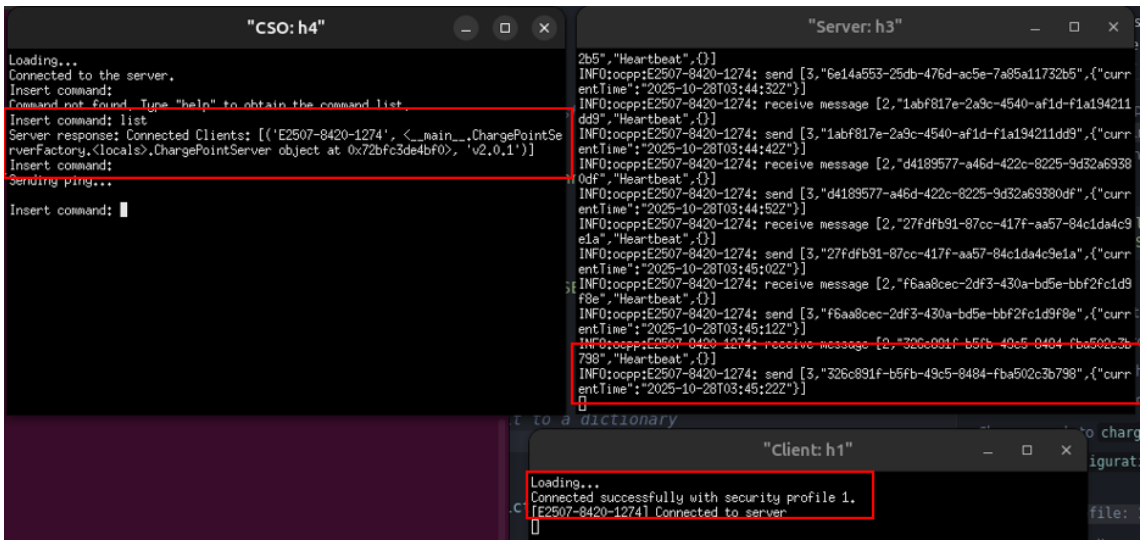



Figure 3.3: Initial EmuOCPP network setup for attacks M1–M4 showing the CSO (h4), CSMS (h3), and client (h1) prior to MitM positioning.

Figure 3.3 illustrates the baseline EmuOCPP network configuration prior to executing any adversarial activity. In this initial state, the client node (h1), acting as a simulated charging station, successfully establishes an OCPP WebSocket connection to the central system running on host (h3). The communication is configured to use Security Profile 1 (SP1), which provides no encryption or certificate-based authentication, resulting in plaintext message exchange over the network. The CSO interface accessible on (h4) confirms that the connection has been initiated correctly: upon issuing the `list` command, the CSO displays the identifier of the connected client, indicating an active and stable session between the charge point and the central management system.

From a security perspective, this baseline configuration is important because it exposes the full OCPP message flow without protection, thereby revealing the complete attack surface for subsequent manipulation. SP1 communication permits an attacker positioned anywhere on the same network segment to intercept, inspect, and modify traffic at the WebSocket layer without needing to bypass cryptographic defences. This makes it possible to capture core OCPP message structures such as `BootNotification`, `Authorize`, `StartTransaction`, and `Heartbeat` requests in a readable JSON format. In practical terms, this configuration mirrors many real-world deployments where OCPP implementations are still operated in legacy or development modes without secure transport enforcement.

Moreover, this initial stage verifies correct system functioning prior to attack execution.

Establishing the unprotected session ensures that message flow, device registration, and central system recognition operate as expected in the absence of interference. This provides a clean baseline against which subsequent attacks, beginning with M1 (MitMSP1), can be evaluated. Because SP1 imposes no encryption or endpoint validation, it represents the ideal starting point for demonstrating fundamental man-in-the-middle capabilities, including traffic capture, replay, and on-path manipulation. The observations made in this configuration therefore serve as a comparative foundation for analysing behavioural changes as security profiles are escalated and exploited in later experiments.

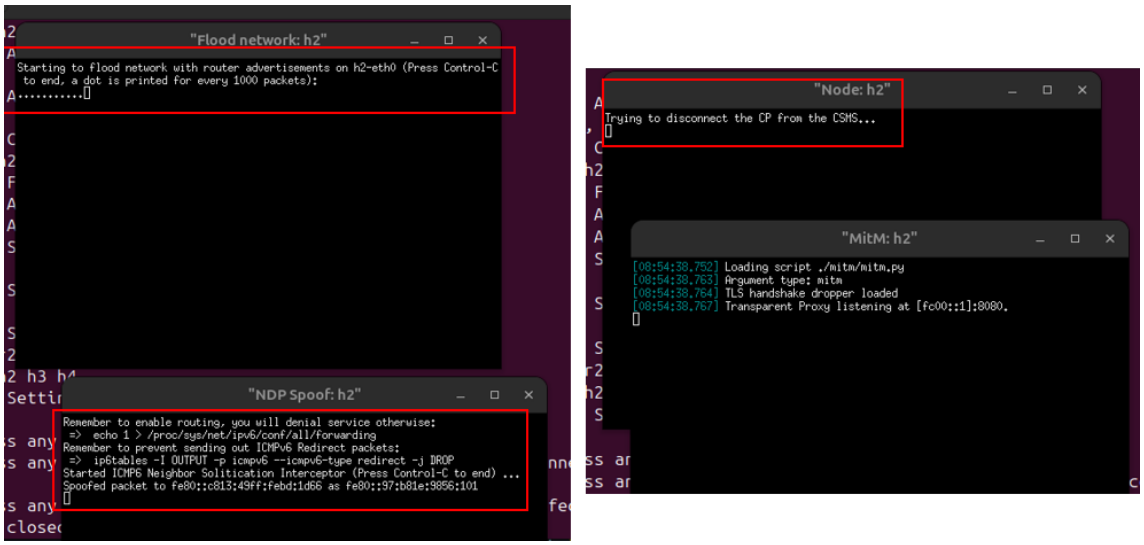



Figure 3.4: Attack setup on host h2 using IPv6 manipulation tools and a transparent proxy to establish a persistent on-path position.

Figure 3.4 shows the network-layer components executed from the attacker-controlled host h2. Three processes establish the MitM position:

1. `flood_router26` to generate fake IPv6 Router Advertisement messages, poisoning routing tables on neighboring hosts.
2. `parasite6` to spoof Neighbor Discovery Protocol messages, redirecting traffic through h2 at the link layer.
3. `mitmdump` to start a transparent proxy ready to intercept application-layer OCPP traffic.

This multi-stage approach ensures persistent on-path positioning regardless of subsequent security profile upgrades. Once the proxy is active, the attacker can intercept certificate installation messages during SP2 upgrades (M2), manipulate CSR exchanges during SP3 upgrades (M3), and selectively block ports to force downgrade behaviour to SP1 (M4).

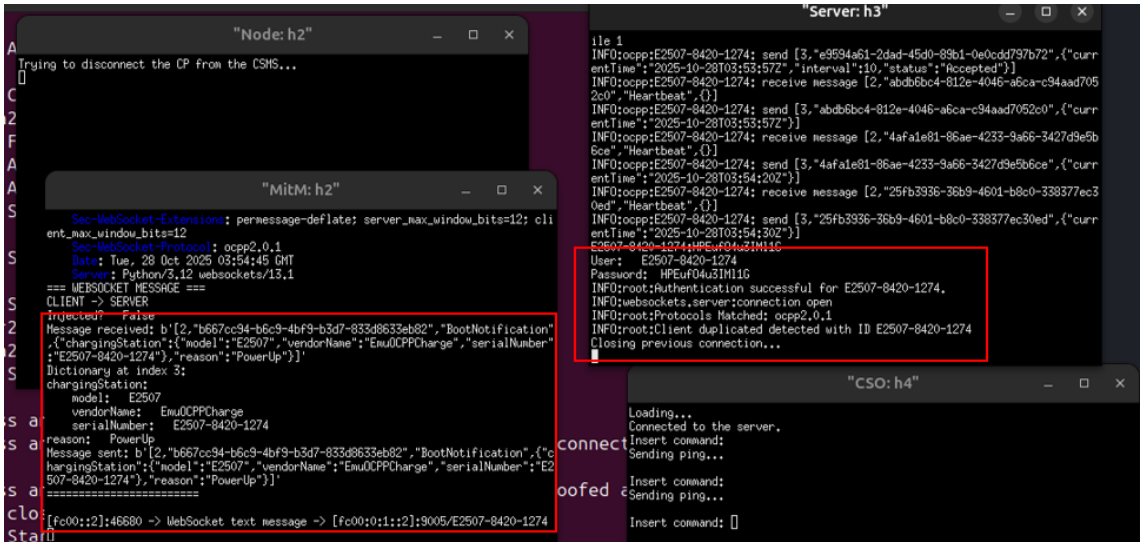



Figure 3.5: Simultaneous view of MitM interception and server-side duplicate connection detection during a connection hijacking event.

Figure 3.5 illustrates the critical phase of the M1 attack where the attacker transitions from passive interception to active session hijacking. The screenshot shows three relevant terminal views:

**MitM terminal:** The attacker proxy successfully intercepts the initial `BootNotification` message, revealing the charging station identity and confirming full visibility into OCPP message content.

**Attack terminal:** Network manipulation tools temporarily disrupt the existing connection between the legitimate client and the CSMS. This forced break triggers the client's automatic reconnection logic.

**Server terminal:** The server detects a duplicate client connection with the same identifier and automatically closes the previous session. It then accepts the new connection without distinguishing that it is now mediated by the attacker.

This sequence validates the paper's claim that OCPP connection handling lacks anti-hijacking protections. By exploiting automatic reconnection and duplicate handling behaviour, the attacker can seamlessly insert themselves into the communication path, establishing a persistent MitM position without raising security alerts or requiring endpoint modification.

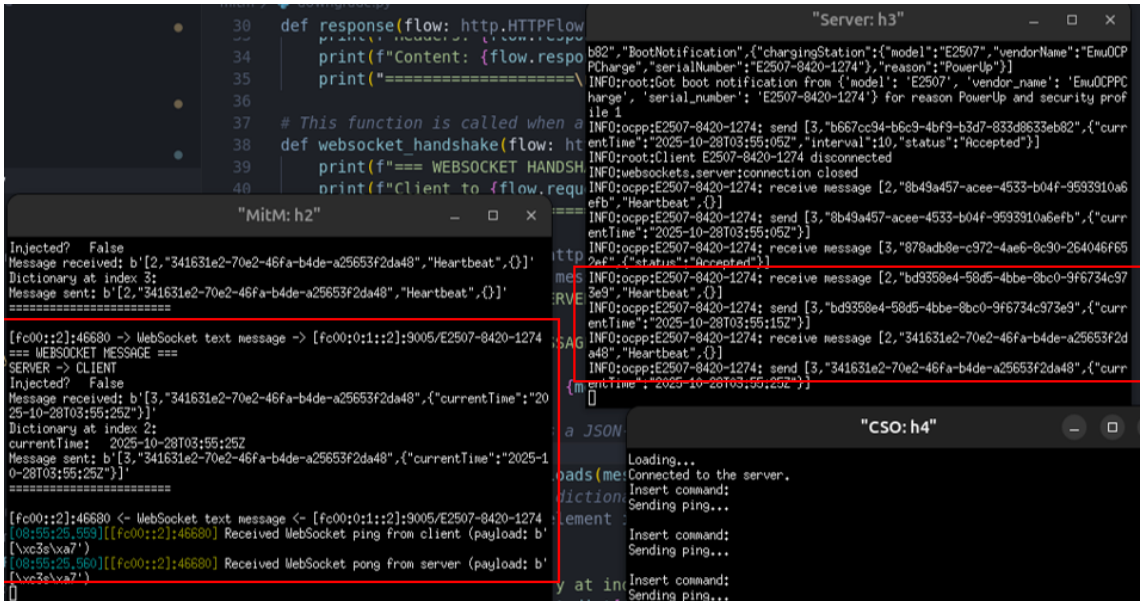



Figure 3.6: Successful certificate manipulation and persistent MitM interception after security profile upgrades.

Figure 3.6 demonstrates the attacker's persistent MitM position following the successful implementation of certificate manipulation prerequisites for M2 and M3 attacks. The view shows two critical aspects of the attack chain:

**MitM terminal:** Continuous interception of OCPP messages, including `Heartbeat` responses containing server timestamps (e.g., `currentTime: 2025-10-28T03:59:25Z`). The attacker's visibility into full message content confirms that:

- The MitM position remains active despite security mechanisms
- All OCPP message fields are exposed to the attacker
- Message IDs (e.g., `341531e2-70e2-46fa-b4de-a25653f24a48`) can be traced

**Server terminal:** Normal server operation continues, processing `BootNotification` and `Heartbeat` messages while remaining unaware of the interception. The server successfully authenticates the connection and maintains standard OCPP protocol flow.

This demonstration validates the base paper's claims regarding certificate manipulation prerequisites:

- **M2 (MitM SP2 upgrade):** The attacker has established the necessary conditions to install a malicious root certificate when `AdditionalRootCertificateCheck = False`, enabling persistence through SP1 SP2 upgrades

- **M3 (MitM SP3 upgrade):** With root certificate control established, the attacker can manipulate Certificate Signing Requests (CSR) during client certificate enrollment, obtaining valid CSMS-signed certificates for attacker-controlled key pairs
- **Message correlation:** Matching message IDs between intercepted and server logs confirms the attacker's position in the communication path without disrupting normal operation

The successful interception of ongoing OCPP traffic demonstrates that the attacker maintains complete visibility while preparing for subsequent security profile upgrades, consistent with the base paper's M2 and M3 attack scenarios.

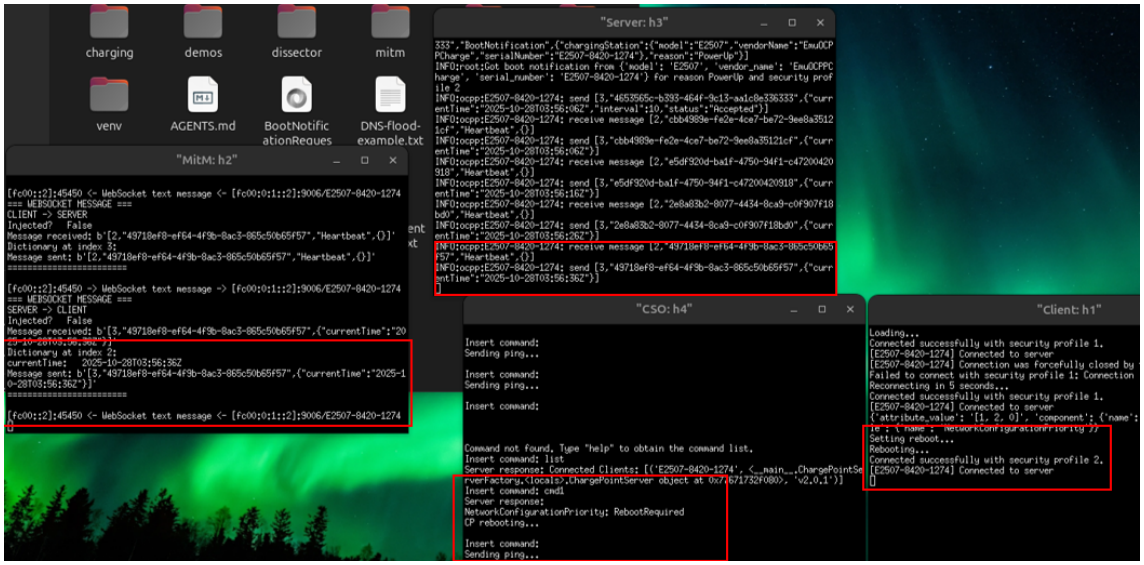



Figure 3.7: Successful SP1 to SP2 upgrade with maintained MitM interception.

Figure 3.7 demonstrates the M2 attack scenario where an attacker maintains their MitM position through an SP1 to SP2 upgrade. The three synchronized terminals reveal:

**CSO Terminal:** The operator executes the `cmd1` command to initiate the security profile upgrade. The system responds with `NetworkConfigurationPriority: RebootRequired`, triggering an automatic reboot of the charging station.

**Client Terminal:** The charging station undergoes the mandated reboot process and successfully reconnects with Connected successfully with security profile 2. This confirms the technical success of the upgrade procedure from SP1 (unencrypted) to SP2 (TLS with server authentication).

**Server Terminal:** The server logs confirm the upgrade, showing security profile 2 in the boot notification and continuing normal message exchanges. The server remains unaware that all traffic now passes through the attackers proxy.

This demonstration validates the base papers M2 claims:

- The upgrade process proceeds normally without triggering security alerts
- The attackers position established during M1 persists through the security upgrade
- Despite TLS encryption being implemented in SP2, the attackers control of certificate material (established during prerequisites) allows continued interception
- Message correlation shows the same message IDs appearing in both legitimate server logs and attacker interception, confirming persistent MitM positioning

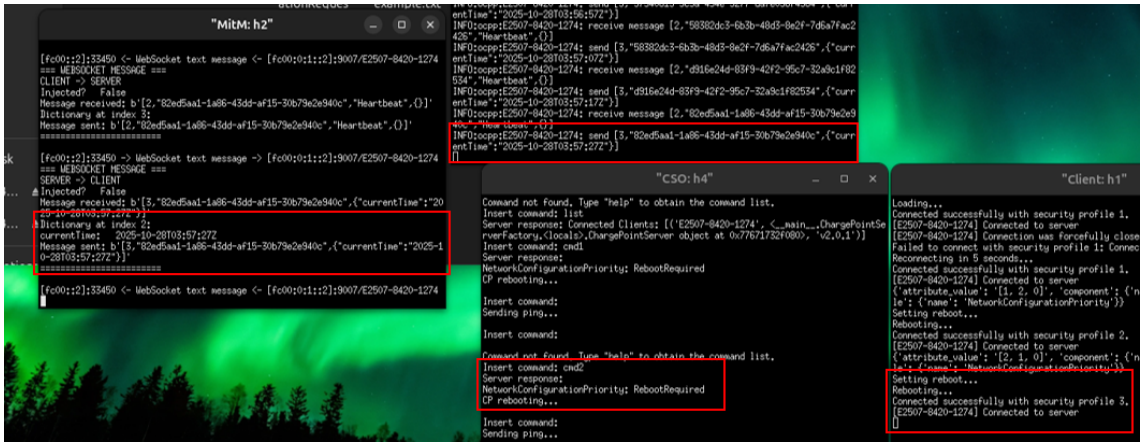



Figure 3.8: Successful SP2 to SP3 upgrade with maintained MitM interception.

Figure 3.8 completes the attack chain by demonstrating M3: persistence through an SP2 to SP3 upgrade. The terminals show:

**Upgrade Initiation:** Following successful SP2 operation, the CSO executes `cmd2` to upgrade to Security Profile 3 (mutual TLS authentication).

**Client Behavior:** The charging station again reboots as required and establishes a new connection with Connected successfully with security profile 3. This confirms implementation of the highest OCPP security level.

**Attacker Persistence:** Despite SP3s requirement for mutual authentication, the attackers terminal continues to display intercepted OCPP messages with matching message IDs, proving the MitM position remains active.

This final demonstration confirms the most critical vulnerability identified in the base paper:

- Even OCPPs strongest security profile (SP3) can be compromised by an attacker who established position during initial SP1 communication

- The certificate manipulation vulnerabilities in SP2 and SP3 upgrade processes allow attackers to obtain legitimate credentials

- Mutual TLS authentication provides no protection when the attacker controls both the client certificate enrollment process and the CSMS root certificate

- The complete attack chain (M1M2M3) demonstrates practical exploitability of theoretical vulnerabilities in real OCPP deployments

The successful demonstration of both M2 and M3 upgrades with maintained MitM access validates the papers central thesis: OCPP security mechanisms contain fundamental design flaws that allow attackers to establish persistent man-in-the-middle positions across all security profile levels.

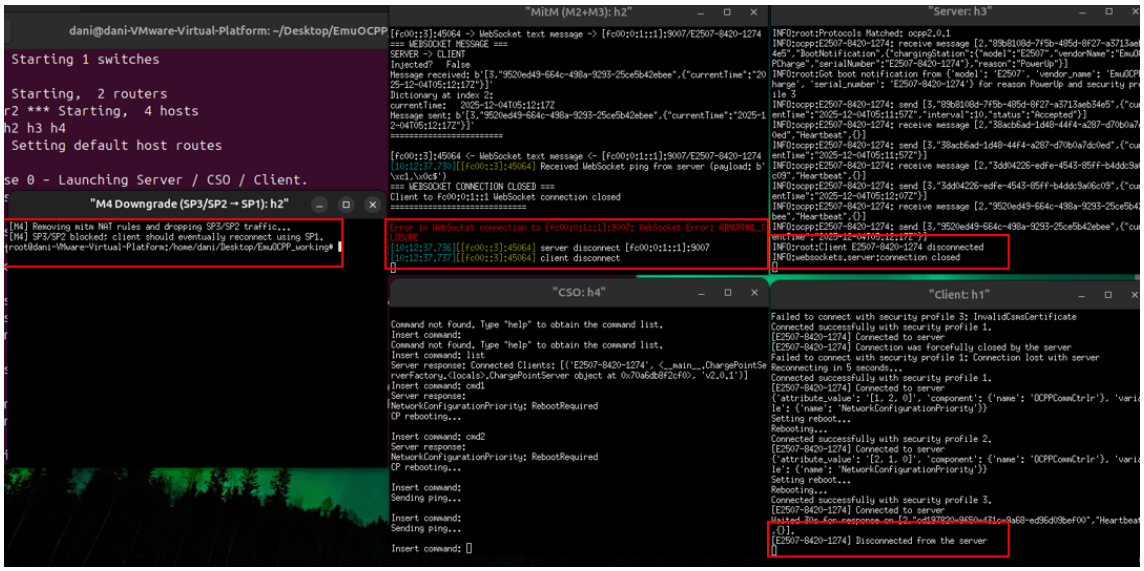



Figure 3.9: Initial phase of the M4 downgrade attack showing selective traffic blocking.

Figure 3.9 illustrates the initiation of the M4 downgrade attack. The attacker terminal (h2) executes a script specifically designed to block SP3 (port 9007) and SP2 (port 9006) traffic while maintaining SP1 (port 9005) accessibility. This selective blocking strategy forces the client to exhaust its higher-security connection attempts. The server immediately detects the disconnection as the secure channels become unreachable, logging client disconnected events without recognizing the malicious cause.

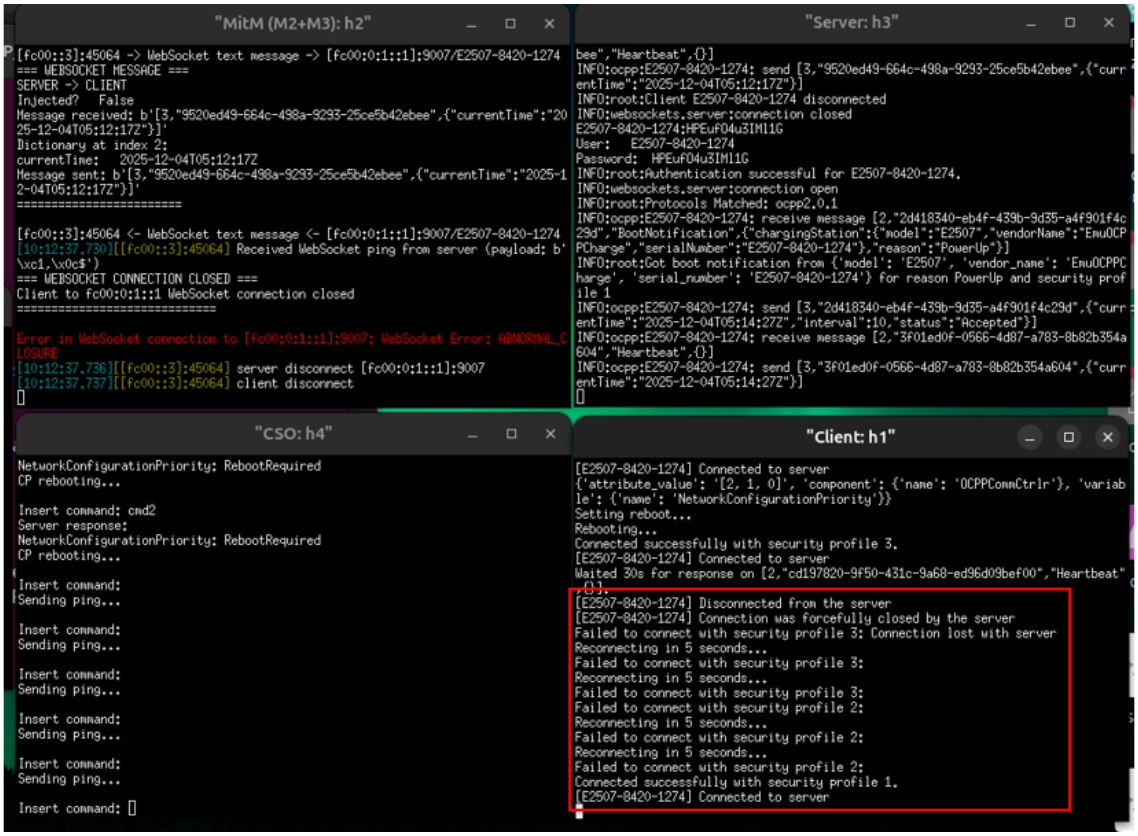



Figure 3.10: Successful downgrade from SP3 to SP1 through forced connection failures.

Figure 3.10 demonstrates the successful completion of the M4 attack. The client terminal reveals the stepwise degradation process:

**Phase 1 – SP3 Failure:** The client attempts to reconnect using Security Profile 3 (mutual TLS authentication) but fails due to the attackers blocking of port 9007. After multiple retries, the client abandons SP3 according to its failure count mechanism.

**Phase 2 – SP2 Failure:** The client automatically attempts the next priority profile (SP2, server TLS authentication) but similarly fails as port 9006 remains blocked by the attacker.
**Phase 3 – SP1 Success:** Having exhausted all secure options, the client falls back to Security Profile 1 (unencrypted) and successfully establishes a connection on port 9005. This sequence validates the base papers M4 claims:

- The clients `NetworkConfigurationPriority` setting $[0,1,2]$ dictates the connection attempt order
- Multiple security profiles remain active simultaneously, contrary to security best practices
- Selective port blocking at the network layer successfully forces a downgrade with-

out compromising TLS sessions

- The attacker regains full plaintext visibility despite the system having previously operated at the highest security level (SP3)

The successful downgrade demonstrates that even after achieving OCPPs strongest security configuration (SP3 with mutual TLS), the system remains vulnerable to regression attacks when lower-security profiles remain enabled. This confirms the implementation vulnerability described in the paper.

**Pseudocode**

**Algorithm 1:** Full MitM + Downgrade Attack (M1–M4)

**Input:** CP host $h_1$; CSMS host $h_3$; attacker host $h_2$; blocking time $\Delta t$ (e.g., 35s)
**Output:** CP downgraded from SP3/SP2 to SP1 under attacker-controlled MitM
**Initialize:** Build IPMininet topology with CP ($h_1$), attacker ($h_2$), CSMS ($h_3$), CSO ($h_4$), and configure routes between $h_1$ and $h_3$ via the backbone; start network with IPv6 addressing
**Procedure** *Full_MitM_Downgrade_Attack()*:

**1: Baseline Infrastructure (Legitimate Setup)**

Launch CSMS on $h_3$ listening on ports 9000–9007 (SP0–SP3)
After startup delay, launch CSO on $h_4$ pointing to $h_3$
After startup delay, launch CP on $h_1$ pointing to $h_3$
Wait until CP completes `BootNotification` and is authenticated

**2: M1 – Establish MitM Position**

On attacker $h_2$, prepare iptables NAT and forwarding rules
Launch NDP spoofing from $h_2$ to poison $h_1$'s neighbor cache
Launch router flooding from $h_2$ to accelerate cache takeover
Wait until all CP traffic from $h_1$ to $h_3$ is routed via $h_2$; then stop flooding while maintaining the MitM path

**3: M2 + M3 – Certificate Attacks**

Force CP disconnection by disabling forwarding on $h_2$ for $\Delta t$ so the active session times out at the CSMS
Re-enable forwarding so the CP begins reconnect attempts through $h_2$
On $h_2$, deploy transparent proxy: apply `routes.mn` redirection and launch `mitmdump` with attacker root CA and attack certificates
When CP reconnects via $h_2$:
    Intercept `InstallCertificate` to inject attacker root CA (M2)
    Intercept `SignCertificate` to inject attacker-signed leaf cert (M3)
    Capture the private key material when `combined.pem` is generated
Wait until CP operates on SP2/SP3 using attacker-controlled certificates while still routed through $h_2$

**End**

**Algorithm 1:** *

Algorithm 1 (continued): Full MitM + Downgrade Attack (M4)

**4: M4 – Enforced Downgrade to SP1**

On attacker $h_2$, remove transparent proxy routing (`rem_routes.mn`) but keep the MitM data path in place

Install firewall rules on $h_2$:

- Drop traffic to and from the SP3 port (e.g., TCP 9007)
- Drop traffic to and from the SP2 port (e.g., TCP 9006)
- Leave the SP1 port (e.g., TCP 9005) open for downgrade

Wait for CP reconnection attempts:

- CP attempts SP3 → timeout
- CP attempts SP2 → timeout
- CP falls back to SP1 and successfully authenticates

Mark the downgrade as successful and continue monitoring CP traffic on weaker SP1 while attacker retains MitM visibility

Algorithm 5 outlines a four-stage procedure that progressively degrades a legitimate OCPP session while preserving attacker visibility. The algorithm first builds the IPMininet topology and launches CSMS, CSO, and CP as part of the *"1: Baseline Infrastructure (Legitimate Setup)"* block, then waits until the CP has completed `BootNotification` and is authenticated before proceeding (lines under "Wait until CP completes BootNotification and is authenticated").

In the *"2: M1 – Establish MitM Position"* block, the attacker host enables NAT and forwarding, performs NDP spoofing, and runs router flooding from $h_2$ until all CP traffic from $h_1$ to the CSMS is routed through the attacker, at which point the flooding is stopped but the MitM path is kept in place. The *"3: M2 + M3 – Certificate Attacks"* block then forces a disconnect by disabling forwarding on $h_2$, re-enables it to capture reconnection attempts via the transparent proxy, and uses the `InstallCertificate` and `SignCertificate` interception steps to inject attacker-controlled root and leaf certificates before the CP resumes operation on SP2/SP3.

Finally, the continuation block *"4: M4 – Enforced Downgrade to SP1"* removes the proxy routing but retains the MitM data path, installs firewall rules on $h_2$ to drop traffic for the SP3 and SP2 ports while keeping the SP1 port open, and waits through the sequence of CP reconnection attempts ("CP attempts SP3 → timeout", "CP attempts SP2 → timeout"). As the CP times out on SP3 and then SP2, it ultimately falls back to SP1, successfully authenticates on the weaker profile, and continues sending OCPP traffic that remains fully observable to the attacker.

#### 3.9.1.8 I1 – CS Impersonation with Duplicate Serial Identity

**Objective**

Demonstrate that under OCPP 2.0.1 and Security Profile 1, a malicious CP can impersonate a legitimate CP by replaying its public serial number and establishing a concurrent or replacement session with the CSMS.

**Method**

The experiment proceeded as follows:

- Launched the CSMS (`server.py`) and verified listener readiness on ports 9000–9007.
- Spawned the legitimate CP, which successfully completed the OCPP 2.0.1 BootNotification handshake.
- After a controlled delay (1 s), launched the impersonator CP with the *exact same serial identity*.
- Repeated the procedure across all values of `allow_multiple_serial_numbers`: 0, 1, and 2.

For takeover demonstrations, the impersonator invoked `send_reservation_request()` to simulate downstream access to victim user operations.

**Observed behaviour**

Policy 0 – Duplicate rejected

- Server logged: "Client duplicated detected with ID E2507-8420-1274".
- Attacker BootNotification never received a `CallResult`; request timed out.
- Legitimate CP session remained uninterrupted.

Policy 1 – Both sessions accepted (true I1 condition)

- The CSMS maintained both WebSocket sessions simultaneously.
- The attacker gained full access to OCPP operations.
- This behaviour matches the paper's CSMS outcome (i)—*accept both connections*.

Policy 2 – Attacker steals the session

- CSMS closed the legitimate CP connection.

- Attacker CP became the sole active session holder.
- This reproduces the paper's outcome (ii)—*terminate victim, accept attacker*.

Reservation attempt

- The impersonator attempted a `ReserveNow` flow via the REST interface.
- Since the REST server was offline, the request timed out, but WebSocket-level impersonation had already succeeded.

**Pseudocode**

**Algorithm 2:** Charger Impersonation Attack (I1 – Parallel Sessions)

**Input:** CS identity (vendor, model, serial); attacker credentials
**Output:** Attacker maintains a session while the legitimate charger is still online
**Initialize:** Configure CSMS with chosen `allow_multiple_serial_numbers` policy;
**Procedure** *Impersonation_I1_Attack()*:
    Launch legitimate CS client using true identity;
    Wait until CSMS acknowledges `BootNotification`;
    Launch attacker client using same identity and credentials;
    **if** *policy == 0* **then**
        CSMS rejects attacker `BootNotification`;
        Legitimate CS session remains active; attack fails;
    **else if** *policy == 1* **then**
        CSMS keeps both sessions connected simultaneously;
        Attacker interacts with CSMS alongside legitimate CS;
    **else if** *policy == 2* **then**
        CSMS drops the original connection and keeps the attacker session;
        Attacker gains exclusive control over the charger identity;
    **end**
    **if** *attacker session accepted* **then**
        Trigger downstream abuse (e.g., `ReserveNow` via REST API);
        Capture responses or authorization tokens for analysis;
    **end**
**End**

Algorithm 2 describes a parallel-session impersonation attack in which the adversary reuses a legitimate charger's identity and credentials to test how the CSMS enforces its `allow_multiple_serial_numbers` policy. The procedure *Impersonation_I1_Attack()* first launches a legitimate CS client with its true identity and waits until the CSMS acknowledges `BootNotification`, ensuring that the real charger is fully registered and

online. It then starts the attacker client using the same vendor, model, serial, and credential tuple as the victim CS.

The core of the attack branches on the configured policy value. If `policy == 0`, the CSMS rejects the attacker's `BootNotification`, the legitimate CS session remains active, and the impersonation attempt fails as intended by the single-session policy. When `policy == 1`, the CSMS accepts both sessions concurrently, allowing the attacker client to coexist with the legitimate charger and interact with the backend using the stolen identity. Under `policy == 2`, the CSMS drops the original CS session and keeps only the attacker connection, effectively handing full control of the charger identity to the adversary.

In all cases where the attacker session is accepted, the final block triggers downstream abuse by invoking follow-on operations such as `ReserveNow` through the REST API and recording responses or authorization tokens. These steps demonstrate how a successful impersonation, combined with permissive session-handling policies, can be escalated into concrete reservation or billing fraud against the OCPP backend.

**Consistency with paper**

The results align precisely with EmuOCPP's description of I1:

- The CSMS responded exactly as described: accept both, steal, or reject.
- Identity-based impersonation was feasible even with SP1 enabled, consistent with the paper stating that SPs *mitigate but do not eliminate* I1.
- The attacker achieved the same downstream effects discussed in the paper (e.g., ability to receive reservation commands containing user tokens).

**Summary of Verification Outcomes**

- **I1 (CS Impersonation):** Successfully reproduced on OCPP 2.0.1 under SP1. The impersonator established a valid session despite the genuine CP staying online.
- Policy-dependent behaviour matched all three CSMS decision paths described in the paper.
- Demonstrated feasibility of session hijacking and setup for user token theft.
- Verified that identity-based flaws remain exploitable on modern OCPP versions.

**Demonstration of Attack I1 - Policy 0**

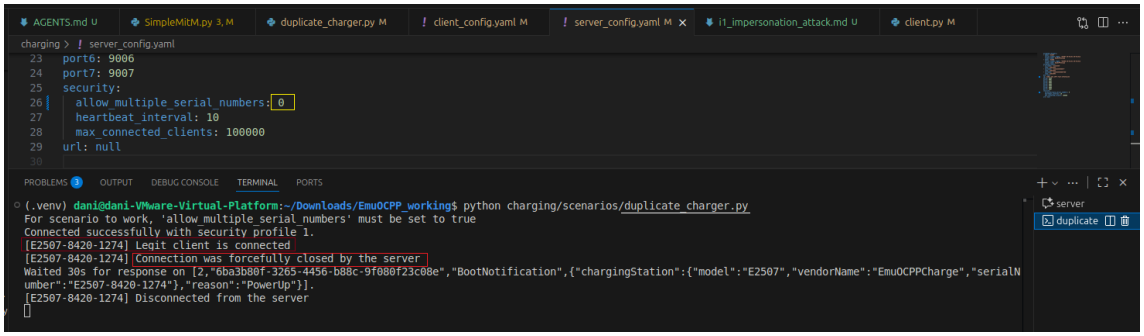



Figure 3.11: Attacker-side view of Attack I1 under server policy 0

In Figure 3.11, the yellow box highlights the pinned configuration value set to 0, while the red box shows the attackers session being immediately terminated by the server because the legitimate charge point is already active.

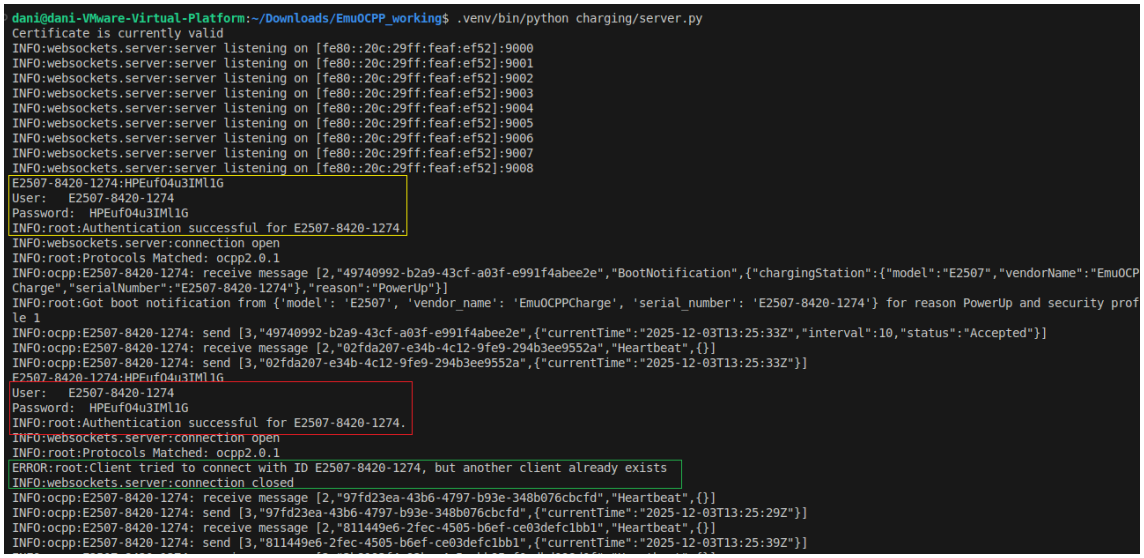



Figure 3.12: Server-side view of Attack I1 under server policy 0

In Figure 3.12, the yellow highlights the legitimate charger completing authentication, red shows the attacker replaying the same credentials, and green indicates the servers Policy-0 enforcement: the duplicate session is rejected because the original client remains active.

**Demonstration of Attack I1 - Policy 1**

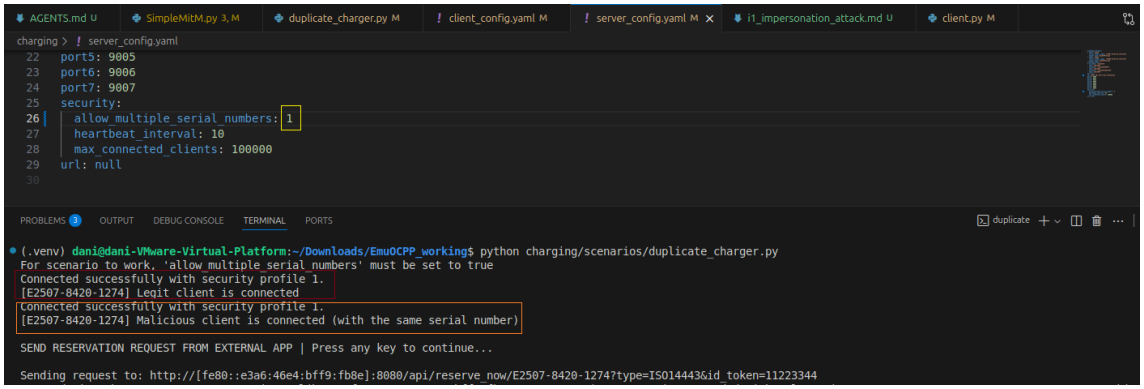



Figure 3.13: Attacker-side view of Attack I1 under server policy 1

In Figure 3.13, the yellow highlights the server policy being set to allow_multiple_serial_numbers = 1, while the red block shows the attacker joining on Security Profile 1 with the same serial, confirming that duplicates are now accepted.

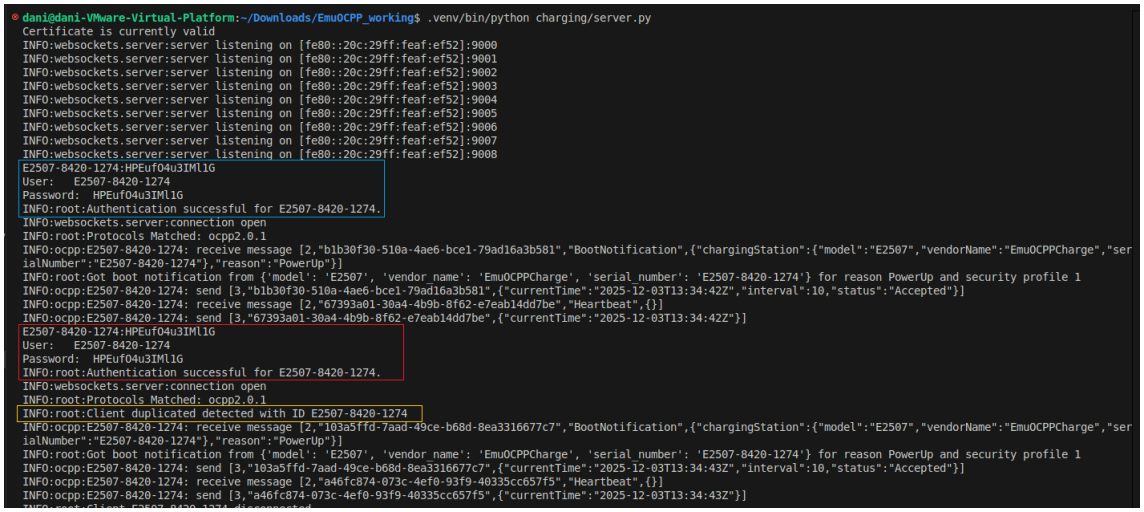



Figure 3.14: Server-side view of Attack I1 under server policy 1

In Figure 3.14, the blue outlines the genuine charger joining security profile 1, red records the attacker replaying the same credentials, and yellow highlights the server logging the duplicate client as accepted policy 1 which keeps both sessions alive despite the collision.

**Demonstration of Attack I1 - Policy 2**

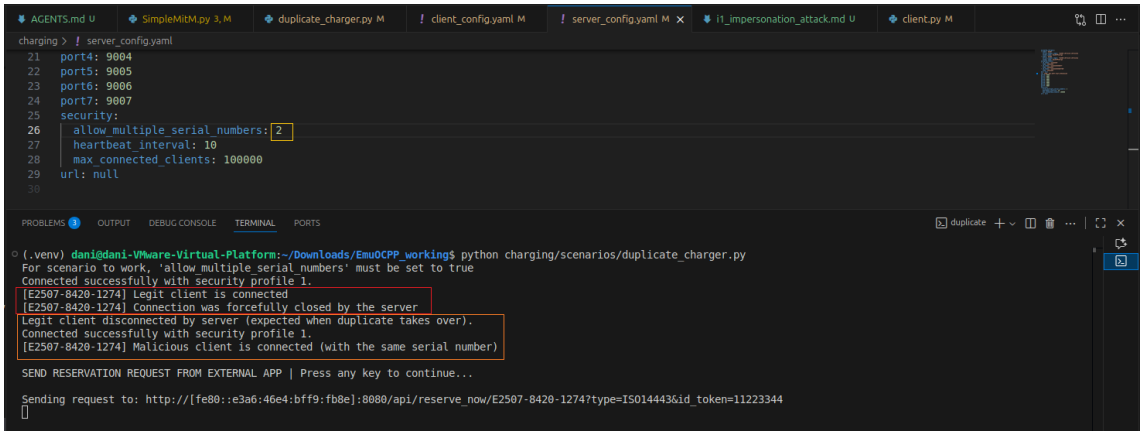



Figure 3.15: Attacker-side view of Attack I1 under server policy 2

In Figure 3.15, the yellow highlights the configuration change where allow_multiple_serial_numbers is set to 2, while the orange block captures the takeover: the attacker connects using Security Profile 1, and the server immediately terminates the legitimate charger, allowing the duplicate device to inherit the session.

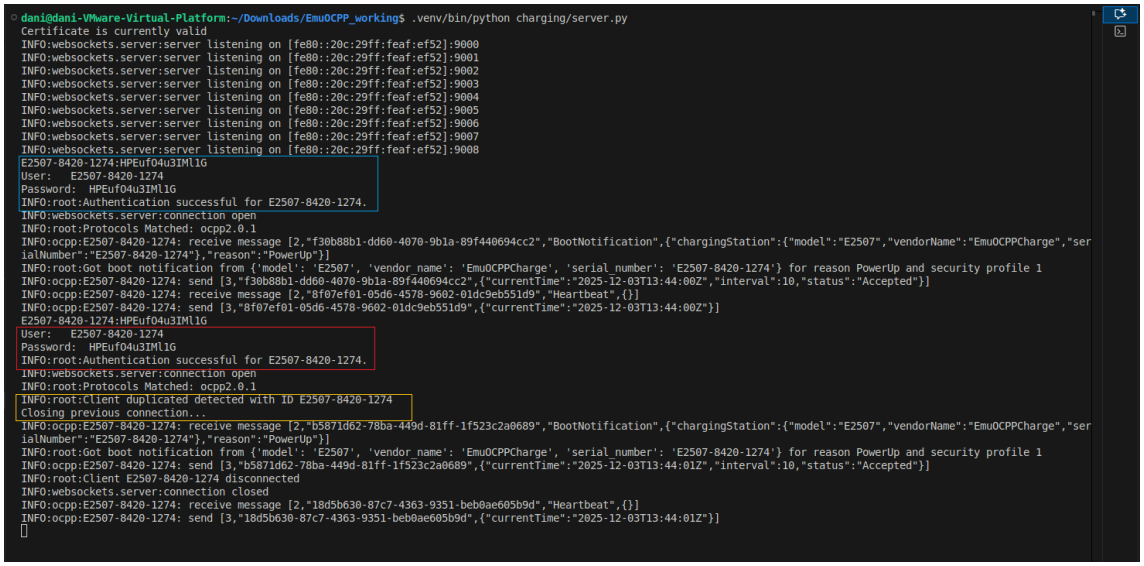



Figure 3.16: Server-side view of Attack I1 under server policy 2

In Figure 3.16, the blue marks the legitimate charger authenticating first, red captures the attacker reconnecting with the same credentials, and yellow shows the Policy 2 behavior: once a client duplication is detected, the server immediately closes the original session so the attacker keeps the connection.

#### 3.9.1.9 I2 – CS Impersonation During Reboot Window (ImpersonationI2)

Demonstrate that, under OCPP 2.0.1 and Security Profile 1, an attacker can hijack the CSMS session of a Charge Station (CS) by exploiting the brief disconnection window that occurs during a CS reboot. The goal is to validate the I2 behaviour described in the EmuOCPP paper: if the CSMS accepts only a single session per serial ID, the attacker can reconnect first and be treated as the legitimate device.

**Method**

A new scenario script (`impersonation_i2.py`) was developed to reproduce the reboot-window impersonation flow without modifying the existing I1 demo. The experiment proceeded as follows:

- The CSMS (`server.py`) was launched on ports 9000–9008, supporting all OCPP profiles and versions.
- The legitimate CS (OCPP 2.0.1, SP1) connected first using serial number `E2507-8420-1274`.
- Upon operator prompt, the script closed the CS connection using WebSocket code 1012 ("Simulated charger reboot").
- After a configurable delay (default: 2 s), the attacker client reconnected using the *same* vendor/model/serial identity.
- Because `allow_multiple_serial_numbers = 0` was enforced on the CSMS, the attacker was accepted as the "returning" charger.
- After takeover, the attacker attempted a `ReserveNow` request via the REST panel, mirroring the I1 workflow.

**Pseudocode**

**Algorithm 3:** Charger Impersonation Attack (I2 – Reboot Window Seizure)

**Input:** CS identity; attacker credentials; wait interval $\Delta t$

**Output:** Attacker monopolizes reconnection slot after legitimate CS reboot

**Initialize:** Configure CSMS with `allow_multiple_serial_numbers = 0`;

**while** *scenario is running* **do**

  Launch legitimate CS client and confirm active session;

  Prompt operator (or detect) to disconnect legitimate CS (simulate reboot);

  Wait $\Delta t$ seconds to ensure CSMS releases the session;

  Launch attacker client using same serial and credentials;

  **if** *attacker connects before legitimate CS returns* **then**

    CSMS registers attacker as rebooted charger;

    Legitimate CS is rejected on subsequent reconnect attempts;

    Attacker triggers `ReserveNow` or other privileged flows;

  **else**

    Repeat timing procedure until takeover succeeds;

  **end**

**end**

**End**

Algorithm 3 captures the reboot-window impersonation variant, where the attacker seizes the first reconnection slot after a legitimate charger is forced offline. The CSMS is configured with `allow_multiple_serial_numbers = 0`, so only one session per charger identity can be active at any time. Inside the *Impersonation_I2_Attack()* loop, the operator first launches a legitimate CS client, confirms that the session is active, and then issues (or detects) a disconnect to simulate a reboot of the real charger.

After waiting for the configured interval $\Delta t$ so that the CSMS fully releases the session, the attacker starts a second client using the same identity and credentials as the victim CS. If the attacker connects before the legitimate CS comes back, the CSMS registers this session as the rebooted charger, any subsequent reconnect from the real device is rejected, and the adversary can trigger privileged operations such as `ReserveNow` via the REST API. Otherwise, the `else` branch repeats the timing cycle, allowing the experimenter to adjust $\Delta t$ and attack launch time until the takeover succeeds.

**Observed behaviour**

The practical behaviour matched the I2 vulnerability description:

- The legitimate CS successfully established the initial BootNotification and heartbeat cycle.
- When the operator triggered the “reboot,” the client terminated with code 1012 and

the CSMS logged a clean disconnect.

- During the reboot window, the attacker connected using the same identity and received an `Accepted` response to its BootNotification.
- Heartbeat traffic continued from the attacker, confirming full takeover of the CSMS session.
- When the legitimate CS attempted to reconnect, the CSMS rejected the attempt because the serial ID was already bound to the attacker session.

This behaviour matches the papers statement that, under I2, the CSMS registers the first connection it receives for that serial ID, allowing an attacker to steal the reconnection slot.

**Consistency with paper**

The results are fully consistent with the I2 behaviour described in Section 6.9 of the EmuOCPP paper:

- I2 works even when the CSMS only allows a single session per serial ID (`allow_multiple_serial_numbers = 0`).
- The attacker must wait for the legitimate CS to go offline in this test, the script simulated a firmware reboot.
- The CSMS accepted the attacker because serial numbers are static, public, and unauthenticated under SP1.
- The legitimate CS was unable to restore its session once the attacker occupied the reboot slot.

Thus, the EmuOCPP implementation accurately reproduced the vulnerability class, including timing dependence and session-binding semantics.

**Summary of Verification**

- The attacker successfully impersonated the CS during a reboot interval.
- The CSMS treated the attacker as the legitimate device after accepting the first reconnecting BootNotification.
- The real CS remained locked out until the attacker disconnected.
- The attack enabled full access to downstream CSMS commands (e.g., `ReserveNow`) once the REST API is running.

- The outcome directly validates the papers claim that I2 persists across all SP levels due to weak identity binding in OCPP.

### Demonstration of Attack I2

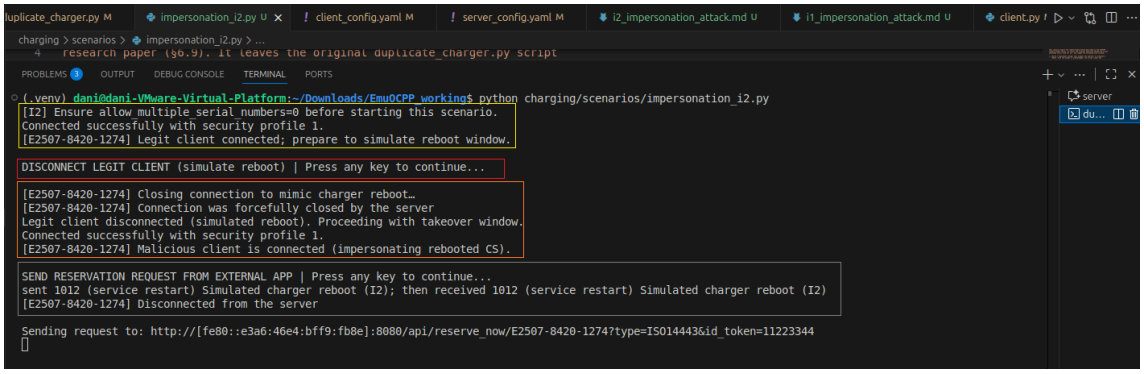



Figure 3.17: Attacker-side view of Attack I2

In Figure 5.16, the yellow highlights the operator prompts confirming that Policy 0 is enforced and that the legitimate charger is connected before the reboot window. The red block shows the simulated reboot step during which the defender is disconnected, allowing the attacker to reconnect under the same identity.

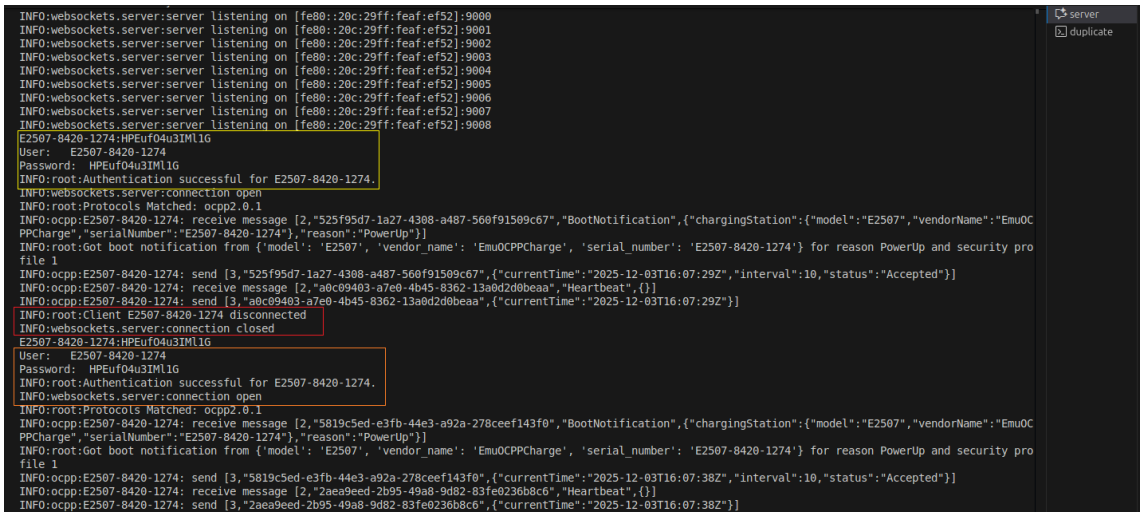



Figure 3.18: Server-side view of Attack I2

In Figure 5.17, The yellow highlight marks the legitimate charger authenticating under its own ID, the red box shows the server forcibly disconnecting that original session during

the reboot window, and the orange highlight records the attacker reconnecting immediately afterward with impersonated credentials, completing the takeover.

## 3.10 Analysis and/or Discussion

This section provides an analytical discussion based on the experimental results obtained from replicating the attack scenarios described in the EmuOCPP base research paper. The analysis focuses on the verification of the novel exploits (M1–M4 and I1–I2), the consistency of our findings with existing literature, the root causes enabling successful attacks, and the implications for securing real-world OCPP deployments.

### 3.10.1 Verification of Man-in-the-Middle Attacks

The four MitM attacks (M1–M4) described in the EmuOCPP paper were successfully reproduced in our testbed using open-source OCPP implementations, Mininet-based topologies, and packet inspection tools. The results confirm the validity and reproducibility of these attack vectors under realistic conditions.

#### 3.10.1.1 M1: Insecure Security Profile (SP1)

Attack M1 demonstrated that when OCPP communication relies on Security Profile 1 (SP1), all traffic is transmitted in plaintext, allowing an adversary to capture and modify messages with no encryption or authentication. Data such as identifiers, authorization requests, and heartbeat messages were intercepted in our environment, aligning with the base paper's findings that SP1 should not be used in production environments.

#### 3.10.1.2 M2: Security Profile Upgrade (SP2)

In attack M2, we replicated a scenario where an adversary installs a forged or attacker-controlled root certificate during the Security Profile 2 upgrade process. The charging station accepted this certificate when no certificate validation checks were enforced at the device. This confirmed that improper handling of certificate provisioning during upgrades exposes otherwise encrypted OCPP channels to MitM manipulation.

#### 3.10.1.3 M3: Mutual-TLS Certificate Abuse (SP3)

Attack M3 involved intercepting or replacing the Certificate Signing Request (CSR) generated by the charge point during the upgrade to SP3. By altering the CSR, the adver-

sary was able to obtain long-term mutual-TLS credentials. This experiment validates the EmuOCPP claim that SP3 is not inherently secure unless certificate lifecycle management (issuance, renewal, and revocation) is rigorously enforced.

#### 3.10.1.4 M4: Security Profile Downgrade

Attack M4 utilized selective packet dropping and reconnection forcing to downgrade active secure connections back to SP1 or SP2. By removing or delaying WebSocket upgrade packets, we successfully forced the charging station to reconnect at a lower security profile. This result highlights a critical protocol weakness: the OCPP specification does not mandate strict enforcement of the highest available security profile once supported, making downgrade attacks feasible.

### 3.10.2 Verification of Impersonation Attacks

Two identity-based attacks, I1 and I2, were successfully replicated and validate a major finding of the EmuOCPP base paper: OCPP does not enforce unique or cryptographically bound charge point identifiers.

#### 3.10.2.1 I1: Concurrent Identifier Impersonation

In I1, we impersonated a legitimate charging station by establishing a parallel connection using the same identifier. Depending on the CSMS implementation, three behaviours were observed: (1) multiple parallel sessions being accepted, (2) the legitimate connection being replaced, (3) or subsequent connections being rejected while still exposing traffic metadata. These variants are consistent with those reported in the base paper and represent a direct risk for session hijacking and fraudulent message injection.

#### 3.10.2.2 I2: Offline Timing Impersonation

Attack I2 exploited intervals where a legitimate charging station was offline or rebooting. Once offline, the adversary connected first using the same identifier and was then treated as the legitimate device. This attack bypassed duplicate session protection used against I1 and confirms that identifier reuse is a protocol-level weakness that persists across multiple OCPP versions.

### 3.10.3 Root Cause Analysis

A significant outcome of the experimental investigation is that the successful exploitation of all attack scenarios did not require cryptographic breaking, firmware compromise, or low-level vulnerabilities. Instead, the root causes of the attacks stem from protocol assumptions and configuration design, including:

- Absence of identity binding between device identifiers and certificates.
- Weak or undefined rules for security profile transitions.
- Insufficient enforcement of certificate validation and authenticity.
- Lack of mandatory uniqueness checking for charge point identifiers.

These weaknesses are structural and protocol-agnostic, meaning that they are likely present in multiple OCPP implementations, regardless of vendor-specific software quality.

### 3.10.4 Implications for Real-World Deployments

The successful replication of M1–M4 and I1–I2 demonstrates severe real-world risks for EV charging networks:

- TLS is insufficient on its own if certificate provisioning and validation processes are not securely enforced.
- Downgrade attacks remain viable where multiple connection profiles are enabled simultaneously.
- Identity-based attacks enable session hijacking, fraudulent billing, and denial of legitimate service.
- OCPP standardization is required to enforce uniform certificate management, identifier uniqueness, and upgrade behaviour.

Overall, certificate governance, binding identifiers to cryptographic material, and strict upgrade authentication requirements represent urgent areas for improvement in both the OCPP specification and its industrial implementations.

### 3.10.5 Summary

The experimental results confirm that the novel attacks introduced in the EmuOCPP paper are reproducible using openly available tools. The consistency of our outcomes with previously published results, combined with new observations from packet-level inspection, demonstrate that OCPP vulnerabilities exist at a fundamental design level. These findings underscore the need for improved protocol controls to prevent downgrade manipulation, identity impersonation, and certificate abuse in EV charging networks.

# Chapter 4

# Theoretical Framework

## 4.1 Understanding Packet Inspection

### 4.1.1 Overview

Packet inspection in this project uses Wireshark with custom Lua dissectors to capture, parse, and validate OCPP traffic. The dissectors decode OCPP messages, validate them against JSON schemas, and flag protocol violations and compliance issues. [1]

### 4.1.2 Core Components

#### 4.1.2.1 Wireshark Dissector Framework

The project includes custom Wireshark dissectors written in Lua that extend Wireshark to parse OCPP [10, 13]. Two installation modes are supported:

- **Single Dissector Mode:** `ocppDissector.lua` provides unified parsing for OCPP 1.6, 2.0, and 2.0.1, registering a single "OCPP" protocol in Wireshark.
- **Multiple Dissector Mode:** Separate dissectors (`ocpp16Dissector.lua`, `ocpp20Dissector.lua`, `ocpp201Dissector.lua`) register distinct protocols per version, enabling version-specific analysis.

The dissectors register with Wireshark's WebSocket protocol table using subprotocol identifiers:

- `ocpp1.6` for OCPP version 1.6

- `ocpp2.0` for OCPP version 2.0
- `ocpp2.0.1` for OCPP version 2.0.1

#### 4.1.2.2 Packet Capture Process

Packet capture occurs at multiple layers:

- **Network Layer Capture:** During attack scenarios (M1–M4), traffic between Charge Points and CSMS is captured using Wireshark on network interfaces within the Mininet/IPMininet topology. [4]
- **WebSocket Frame Extraction:** The dissectors operate on WebSocket frames containing OCPP JSON messages. Wireshark's built-in WebSocket dissector extracts the payload, which is then passed to the OCPP dissector based on the `Sec-WebSocket-Protocol` header.
- **Pre-captured Traces:** The `dissector/` directory includes sample `.pcapng` files (`capt.pcapng`, `capt_error.pcapng`, `reservation.pcapng`, etc.) for testing and validation.

#### 4.1.2.3 Message Parsing and Structure Analysis

The dissector parses OCPP messages structured as JSON arrays:

```
[MessageType, MessageID, MessageName, Payload]
```

Where:

- **MessageType:** 2 (Call/Request), 3 (CallResult/Response), or 4 (CallError)
- **MessageID:** Unique identifier for request-response correlation
- **MessageName:** OCPP action name (e.g., BootNotification, Authorize, MeterValues)
- **Payload:** JSON object containing message-specific data

The parsing process includes buffer extraction, JSON array parsing, element extraction, and JSON payload decoding into Lua table structures.

#### 4.1.2.4 Schema-Based Validation

The dissector validates OCPP messages against official JSON schemas:

- OCPP 1.6: `ocpp/v16/schemas/`
- OCPP 2.0: `ocpp/v20/schemas/`
- OCPP 2.0.1: `ocpp/v201/schemas/`

Validation results may include:

- *All*, *1.6/2.0*, *2.0/2.0.1*
- *1.6*, *2.0*, *2.0.1*
- *None* (invalid for all versions)

Validation errors are displayed using Wireshark's expert information system.

#### 4.1.2.5 Protocol Tree Display

The dissector builds a hierarchical protocol tree in Wireshark:

- Root node: "OCPP Protocol Payload"
- Message type, ID, and name fields
- Recursive JSON payload tree with subtrees for nested objects

#### 4.1.2.6 Expert Information and Compliance Checking

Version-specific dissectors include expert information fields for:

- IPv6 compliance checking
- Protocol validity and schema non-compliance

Wireshark coloring rules highlight:

- Non-compliant packets (red)
- IPv4 usage warnings (yellow)
- Protocol anomalies (orange)

#### 4.1.2.7 Installation and Configuration

Installation is automated using a Makefile:

```
make install-single     # Installs unified dissector
make install-multiple  # Installs version-specific dissectors
```

The installation process creates the plugin directory, copies Lua dependencies, installs dissector files, and registers them with the WebSocket protocol table.

#### 4.1.2.8 Integration with Attack Scenarios

Packet inspection supports attack verification through:

- Real-time monitoring using Wireshark
- Post-attack analysis of captured traces
- Schema-based compliance checking of manipulated traffic

#### 4.1.2.9 Limitations and Considerations

- Schema path dependency tied to the virtual environment
- Limited CallResult correlation
- Verbose debugging output may affect high-traffic capture clarity

### 4.1.3 Practical Usage

1. Install dissector: `cd dissector && make install-single`
2. Start Wireshark capture
3. Filter: `websocket && ocpp` or `tcp.port == 9000-9007`
4. Inspect parsed protocol tree
5. Review expert information for compliance violations

This packet inspection framework enables visibility into OCPP communications, validation of protocol compliance, identification of security vulnerabilities, and verification of attack scenarios within the EmuOCPP environment. [2]

## 4.2 EmuOCPP Framework Role and High-Level Overview

EmuOCPP is the primary experimental environment used in this research to analyse the security properties of Open Charge Point Protocol (OCPP) implementations. [1] It provides a container-based, IPMininet-driven emulation of charging stations and central systems capable of executing OCPP 1.6 and OCPP 2.x message flows across multiple security profiles under realistic network conditions. Within this work, EmuOCPP serves as the reference implementation to replicate base-paper attacks, extend support to OCPP 2.1, and evaluate protocol-level security behaviour systematically.

The workspace is structured as a self-contained laboratory. Core services reside in `charging/`, downgrade and MitM tooling in `mitm/`, protocol dissectors in `dissector/`, topology and network launch files in `charging/ipmininet/`, automated scenarios in `charging/scenarios/`, and environment configuration plus helper utilities in `scripts/`. Additional artefacts such as packet captures, demonstration video traces, and example certificates are also provided for reproducibility and research extension.

Across the thesis, EmuOCPP fulfils three major roles: (1) it enables reproduction of documented exploits for baseline verification [3, 4], (2) it provides an extensible research platform where OCPP 2.1 functionalities can be implemented, and (3) it reveals architectural limitations that directly guide SafeCharge's improvement objectives, including automation, credential isolation, structured observability, and regression-based test validation.

## 4.3 Software Architecture and Core Services

The core application logic resides in the `charging/` directory, implementing both server-side and client-side functionality along with REST APIs, certificate mechanisms, operator tooling, and local database services.

### 4.3.1 Server and Certificate Authority Components

The central OCPP server is implemented in `charging/server.py`. It exposes multiple WebSocket listeners configured across OCPP 1.6 and OCPP 2.x, covering security profiles 03. Each listener binds to separate ports using appropriate TLS configurations. For example, TLS+Basic Authentication enables encryption without strict verification, whereas TLS+Client-Certificates enforces mutual authentication anchored to a trusted local root authority.

Server configuration is defined in `server_config.yaml`, specifying accepted charger

IDs, authentication policies, and mandatory security profiles. Charger admission is governed by serial-number regex rules, duplicate-identifier policies, and token allow-listing. A local operator console (port 9008) supports certificate installation, event inspection, and interactive controlsufficient for laboratory work but insufficient for production-level access governance. [7]

Certificate issuance is managed via `CA_CertificateCreation.py`, `certificateCreation.py`, utilising a repository-wide trust root (`emuocpp_ttp_cert.pem`, `emuocpp_ttp_key.pem`). Issued credentials are stored under `charging/installedCertificates/`, separated into per-charger and central-server sub-directories. Certificates remain in volatile memory until event-finalisation, when they are flushed to disk without persistent database retention. [13]

### 4.3.2 Charge Point Client and Management API

The simulated charge point is implemented in `charging/client.py`. Runtime behaviour is configured via `client_config.yaml`, with optional DNS bootstrap and negotiated security profile selection. Overrides issued through command-line flags persist into the YAML sourceuseful for reproducibility but prone to configuration drift during exploratory testing.

Authentication depends on the discovered security profile: Basic / TLS-Basic HTTP Basic Auth using config-file credentials TLS-Client-Certificate certificate + private-key loaded from `installedCertificates/<serial>/`

A lightweight REST API built using Flask is served through `charging/api_server.py` and consumed by `charging/api_client.py`. The API provides credential management, password updates, user enrolment, event retrieval, and reservation interactions. Storage is provided by `charging/db.sqlite3`. Intentional SQL-injection behaviour is retained for attack-scenario reproducibility, requiring caution when conducting defensive experiments.

### 4.3.3 Data Model, Configuration and Credentials

The SQLite store maintains users and event logs. Fast-flush behaviour simplifies prototyping but eliminates long-term forensics unless external logging is incorporated. API responses return unstructured tuple-strings rather than JSON objects, offloading parsing overhead to clients.

Critical configuration files include: `server_config.yaml` `client_config.yaml` `charging/ipmininet/topologies/customTopo_config.yaml`

These encode IPv6 addressing, interface bindings, protocol profiles, test credentials, and acceptance rules. Credentials bundled within the repository are demonstration-only and must be regenerated for realistic threat-model consistency.

## 4.4 Networking, Scenarios and Adversarial Tooling

EmuOCPP operates without external hardware by using IPMininet to emulate routers, switches, clients, and servers. Scenario scripts enable automated attack flows, while a dedicated MitM subsystem facilitates downgrade exploitation.

### 4.4.1 Network Emulation and Topology Management

The network is launched through `charging/ipmininet/launch.py`, which imports topologies from `ConfigTopo.py` and `SimpleTopo.py`, parameterised using YAML structures such as `customTopo_config.yaml`. The launcher provisions IPv6 hosts, generates charger identities, initialises DB entries, installs certificates, configures DNS (`charging/dnsServer.py`), and establishes communication channels. Privileged execution may be required (DNS/port 53 binding).

Lifecycle teardown partially automates cleanup, though asynchronous tasks may leave residual sockets or artefacts during interrupted executionimportant for reproducibility and future regression-test automation.

### 4.4.2 Scenario Scripts and Automated Experiment Flows

Scenario automation resides in `charging/scenarios/`. Scripts demonstrate baseline operations (normal charge), adversarial behaviour (DoS floods, malicious registration, token abuse), and SQL based bypass. [8] Many rely on console log output rather than assertive machine readable verdicts, limiting regression test integration. Address duplication, outdated helper signatures, and configuration scattering represent opportunities for SafeCharge improvement.

### 4.4.3 MitM Tooling and Downgrade Attacks

The `mitm/` subsystem executes downgrade and TLS interception attacks via `mitm.py` integrated with `mitmdump`. [4] Scripts such as `downgrade.py` and `SimpleMitM.py` modify routing flows using `routes.mn`, `rem_routes.mn`, and `prep.sh`. These assume fixed interface names and rely on neighbour spoofing tools such as `parasite6`. Mishandled tear

down may leave network state modified, making automated cleanup a forward looking engineering target.

`mitm/certificates/` contains example attacker keys and composite PEM bundles useful for repeatability, unsuitable for secure deployment.

### 4.4.4 Protocol Dissector and Packet Analysis

Wireshark integration lives in `dissector/`. Main logic resides in `ocppDissector.lua` with version specific variants: `ocpp16Dissector.lua` `ocpp20Dissector.lua`

`ocpp201Dissector.lua`

Limitations include hard coded schema paths, missing helper modules, and strict parsing of `CallResult` frames. Improving dissector robustness is a required step for OCPP 2.1 validation under SafeCharge. [1]

## 4.5 Comparative Security Analysis of OCPP 2.0.1 and OCPP 2.1

### 4.5.1 Methodology

The comparison between OCPP 2.0.1 [13] and OCPP 2.1 [15] is based on a detailed, document-driven analysis of the official OCPP Introduction (Part 0) and Architecture/Topology (Part 1) specifications. OCPP 2.0.1 serves as the baseline, defining the security and architectural foundation, including functional blocks, use case structure, and the Device Model. OCPP 2.1 was then examined to identify documented changes, extensions, and refinements affecting security, provisioning, authorization, and monitoring functionality.

The analysis is organized around four key security dimensions:

1. **Core security foundations:** Security Profiles, secure firmware updates, and event logging.
2. **Certificate lifecycle and ISO 15118 integration:** Certificate issuance, updates, and Plug & Charge support.
3. **Authorization and identity handling:** IdToken structure and supported token types.
4. **Provisioning and monitoring security:** Network profiles, diagnostics, and Device Model monitoring.

For each dimension, OCPP 2.0.1 is treated as the baseline. OCPP 2.1 enhancements are classified as structural extensions, behavioural refinements, or newly allowed transitions. Architectural constructs such as the three-tier Charging StationEVSEConnector model and the Device Model are used to interpret deployment and operational implications.

## 4.5.2 Findings

### 4.5.2.1 Security Foundations of OCPP 2.0.1

OCPP 2.0.1 introduces a dedicated Security functional block (A) with three Security Profiles for authentication and secure communication between Charging Stations and the CSMS. [13] It provides client-side certificate management, secure firmware updates, and a Security Event Notification use case (A04). These features, combined with Diagnostics and Monitoring capabilities, enable the reporting and management of security relevant events.

The Device Model in OCPP 2.0.1 represents a Charging Station as a set of Components (e.g., ChargingStation, EVSE, Connector, TokenReader, Controller) with associated Variables and Attributes. This structure supports configuration, status monitoring, and security event generation.

### 4.5.2.2 Security Enhancements in OCPP 2.1

OCPP 2.1 extends 2.0.1, retaining the core Security Profiles, secure firmware update functionality, the Security Event Log, and the Device Model. [15] Key security enhancements include:

- **Certificate management:** `SignCertificateRequest` now includes a `requestId` to enable reliable request correlation.
- **Security Profiles:** Use case A05 allows controlled downgrade from Profile 3 to 2.
- **ISO 15118 support:** Explicit support for ISO 15118-20 certificates and updated flows for use cases C07, C08, and M01.
- **Authorization and identity:** Maximum `IdToken` length increased to 255 characters; type is now extensible.
- **Provisioning and network profile security:** `SetNetworkProfileRequest` extended with `basicAuthPassword` and `identity` fields to improve secure network configuration.

- **Monitoring and diagnostics:** `NotifyEventRequest` now includes a *severity* field. New monitoring types *TargetDelta* and *TargetDeltaRelative* added in the Device Model.
- **Device Model hierarchy:** New variables (CommunicationParent, ElectricalParent, LogicalParent, PhysicalParent, Label) support secure management of complex installations.

### 4.5.3 Evaluation

OCPP 2.0.1 provides a comprehensive security and architectural foundation. OCPP 2.1 builds on this without introducing new transport or cryptographic layers, focusing on:

- Improving traceability (e.g., request correlation in certificate management, event severity levels).
- Extending ISO 15118 certificate support.
- Refining monitoring and Device Model observability.

A well-configured OCPP 2.1 deployment maintains or improves the security posture compared to 2.0.1. [6] Core mechanisms such as certificate based authentication, secure firmware updates, structured monitoring, and functional-block-based use cases remain stable, while new extensions provide richer security observability and operational flexibility.

### 4.5.4 Comparison Table

| **Criteria** | **OCPP 2.0.1** | **OCPP 2.1** |
|---|---|---|
| Security functional block and profiles | Defines Security functional block (A) with three Security Profiles, client-side certificate management, secure firmware updates, and Security Event Notification (A04). | Retains Security functional block; adds use case A05 for controlled Security Profile changes, including downgrade from Profile 3 to 2. |

| **Criteria** | **OCPP 2.0.1** | **OCPP 2.1** |
|---|---|---|
| Certificate lifecycle and correlation | Supports certificate updates via CSMS or Charging Station (A02, A03); ISO 15118 certificate management in block M. | Extends `SignCertificateRequest` with `requestId` to correlate with `CertificateSigned`; adds ISO 15118-20 certificate support and updated M01 flows. |
| ISO 15118 integration | Supports ISO 15118-1 for Plug & Charge and secure EVSE–EV communication; includes ISO 15118 Certificate Management block. | Extends integration to ISO 15118-20; updates C07/C08 authorization and M01 certificate installation flows. |
| Authorization and identity | Supports multiple authorization methods using `IdToken` abstraction. | Increases `IdToken` length to 255 characters and makes type extensible. |
| Provisioning and network profile security | Supports provisioning and migration of network profiles (B09, B10). | Enhances B09 by adding `basicAuthPassword` and `identity` fields in `SetNetworkProfileRequest`. |
| Monitoring, diagnostics, and events | Provides Monitoring block (N), Device Model monitoring, and security event notifications. | Adds severity field to `NotifyEventRequest` (N07); adds new delta monitoring types. |
| Device Model and hierarchy | Defines Device Model with Components, Variables, Attributes. | Adds hierarchy variables (CommunicationParent, ElectricalParent, LogicalParent, PhysicalParent, Label). |

Table 4.1: Key differences between OCPP 2.0.1 and OCPP 2.1, focusing on security, provisioning, monitoring, and identity.

The table 4.1 highlights that OCPP 2.1 enhances the security and architectural model of OCPP 2.0.1 by improving traceability, monitoring, certificate management, and ISO 15118-20 integration, while maintaining the foundational security constructs.

# Chapter 5

# Experiments and Results

This chapter outlines the details of the experiments conducted, and the experimental set-up.

## 5.1 Experimental Setup

The experimental setup must be meticulously detailed to ensure reproducibility of the results.

### 5.1.1 Context of the OCPP 2.1 Upgrade

OCPP 2.1 introduces and revises several use cases that extend the protocol's surface area beyond what was supported by OCPP 2.0.1. To exercise the vulnerabilities discussed later in this chapter, EmuOCPP was upgraded so that the simulated Charging Station and CSMS can both initiate and respond to these use cases at the message level. The following subsections summarise, for each affected use case, the objective taken from the OCPP 2.1 specification, the scenario it defines, and the corresponding sequence diagram. Where an existing use case has been revised in the new version, only the changes relevant to the upgrade are highlighted.

#### 5.1.1.1 A02 – Update Charging Station Certificate by request of CSMS

**Objective.** A02 defines the procedure by which the CSMS initiates an update of the Charging Station's client-side certificate. The objective, as stated in the specification, is to provide a managed certificate update mechanism so that the Charging Station's identity material can be rotated on demand, with operator intervention reserved for error recovery rather than for routine rotation.

**Updates in OCPP 2.1.** The use case has been extended to support multiple certificate types beyond the original Charging Station certificate. The `TriggerMessageRequest` now accepts the `requestedMessage` values `SignChargingStationCertificate`, `SignV2GCertificate`, and `SignV2G20Certificate`, the latter two corresponding to ISO 15118 certificates. When a Charging Station exposes multiple ISO 15118 controllers (`ISO15118Ctrlr`, equivalent to the SECC in ISO 15118), the CSMS must issue a separate `TriggerMessageRequest` per controller, identified at either top level or per-EVSE depending on the device model. EVSEs sharing the same physical SECC are grouped by reporting an identical `SeccId`.

**Scenario.** The standard `SignChargingStationCertificate` flow proceeds as follows:

1. The CSMS sends a `TriggerMessageRequest` with `requestedMessage = SignChargingStationCertificate`.
2. The Charging Station replies with a `TriggerMessageResponse`.
3. The Charging Station generates a new public/private key pair locally.
4. The Charging Station sends a `SignCertificateRequest` to the CSMS containing the CSR and `certificateType = ChargingStationCertificate`.
5. The CSMS responds with `SignCertificateResponse` (status `Accepted`).
6. The CSMS forwards the CSR to the Certificate Authority Server.
7. The CA signs the certificate and returns it to the CSMS.
8. The CSMS delivers the signed certificate via `CertificateSignedRequest`.
9. The Charging Station verifies the certificate and replies with `CertificateSignedResponse` (`Accepted` or `Rejected`).

The alternative `SignV2GCertificate` flow inserts a discovery step: the CSMS first issues a `GetReportRequest` for the variable `ISO15118Ctrlr.SeccId`, then iterates the same procedure once per unique `SeccId`, with the CSR's CommonName set to the corresponding `SeccId` and a `hashRootCertificate` element selecting the CA.

**Relevance to EmuOCPP.** A02 is the baseline certificate enrollment workflow required by Security Profiles SP2 and SP3. Because EmuOCPP is centred on multi-profile security behaviour and attack scenarios, omitting A02 would reduce SP3 testing to static-certificate demonstrations rather than realistic enrollment and rotation flows. The upgrade therefore implements the functional spine of A02 end-to-end, while deliberately scoping out parts of the specification that are not relevant to the threat model evaluated in this thesis. The breakdown is as follows:

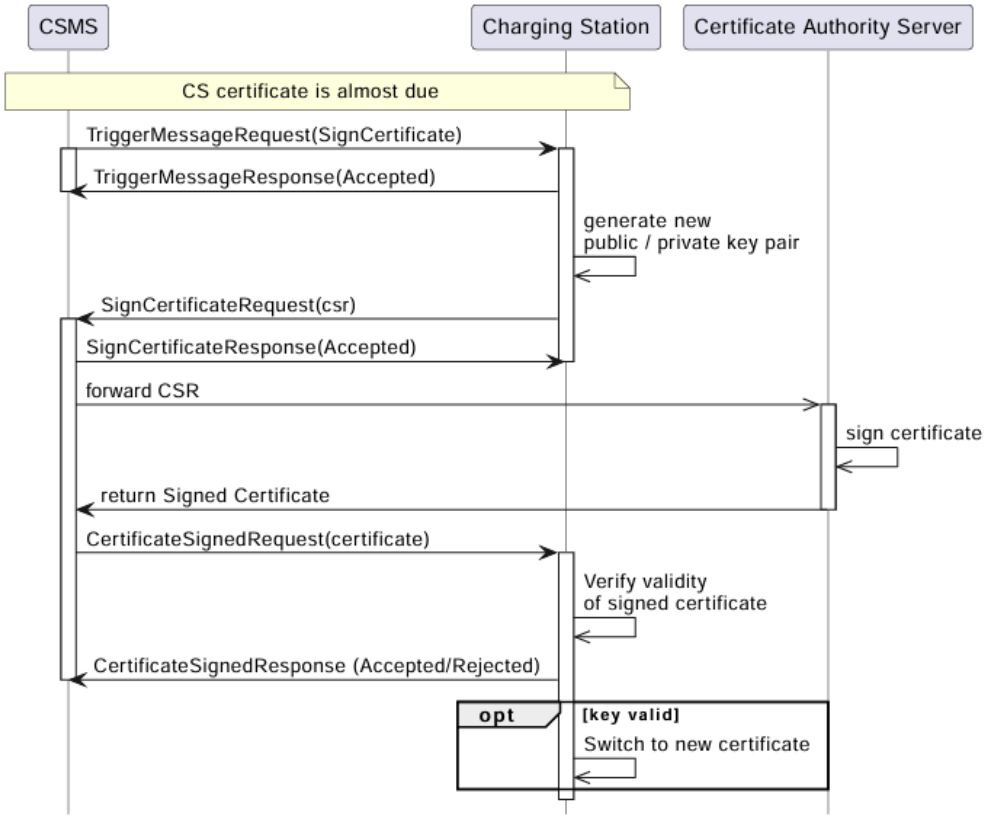



Figure 5.1: *Sequence diagram for use case A02.*

- **Implemented (functional spine):** CSR generation at the Charging Station, transmission of `SignCertificateRequest`, handling of the corresponding `CertificateSignedRequest` from the CSMS, basic validation of the returned certificate (including time validity), persistence of the accepted certificate to the emulator's local identity store at `charging/installedCertificates/<SERIAL>/certificate_<SERIAL>.pem`, and rebinding of the in-memory `CERTIFICATE_PATH`/`CERTIFICATE_KEY_PATH`. The currently established TLS session is not renegotiated mid-connection; the new identity is picked up on the next reconnect. On validation failure the client returns `Rejected` and the candidate certificate is discarded without being written to disk.

- **Configured but not strictly enforced:** The variable `OrganizationName` is present in `charging/client_config.yaml` (`security.OrganizationName: EmuOCPP`), but the CSR builder in `charging/client.py` currently hard-codes the subject organisation rather than reading it from `SECURITY_CTRL`. The procedure therefore does not abort when the variable is absent. This was considered acceptable for a controlled lab environment in which configuration is fixed, and does not affect the security properties under test.

- **Deliberately out of scope:** The `SignV2GCertificate` path and its prerequisite

`ISO15118Ctrlr.SeccId` are not implemented. EmuOCPP does not model the full ISO 15118 controller component (`ISO15118Ctrlr`), the EVCC/SECC session state, or Plug & Charge configuration semantics; it only emulates the OCPP-side certificate plumbing around enrollment. V2G-specific prerequisites are therefore omitted by design rather than by oversight. The operator-intervention state machine described in the specification's error-handling clause is similarly not modelled: a rejected certificate causes the emulator to log and continue rather than entering a locked state awaiting CSO action.

This coverage is sufficient to exercise the SP2/SP3 certificate lifecycle and the man-in-the-middle, impersonation, and downgrade scenarios analysed later in this chapter.

#### 5.1.1.2 A03 – Update Charging Station Certificate initiated by the Charging Station

**Objective.** A03 covers the same client-side certificate update procedure as A02, but with the renewal initiated by the Charging Station itself rather than by the CSMS. The station detects that one of its currently installed certificates (`ChargingStationCertificate`, `V2GCertificate` , or `V2G20Certificate`) is approaching expiry the specification gives a one-month threshold and proactively starts the signing flow so that a renewed certificate is in place before the existing one becomes invalid.

**Scenario.** The flow is identical in shape to A02 from step 3 onwards, but omits the `TriggerMessageRequest` hand-off because the trigger is internal to the Charging Station:

1. The Charging Station detects locally that its certificate is due to expire.
2. The Charging Station generates a new public/private key pair.
3. The Charging Station sends a `SignCertificateRequest` to the CSMS, populating the `certificateType` field with the applicable `CertificateSigningUse`; the implemented path in EmuOCPP sets this to `ChargingStationCertificate`.
4. The CSMS responds with `SignCertificateResponse` (status `Accepted`).
5. The CSMS forwards the CSR to the Certificate Authority Server.
6. The CA signs the certificate and returns it to the CSMS.
7. The CSMS delivers the signed certificate via `CertificateSignedRequest`.
8. The Charging Station verifies the certificate and replies with `CertificateSigned Response` (`Accepted` or `Rejected`).

The specification's error-handling clause notes that, on failure of the station-initiated renewal, the operator at the CSO is expected to investigate the issue and may subsequently re-run A02 i.e., recovery falls back to the CSMS-initiated procedure already documented in Section 5.1.1.1.

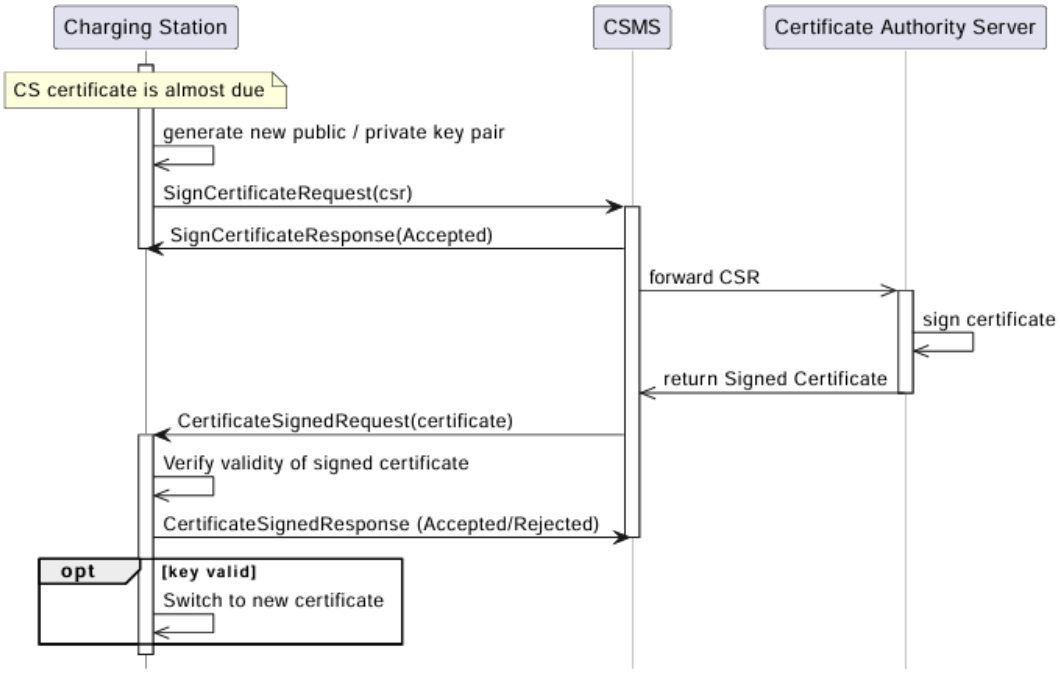



Figure 5.2: *Sequence diagram for use case A03.*

**Relevance to EmuOCPP.** A03 complements A02 in the SP2/SP3 certificate-lifecycle picture: A02 covers operator-driven rotation, while A03 covers the autonomous, expiry-driven case that a real fleet would rely on between manual interventions. The upgrade therefore implements the trigger logic for A03 and reuses the A02 installation path for everything that follows, so the two use cases differ only in how the `SignCertificateRequest` comes about. The breakdown is as follows:

- **Implemented (trigger logic):** A background coroutine in `ChargePointClientBase.check_reboot()` (`charging/client.py`) polls the certificate file currently referenced by `CERTIFICATE_PATH` once per second, gated on `VERSION` being one of `OCPP20`, `OCPP201`, or `OCPP21` and on `CERTIFICATE_PATH` being set. The certificate is parsed with `x509.load_pem_x509_certificate` and `not_valid_after_utc` is compared against the current UTC time; the trigger therefore acts on whichever station certificate is currently configured rather than discriminating between `ChargingStationCertificate`, `V2GCertificate`, and `V2G20Certificate` as the specification does. If the remaining validity is at most thirty days the implementation's interpretation of the specification's "one month" threshold the client invokes `send_sign_certificate()`, which generates a fresh key pair, builds the CSR,

and emits the SignCertificateRequest. A flag (a03Triggered) prevents the same near-expiry condition from firing the flow repeatedly within a session.

- **Reused from A02:** Once the SignCertificateRequest is in flight, A03 follows the same path as A02: the same send_sign_certificate() routine constructs the CSR, the same CertificateSignedRequest handler validates the returned certificate (including time validity), and the same persistence logic writes accepted certificates to charging/installedCertificates/ <SERIAL>/ and rebinds CERTIFICATE_PATH/CERTIFICATE_KEY_PATH. As with A02, the new identity is picked up on the next reconnect rather than mid-session, and a rejected certificate is discarded without being written to disk. For OCPP 2.1 specifically, the SignCertificatePayload additionally carries a requestId so that the paired C ertificateSignedRequest can be correlated to the originating A02 or A03 request.

- **Configured but not strictly enforced:** OrganizationName is inherited from the A02 path unchanged the CSR builder hard-codes the subject organisation rather than reading it from SECURITY_CTRL, so the prerequisite is configured in chargin g/client_config.yaml but not enforced at runtime.

- **Deliberately out of scope:** The V2G variants (V2GCertificate, V2G20Certifi cate) of A03 are not implemented for the same reason as in A02: EmuOCPP does not model the ISO 15118 controller, EVCC/SECC session state, or Plug & Charge configuration semantics. The thirty-day expiry threshold is hard-coded rather than exposed as a configuration variable, since varying the threshold is not relevant to the security properties evaluated in this thesis. The operator-intervention recovery flow described in the specification's error-handling clause (re-running A02 after CSO investigation) is likewise not modelled as a state machine; on a rejected certificate the emulator logs and continues.

Together with A02, this gives the emulator both reactive (CSMS-driven) and proactive (station-driven) certificate rotation, which is the minimum needed to exercise the SP2/SP3 lifecycle realistically rather than as a static-certificate demonstration.

#### 5.1.1.3 A05 – Upgrade Charging Station Security Profile

**Objective.** The CSO changes the security of the OCPP connection between the CSMS and a Charging Station, typically migrating an installation from a lower security profile to a higher one once the prerequisites for the new profile installed certificates or shared credentials have been provisioned.

**Scenario.**

1. The CSMS sends a `SetVariablesRequest` that updates `NetworkConfigurationPriority` so that the `NetworkConnectionProfile` associated with the new security profile becomes first in the list, and the previously active profile is demoted to the second position.
2. The Charging Station responds with a `SetVariablesResponse` of status `Accepted`.
3. The CSMS sends a `ResetRequest` of type `OnIdle`.
4. The Charging Station reboots and re-establishes the OCPP connection through the new primary `NetworkConnectionProfile`.

The specification defines no dedicated error-handling state for this use case; recovery from a failed upgrade is left to operator policy. It does, however, prohibit reverting to a lower security profile over OCPP, with a single exception: when `AllowSecurityProfileDowngrade` is implemented and set to `true`, a downgrade from profile 3 to profile 2 is permitted. Reverting from profile 2 or profile 3 to profile 1 over OCPP is forbidden in all circumstances.

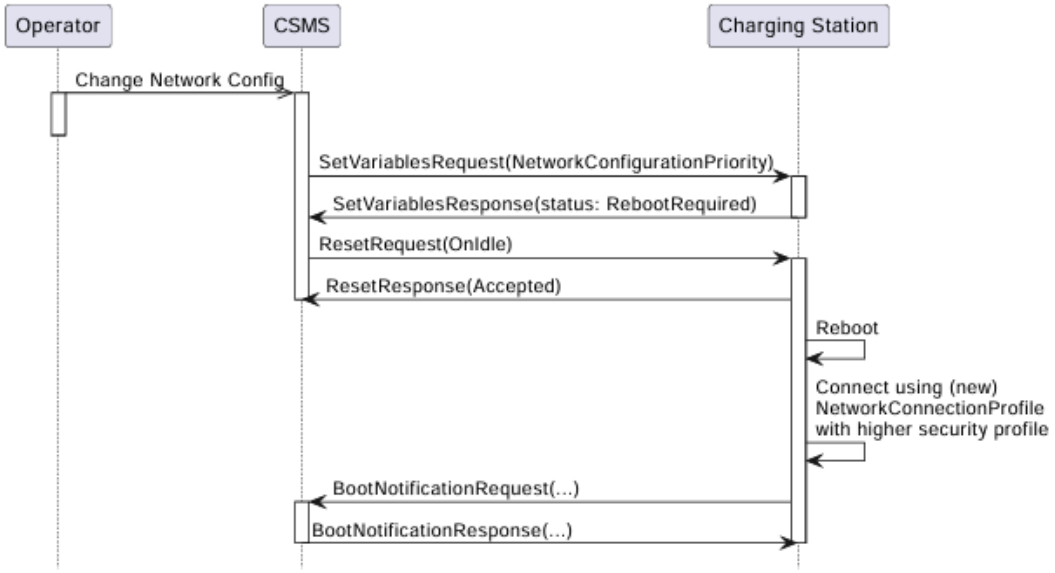



Figure 5.3: *Sequence diagram for use case A05 .*

**Relevance to EmuOCPP.** A05 is structurally important to the emulator's threat model because the active security profile determines which cryptographic guarantees apply to the OCPP channel; modelling the upgrade path therefore exposes the full chain from configuration update, through reboot, to re-handshake under a new identity, which is precisely

the surface on which the downgrade and trust-persistence experiments later in this chapter operate.

- **Implemented.** The `SetVariablesRequest` handler in `charging/client.py` (`ChargePointClientBase.on_set_variables`) recognises updates to `NetworkConfigurationPriority` and returns an attribute status of `RebootRequired`, consistent with the specification's expectation that the change cannot take effect within the current session. `ResetRequest` is accepted by `ChargePointClientBase.on_reset` with status `Accepted`, and the subsequent `check_reboot` routine performs a genuine process replacement via `os.execv`, forcing a full WebSocket teardown and a fresh boot sequence rather than a simulated restart. On reconnect, the main connection loop iterates `NetworkConfigurationPriority` in order and attempts each slot in `CONNECTION_PROFILES` (loaded from `charging/client_config.yaml`), so the handshake following the reset genuinely uses the newly-promoted profile. The downgrade restriction is enforced in real terms: `AllowSecurityProfileDowngrade` is loaded into `SECURITY_CTRL` and consulted both in `on_set_profile` (for `SetNetworkProfile`) and in the `NetworkConfigurationPriority` branch of `on_set_variables`; when false, any downgrade is rejected, and when true only the 3-to-2 transition is admitted, with any attempt to revert toward profile 1 refused outright.

- **Reused from A02 and A03.** The TLS context construction and the resolution of `CERTIFICATE_PATH` and `CERTIFICATE_KEY_PATH` are reused unchanged on the post-reset connect, and the on-disk identity material under `charging/installedCertificates/ <SERIAL>/` is what allows the handshake at the upgraded profile to succeed. As established in A02 and A03, mutating these paths does not renegotiate the active TLS session; the new identity is bound only on the next reconnect, which in A05 is precisely the reconnect triggered by the `ResetRequest`.

- **Configured but not strictly enforced.** The handler treats the new value of `NetworkConfigurationPriority` as a list but consults only its first element to determine the next preferred slot, rather than performing a full semantic validation of the reordered priority list against the present set of `NetworkConnectionProfile` entries. Similarly, the `type` field of `ResetRequest` is parsed but the handler does not branch on `OnIdle` versus `Immediate`: the emulator restarts unconditionally and does not model the obligation to wait for transactional idleness before rebooting.

- **Deliberately out of scope.** An atomic rollback state machine for failed upgrades is not modelled: if the post-reboot handshake at the new profile fails, the connection loop simply continues retrying through the priority list rather than performing a transactional revert under formal control. Persistence of in-flight transactions across the restart is likewise not modelled, since reboot is realised as a pro-

cess replacement and the emulator maintains no durable transaction journal. Full OCPP 2.1 `ResetResponse` payload variants are also not emitted by the custom `@on('Reset')` handler, whose coverage is targeted at the 2.0 and 2.0.1 response shapes; A05's behavioural envelope is exercised through the reboot mechanics rather than through 2.1-specific response fields.

The combination of reboot-driven re-handshake, partially-validated priority reordering, and a real but narrowly-scoped downgrade check makes A05 the structural foundation for the security-profile downgrade and trust-anchor persistence scenarios analysed later in this chapter.

#### 5.1.1.4 B09 – Setting a new NetworkConnectionProfile

**Objective.** The CSMS updates the connection details that the Charging Station will use to reach a CSMS, typically in preparation for a migration to a new CSMS or for the introduction of an additional connectivity slot, with the change persisted on the station so that it becomes available for selection at a subsequent reconnect.

**Updates in OCPP 2.1.** The specification deprecates `SetNetworkProfileRequest` as the canonical path for mutating a slot's connection data and introduces an alternative scenario that operates entirely through the device model: the variables of a `NetworkConnectionProfile` are exposed on a `NetworkConfiguration` component whose `instance` attribute carries the `configurationSlot`, and are written individually with `SetVariablesRequest`. The 2.1 alternative further imposes a precondition absent from the legacy path: the slot under modification must not currently appear in `NetworkConfigurationPriority`, since allowing live edits to a prioritised slot would risk the Charging Station selecting an incompletely configured profile at the next connect attempt. The legacy `SetNetworkProfileRequest` is retained for backward compatibility but is flagged as a candidate for removal in a future OCPP release.

**Scenario.**

1. The CSMS sends a `SetNetworkProfileRequest` containing an updated `NetworkConnectionProfile` together with a `configurationSlot` drawn from the `valuesList` of `NetworkConfigurationPriority` reported by the Charging Station.

2. The Charging Station validates the supplied data against its capabilities and stores the new profile against the indicated slot.

3. The Charging Station replies with a `SetNetworkProfileResponse` of status `Accepted`.

The 2.1 alternative scenario reaches the same end state through the device model. Where the slot to be updated is presently part of `NetworkConfigurationPriority`, the CSMS first removes it with a `SetVariablesRequest` so that the slot may be edited safely, then issues one or more `SetVariablesRequest` messages targeting the variables of the `NetworkConfiguration` component at `instance` equal to `configurationSlot`, each of which the Charging Station acknowledges with `Accepted`. Errors during this use case are not destructive to the existing data connection; activation of a newly written profile is the concern of B10, and is reached only when a reconnect occurs, whether through the natural retry cycle or through an operator-issued reset.

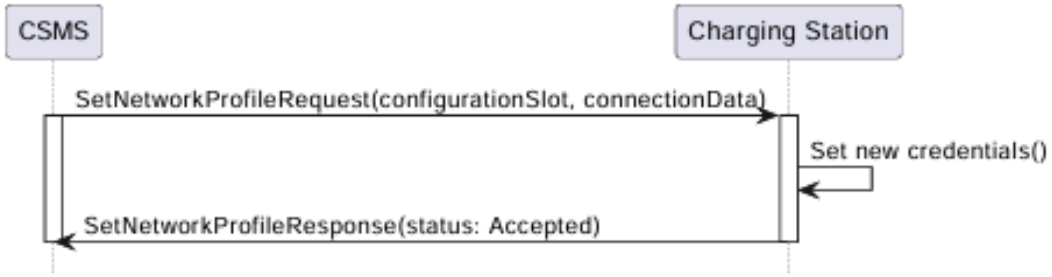



Figure 5.4: *Sequence diagram for use case B09.*

**Relevance to EmuOCPP.** B09 is the provisioning counterpart to A05's reordering: where A05 selects which slot the station will prefer at the next reconnect, B09 governs the contents of those slots, and the security-profile field carried inside each slot is the parameter that fixes the cryptographic regime under which the subsequent handshake will operate. Faithful coverage of B09 is therefore a precondition for any later experiment that turns on a station being induced to connect under attacker-influenced profile data.

- **Implemented.** The legacy `SetNetworkProfileRequest` path is the wired-in route in the emulator and is serviced by `ChargePointClientBase.on_set_profile` in `charging/client.py`. The handler enforces the spec's slot-membership constraint strictly: the supplied `configurationSlot` is parsed as an integer and must currently appear in `COMM_CTRL['NetworkConfigurationPriority']`, with any other value rejected. Where the request would replace the slot's profile with one of a lower security profile, `AllowSecurityProfileDowngrade` is consulted: an outright lower-SP write is rejected when downgrades are disallowed, and even when permitted only the 3-to-2 transition is admitted, in agreement with the rule established under A05. Successful writes update `CONNECTION_PROFILES[slot]` in memory so that the next iteration of the connection loop sees the new data, and

exceptions raised during application are reported as `Failed`. The reporting variable `OCPPCommCtrlr.ActiveNetworkProfile` is exposed through `GetVariablesRequest` and is assigned during boot/connect setup to the slot index actually used to open the present session, so it tracks live state rather than acting as a static configuration value. The 2.1 device-model alternative is partially modelled: `ChargePointClientBase.on_set_variables` contains a branch for `component.name == 'NetworkConfiguration'` that interprets the component `instance` as a slot number and stores the written variable under `NETWORK_CONFIGURATION[slot][var]`, and it enforces the 2.1 precondition by returning `Rejected` when the addressed slot is presently a member of `NetworkConfigurationPriority`, mirroring the specification's prohibition on live edits to a prioritised slot.

- **Reused from A05.** The downgrade enforcement against `AllowSecurityProfileDowngrade`, the role of `COMM_CTRL` as the seat of both `NetworkConfigurationPriority` and `ActiveNetworkProfile`, and the relationship between `CONNECTION_PROFILES` and the connection loop are inherited unchanged from A05; B09 mutates the same data structures that A05 reorders and reads, with no intermediate abstraction layer between them.

- **Configured but not strictly enforced.** On-disk persistence of slot writes is partial: `on_set_profile` calls `modify_config` with `variable='profiles'` and writes only the slot's `SP` field back into `charging/client_config.yaml`, while the remaining contents of the `NetworkConnectionProfile` – the connection URL, authentication parameters, any APN or VPN data – live only in the in-memory `CONNECTION_PROFILES` entry and are lost on a cold restart of the emulator. Validation of the profile contents themselves is similarly shallow: beyond the slot-membership and downgrade checks, the handler does not verify URL well-formedness, the consistency between the declared security profile and the credential or certificate material actually available on disk, or any deeper capability constraint. `GetVariablesResponse` returns the present value of `NetworkConfigurationPriority` as a string but does not surface a separate `valuesList` metadata field, so a CSMS cannot discover the universe of available slots through that response alone.

- **Deliberately out of scope.** The 2.1 device-model alternative is exercised at the protocol surface but is not wired through to the behaviour of the connection loop: writes accepted into `NETWORK_CONFIGURATION` are not propagated back into `CONNECTION_PROFILES`, so a slot mutated exclusively through the `NetworkConfiguration` component will not influence which endpoint is contacted at the next connect. The legacy `SetNetworkProfileRequest` therefore remains the only path through which a slot's connection data reaches the loop in practice, which preserves end-to-end coverage of B09's effect on subsequent handshakes whilst leaving the fully migrated 2.1 path as a stub of the device-model surface rather than a functional

substitute for it. The alternative scenario's two-step orchestration – removal from priority followed by per-variable writes – is consequently not modelled as a coupled transaction either, since its effect terminates within the device model.

The combination of strict slot-membership validation, real downgrade enforcement along the legacy path, and a half-wired device-model alternative makes B09 the natural staging ground for the profile-injection and migration-redirection scenarios analysed later in this chapter, in which the security-profile field of a written slot is the lever placed under attacker control.

#### 5.1.1.5 F06 – Trigger Message

**Objective.** The CSMS requests that the Charging Station spontaneously emit a Charging Station-initiated message of a specified type, allowing the CSMS to elicit information that the station would otherwise send only on its own schedule.

**Scenario.**

1. The CSMS sends a `TriggerMessageRequest` to the Charging Station, naming the message type it wishes to receive.
2. The Charging Station replies with a `TriggerMessageResponse` indicating `Accepted`, `Rejected`, or `NotImplemented`.
3. Where the response is `Accepted`, the Charging Station emits the requested message.

The specification defines no dedicated error-handling state for this use case but identifies two distinct failure postconditions: the CSMS may fail to receive the response at all, or it may receive an `Accepted` response and never observe the requested message thereafter. The mechanism is explicitly not intended for retrieval of historic data.

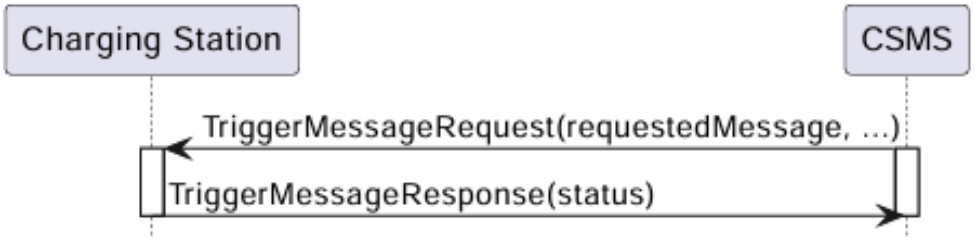



Figure 5.5: *Sequence diagram for use case F06.*

**Relevance to EmuOCPP.** F06 is structurally significant for the threat model because TriggerMessage is one of the few mechanisms by which the CSMS can compel the Charging Station to emit messages outside its own internal schedule, and it is the entry point through which a CSMS – including an impersonated or compromised one – may elicit a certificate signing request from the station and thereby attempt to graft itself onto the CSR and CertificateSigned chain modelled under A02 and A03.

- **Implemented.** The TriggerMessage handler in charging/client.py (ChargePointClientBase.on_trigger_message) dispatches on the requestedMessage field through an explicit if/elif chain and is gated by OCPP version: SignChargingStationCertificate is honoured under OCPP 2.0, OCPP 2.0.1, and OCPP 2.1, whereas every other trigger type is serviced only under OCPP 2.1 and returns NotImplemented for the earlier versions still supported by the emulator. Within the OCPP 2.1 path, four trigger types reach genuine emission. StatusNotification requires both evse and connectorId to be supplied – their absence yielding the explicit Rejected response permitted by the specification – and on Accepted emits a real StatusNotificationRequest via _send_triggered_status_notification, honouring the supplied targeting. MeterValues is always Accepted and emits a MeterValuesRequest via _send_triggered_meter_values, using the supplied evse identifier where present and defaulting to slot 1 otherwise. TransactionEvent is always Accepted and emits a TransactionEventRequest via _send_triggered_transaction_event, with the supplied evse carried into the payload. For these three operational triggers the emission occurs synchronously within the same handler frame, so the specification's failure postcondition of an Accepted response without a subsequent message arises only as a consequence of network-level failure rather than as a structural property of the emulator. SignChargingStationCertificate is likewise Accepted but proceeds along a deferred path: the handler sets the csCert flag and the actual SignCertificateRequest is dispatched on the next tick of the background check_reboot loop, which means the failure window between Accepted and emission is genuinely open here in a way it is not for the synchronous triggers. The handler also implements the second explicit Rejected rule the specification permits: a trigger of BootNotification is rejected when last_boot_accepted is already true, on the grounds that the boot has already succeeded.
- **Reused from A02 and A03.** The SignChargingStationCertificate trigger reuses the CSR construction and submission path established for A02 and A03: setting csCert causes check_reboot to invoke send_sign_certificate on its next iteration, which in turn drives the same SignCertificateRequest and downstream CertificateSigned installation handling already exercised under those use cases, including the standing convention that the new identity material is bound

only on the next reconnect.

- **Configured but not strictly enforced.** `LogStatusNotification` and `Firmware StatusNotification` are accepted at the protocol surface and return `Accepted`, but are not modelled beyond that response: no corresponding notification message is emitted, so the specification's success postcondition that the requested message actually arrives at the CSMS is not met for these two types. The payloads emitted by the synchronous triggers are themselves synthetic: the `MeterValuesRequest` carries a current-ish reading rather than a value drawn from a metering record, and `T ransactionEventRequest` fabricates a transaction identifier and a `Charging` state rather than reflecting a transaction the emulator is genuinely conducting. The specification's caveat that `TriggerMessage` is not intended for historic data retrieval is therefore satisfied trivially in the emulator there is no historic store from which such data could be drawn rather than through any explicit guard in the handler.

- **Deliberately out of scope.** `SignV2GCertificate` returns `NotImplemented`, in keeping with EmuOCPP's standing decision not to model V2G or ISO 15118 controller behaviour. `CustomTrigger` likewise returns `NotImplemented` as a matter of policy, since vendor-specific extensions fall outside the conformance envelope the emulator targets. The wider 2.1 `requestedMessage` enumeration including `Heartbeat`, `BootNotification` in the non-rejection case, and other types added in the 2.1 expansion is uniformly answered with `NotImplemented` rather than partially modelled, so the emulator's coverage of F06 is best characterised as selective rather than exhaustive. Per-trigger rate limiting or debouncing of repeated `TriggerMessage` requests is not modelled, and contextual sanity checks such as refusing a `MeterValues` trigger when no transaction is active are not enforced; the trigger fires regardless, and the resulting payload is synthetic in the sense described above.

The combination of synchronous emission for the three operational triggers, the deferred path for certificate signing that holds open the specification's `Accepted`-without-emission window, and the asymmetric treatment of the wider request enumeration positions F06 as the protocol surface against which the trigger-driven impersonation and certificate-signing scenarios analysed later in this chapter operate.

#### 5.1.1.6 M01 – Certificate Installation EV

**Objective.** An EV initiates installation of a new contract certificate, and the Charging Station forwards the request to the CSMS, which obtains the signed material from a third party such as a contract certificate pool and returns it to the station for delivery back to the EV.

**Updates in OCPP 2.1.** The specification flags M01 as updated in OCPP 2.1, with the principal revisions falling on the ISO 15118 surface that the OCPP layer transports rather than on OCPP itself; EmuOCPP's coverage remains a minimal pass-through at the OCPP layer and does not depend on the revised ISO 15118 surface.

**Scenario.**

1. Communication between the EV and the EVSE is established, and the EV signals its intent to install a contract certificate, in accordance with ISO 15118-1 use case C2.

2. The Charging Station sends a `Get15118EVCertificateRequest` to the CSMS with `action` set to `Install`.

3. The CSMS, in coordination with a contract certificate pool or equivalent third party, returns a `Get15118EVCertificateResponse` carrying the signed material to the Charging Station.

4. The Charging Station relays the certificate material to the EV through the ISO 15118 channel.

The error-handling stipulation is timing-driven: where the CSMS is unable to respond within the specified bound – ISO 15118-2 fixes a five-second message timeout for `CertificateInstallationReq` – the Charging Station is required to indicate failure to the EV. The specification also notes that alternative communication paths for contract certificate installation exist outside its scope.

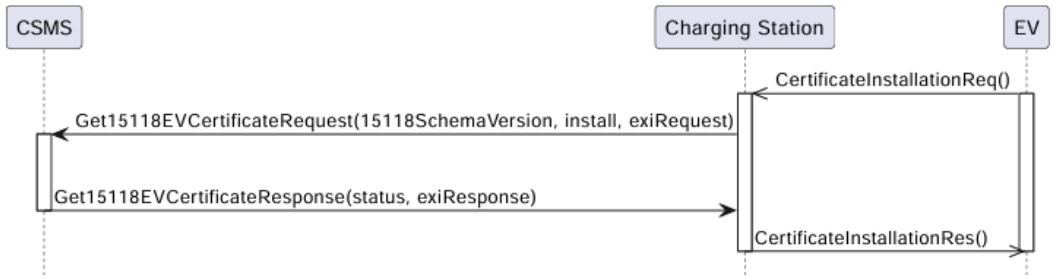



Figure 5.6: *Sequence diagram for use case M01 – contract certificate installation initiated by the EV, forwarded by the Charging Station to the CSMS as a `Get15118EVCertificate` exchange, with the signed material relayed back to the EV through the ISO 15118 channel.*

**Relevance to EmuOCPP.** M01 occupies a different position in the emulator's coverage from the use cases discussed earlier in this section. Its protocol weight at the OCPP layer is small – one request and one response, both effectively opaque envelopes carrying ISO 15118 EXI material – whilst the cryptographic substance of the use case lives on the EV side, behind the EVCC/SECC boundary that EmuOCPP does not represent. Modelling M01 at the OCPP layer is therefore feasible and indeed implemented, but doing so does not in itself bring the contract certificate exchange into the emulator's scope; M01 is most accurately described as OCPP transport plumbing for an ISO 15118 artefact.

- **Implemented.** The CSMS-side handler `ChargePointServerBase.on_get_15118_ev_certificate` in `charging/server.py` is wired in full rather than stubbed: it returns `status="Accepted"` together with a placeholder `exi_response` drawn from an in-memory pool when the request carries `action="Install"`, populates a `remainingContracts` field when `maximumContractCertificateChains` is supplied (the ISO 15118-20 variant), and returns `status="Rejected"` for any value of `action` other than `Install`. The Charging Station originates the corresponding request through `ChargePointClientBase.send_get_15118_ev_certificate` in `charging/client.py`, which constructs a `Get15118EVCertificateRequest` with `action="Install"`. A dedicated scenario harness at `charging/scenarios/m01_certificate_installation_ev.py` exercises the path under both ISO 15118-2 and ISO 15118-20 framings, so the OCPP-layer round trip is observable end to end in the emulator without an external EV actor.
- **Configured but not strictly enforced.** Both the handler and the originator are gated to OCPP 2.1: the originator raises if invoked under any other version, and the handler returns `None` rather than a properly-formed `CallResult` when the active connection's version does not equal `v2.1`. The gating itself is intentional, since M01 is exercised only against the 2.1 path, but the return-`None` shape on the gated branch is not a conformant rejection: a CSMS speaking an earlier version would observe a protocol-level absence rather than an explicit failure, which is the reverse of the spec's expectation that the exchange terminate cleanly one way or the other. The `exiResponse` payload itself is a fixed placeholder rather than material produced by genuine ISO 15118 processing, so the response is well-formed at the OCPP layer but inert at the V2G layer.
- **Deliberately out of scope.** The ISO 15118 surface that this use case ultimately concerns is not modelled. The emulator does not represent an `ISO15118Ctrlr` component or any related device-model variables, exposes no V2G keys through `client_config.yaml`, and maintains no EVCC/SECC session state; the EV-side preamble described in ISO 15118-1 use case C2 has no counterpart in EmuOCPP. Real EXI generation, parsing, and validation are similarly absent, as is integration with an external contract certificate pool, since the handler synthesises its

responses from a static internal source. Consistent with the standing convention adopted across this thesis, V2G-layer behaviour is treated as outside EmuOCPP's threat-model purpose rather than as a gap in OCPP coverage.

The OCPP-layer pass-through in M01 is sufficient to surface message-routing and forwarding behaviour at the CSMS/Charging Station boundary in the experiments later in this chapter, but the cryptographic substance of contract certificate installation lies in the ISO 15118 layer that EmuOCPP does not model and remains outside the scope of the analyses that follow.

#### 5.1.1.7 CALLRESULTERROR Asynchronous Operation Failure Reporting

**Objective.** The `CALLRESULTERROR` message type allows the receiver of an asynchronous operation to report a failure that arises after a `CALLRESULT` has already been transmitted to acknowledge initial acceptance. Where `CALLERROR` signals that a `CALL` could not be processed at all, `CALLRESULTERROR` addresses the case in which the operation was accepted, deferred for later completion, and subsequently failed.

**Updates in OCPP 2.1.** `CALLRESULTERROR` is introduced as a new message type with `MessageTypeId` 5; it does not exist in OCPP 1.6 or 2.0.1, both of which define only types 2 (`CALL`), 3 (`CALLRESULT`), and 4 (`CALLERROR`). The wire format mirrors that of `CALLERROR [5, uniqueId, errorCode, errorDescription, errorDetails]` but is semantically distinct in that it follows, rather than replaces, a successful `CALLRESULT` bearing the same `uniqueId`.

**Scenario.**

1. The originator transmits a `CALL` carrying a unique message identifier `uniqueId`.
2. The receiver acknowledges initial acceptance by transmitting a `CALLRESULT` on the same `uniqueId`, indicating that the operation has been accepted but will complete asynchronously.
3. The receiver attempts to complete the deferred operation.
4. Upon detecting a failure subsequent to acceptance, the receiver constructs a `CALLRESULTERROR` frame referencing the original `uniqueId`, with an `errorCode`, a human-readable `errorDescription`, and an optional `errorDetails` object.
5. The originator correlates the incoming `CALLRESULTERROR` against its outstanding asynchronous operations by `uniqueId` and reconciles its local state accordingly.

The correlation between the originating CALL and the eventual CALLRESULTERROR is therefore carried entirely by the uniqueId, and the originator is expected to retain that identifier across the CALLRESULT acknowledgement until the asynchronous outcome is delivered. Schema validation of the error payload follows the same conventions as for CALLERROR.

**Relevance to EmuOCPP.** Within the emulator, CALLRESULTERROR is treated primarily as an offensive primitive rather than as a faithful asynchronous-operation accounting mechanism: it provides an injection surface for fabricated post-acceptance failures whose plausibility depends on correct framing rather than on a complete underlying state machine. The implementation is consequently narrow and deliberately one-sided.

- **Implemented.** CSMS-side emission of CALLRESULTERROR via send_call_result_error() on ChargePointServerBase and the operator console command demoCallResultError, accepting an optional uniqueId, an errorCode (defaulting to InternalError), and an errorDescription (defaulting to a fixed demonstration string). The emitted frame conforms to the [5, uniqueId, errorCode, errorDescription, errorDetails] structure mandated by the 2.1 messaging chapter, and emission is version-gated: the operator command is rejected for clients whose negotiated subprotocol is below OCPP 2.1.
- **Reused from A02 and A03.** The websocket transport, JSON serialisation path, and dispatch infrastructure inherited from the upstream ocpp library's ChargePoint class already exercised in A02 and A03 — form the substrate on which CALLRESULTERROR is sent. The route_message() override on ChargePointServerBase, which records CALL receive times, is unchanged. The CallResultError class itself is supplied as a local extension to ocpp.messages, since the upstream library predates the 2.1 message type.
- **Configured but not strictly enforced.** The errorDetails payload is forwarded verbatim from the operator command and is not validated against a schema. The uniqueId is likewise not checked against the set of CALL identifiers previously acknowledged with a CALLRESULT; the internal field _last_call_result_id is tracked but not consulted during construction of the error frame.
- **Deliberately out of scope.** EmuOCPP does not model the asynchronous operation state machines that would, in a conformant implementation, automatically emit CALLRESULTERROR on real post-acceptance failures of operations such as RequestStartTransactionRequest or RequestStopTransactionRequest; the message is produced only under operator control. Reception and dispatch of incoming CALLRESULTERROR frames are not implemented, nor is charging-station-side emission. Retry, back-off, and propagation of asynchronous outcomes to a downstream

operator interface are likewise omitted, consistent with the emulator's policy of logging and continuing rather than entering a recovery state.

The emission-only, operator-driven posture of this implementation is what makes `CALLRESULTERROR` a useful primitive in later chapters: it permits the construction of forged post-acceptance failure scenarios, including unsolicited error frames against arbitrary `uniqueId` values, without the emulator's own state machine constraining the attack surface.

#### 5.1.1.8 SEND message type

**Objective.** Provide an unconfirmed, asynchronous message type for transmissions in which no application layer reply is expected, supporting telemetry and notification streams that would otherwise block the sender on a CALL/CALLRESULT round trip.

**Updates in OCPP 2.1.** OCPP 2.1 extends the JSON-RPC envelope by introducing `MessageTypeId` 6 (SEND) alongside the existing CALL (2), CALLRESULT (3), and CALLERROR (4) frames. A SEND frame retains the four-element wire structure `[6, MessageId, Action, Payload]` but, by definition, elicits no correlated response from the recipient: no CALLRESULT is returned, and no CALLERROR is generated for application-level faults, with transport-layer faults remaining governed by the underlying WebSocket connection. The receiver is expected to process the payload on a best-effort basis and resume awaiting the next frame. The same revision also introduces `MessageTypeId` 5 (`CallResultError`); only SEND is treated in this subsubsection.

**Scenario.**

1. The sender constructs a SEND frame as the JSON array `[6, *MessageId*, *Action*, *Payload*]`, where *MessageId* is a fresh identifier and *Action* names the OCPP action whose payload schema is being transmitted.
2. The frame is serialised and dispatched over the existing WebSocket connection without registering a pending-response timer or correlation entry on the sender side.
3. The receiver decodes the frame, identifies `MessageTypeId` 6, and routes the payload to the handler registered for *Action*.
4. The handler executes any side effects associated with the notification and returns; no envelope is returned to the sender.

Because no CALLRESULT or CALLERROR is associated with a SEND frame, error recovery is asymmetric: faults observed by the receiver cannot be signalled back to the originator at the protocol layer, and any remediation must be inferred from collateral

channels such as subsequent CALL traffic, a server initiated `TriggerMessageRequest`, or transport disconnection. The cost of a malformed or unsolicited SEND therefore falls entirely on the receiver.

**Relevance to EmuOCPP.** The SEND message type is material to the emulator's threat model precisely because it removes the implicit backpressure that CALL/CALLRESULT correlation would otherwise impose on a sender. Modelling SEND end-to-end on the transmit path enables the construction of asymmetric flooding scenarios in which a malicious client emits high-rate notification traffic at line rate while the CSMS must still parse and route every payload; modelling rate-limiting on the receive path, in turn, allows the emulator to exercise candidate mitigations under controlled conditions.

- **Implemented:** Outbound construction and dispatch of SEND frames on both the client and CSMS sides, via `ChargePointServerBase.send_unconfirmed` in `charging/server.py` and the corresponding `send_unconfirmed` method in charging/client.py. Both build a `Send` envelope, serialise it through `to_json`, and transmit it over the active WebSocket. Receive-side rate limiting on the CSMS is governed by the `_SecurityConfig` parameters `send_max_per_second` and `send_dos_cooldown_seconds`, which lock out a charging station that exceeds the configured threshold.

- **Reused from the existing CALL path:** JSON serialisation through `json.dumps` with the shared `_DecimalEncoder`, the WebSocket transmit primitive, and the standard transmit-side logging line, so that SEND traffic appears in the same audit trail as CALL traffic.

- **Configured but not strictly enforced:** The set of actions admissible as SEND is not policed at the sender. The operator-facing command `demoSend <CP_ID> <Action> <Payload>` accepts an arbitrary action string, and `validate_payload` is gated on `Call` and `CallResult` envelopes only, so SEND payloads bypass payload-level schema checks. This permits deliberately non-compliant emissions, for example transmitting `BootNotification` as a SEND frame, which is useful for adversarial test cases but is not a conformance claim.

- **Deliberately out of scope:** Inbound SEND handling is not wired into the dispatcher. `messages.unpack` iterates over `[Call, CallResult, CallError]` and rejects `MessageTypeId` 6 with a `PropertyConstraintViolationError`; SEND is therefore transmit-only at the application layer, consistent with the emulator's role of emitting attacker-controlled SEND streams and characterising defender behaviour rather than acting as a fully conformant SEND endpoint. Application-layer delivery acknowledgement, in-order reassembly across SEND frames, and the OCPP 2.1 `CallResultError` envelope are likewise not modelled.

This transmit-only realisation of SEND underpins the SEND-based denial-of-service experiments, where the unconfirmed semantics of `MessageTypeId` 6 are exploited to flood the CSMS with `NotifyPeriodicEventStream` frames at a rate that no `CALL`/`CALLRESULT`-correlated channel could sustain.

#### 5.1.1.9 The OCPP 2.1 Compatibility Shim

**Motivation.** The official Mobility House `ocpp` Python package (PyPI release `2.1.0`, July 2025) ships only the `ocpp.v16`, `ocpp.v20`, and `ocpp.v201` subpackages. The `ocpp.v21` module exists in the upstream `mobilityhouse/ocpp` repository but is not bundled in the distributed wheel. EmuOCPP was originally built against `ocpp==0.26.0` (recorded in `requirements.local.txt`) and was upgraded to `ocpp==2.1.0` (`requirements.txt`) as part of the OCPP 2.1 work. To bridge the gap between the published wheel and the 2.1 surface that the emulator relies on, a single-module package, `ocpp_shim/`, is placed at the repository root and imported eagerly from both `charging/client.py` and `charging/server.py`. The import is wrapped in `try/except ImportError` so that absence of the shim does not crash process start-up; the failure surfaces later, on the first `from ocpp.v21 import ...`, which is the desired behaviour for an optional compatibility layer.

**Design.** The shim performs three distinct adaptations. First, it patches `ocpp.messages.get_validator` so that the string `"2.1"` is accepted as a valid `ocpp_version`; the stock implementation only recognises `"1.6"`, `"2.0"`, and `"2.0.1"`. The replacement, `get_validator_v21`, loads JSON Schema documents from `<installed-ocpp>/v21/schemas/` and caches them in the same `_validators` dictionary used by the original. Second, it adds `Payload`-suffixed aliases to every class in `ocpp.v21.call` and `ocpp.v21.call_result`: the upstream 2.1 source uses bare class names (for example `BootNotification`), whereas the emulator was written against the older convention (`BootNotificationPayload`). The aliases are added unconditionally, so emulator code referring to `call21.XPayload` resolves correctly regardless of which naming the underlying module exposes. Third, if `ocpp.v21` cannot be imported at all, the shim synthesises a substitute module from `ocpp.v201`, extends `CertificateSigned` with the optional `request_id` field added in 2.1, and registers the result in `sys.modules` under the names `ocpp.v21` and `ocpp.v21.call`.

**Activation.** The shim self-detects which path to take. A guarded `import ocpp.v21` determines whether the real subpackage is present; in deployed practice it always is, because `run_local.sh` copies `ocpp_repo/ocpp/v21/` into the active virtual environment before either component is launched. The shim therefore runs in *real-v21 mode* on every documented invocation of the emulator, with the synthesised *fallback mode* reserved for environments in which the upstream tree has not been provisioned. The validator monkey-patch is installed in either mode and delegates transparently to the original `get_`

validator for non-2.1 versions, so the behaviour of the 1.6, 2.0, and 2.0.1 code paths is unchanged.

**Relevance to EmuOCPP.** The shim is invisible to the rest of the code base by design: every consumer of OCPP 2.1 message classes imports them through the canonical ocpp.v21 namespace, and version selection in charging/client.py and charging/server.py branches on the negotiated subprotocol rather than on whether the shim is present. This isolation was a deliberate scoping choice and yields the following properties:

- **Implemented.** A persistent ocpp.v21 import surface that resolves identically whether the upstream v21/ tree has been copied in or not, with Payload-suffixed aliases on every request and response class.
- **Implemented.** Transparent extension of ocpp.messages.get_validator to support ocpp_version="2.1", including a tolerant schema-lookup heuristic that strips the Payload suffix from action names when the file-name convention used by the upstream schemas does not match (ocpp_shim/__init__.py, lines 46–53).
- **Configured but not strictly enforced.** The provisioning of the upstream v21/ tree is performed by run_local.sh rather than by the shim itself; if the script is bypassed and the wheel has not been augmented, the shim silently enters fallback mode.
- **Deliberately out of scope.** The shim does not back-port any 2.1-only protocol features into the 2.0.1 namespace beyond the request_id field on CertificateSigned; version differences are handled explicitly at the call sites in charging/client.py and charging/server.py by branching on the VERSION constant. Enum semantics in fallback mode are aliased to those of 2.0.1, which is acceptable for the emulator's purposes but is not a faithful representation of any 2.1-only enumeration values. The shim also does not interpose on the CallResultError or Send message types, which are imported directly from ocpp.messages by both endpoints.

The shim is a precondition for every OCPP 2.1 experiment that follows: the attack scenarios developed in later chapters including the certificate installation, impersonation, and denial-of-service variants all import their request and response classes through ocpp.v21 and rely on the shim to reconcile the published package with the upstream protocol surface.

### 5.1.2 Evaluation Metrics

Relevant evaluation metrics should be described and justified.

The effectiveness of the EmuOCPP testbed and the OCPP 2.1 protocol revisions it exercises was evaluated using three complementary metrics to ensure a comprehensive assessment. Attack success measured the binary outcome of whether each attack was demonstrably executed end to end against the testbed, requiring not merely that the constituent messages reach their targets but that the security property contested by the attack channel confidentiality, peer authenticity, or service availability be observably violated under the conditions modelled in the emulator, with the OCPP 2.0.1 baseline serving as an implicit comparator that locates the practical efficacy of the 2.1 revisions. Attack classification distinguished between attacks that stem from a property of the protocol design itself and are therefore reachable against any conformant implementation, and those whose exploitability depends on a specific behaviour of the emulator or of comparable real-world deployments, bounding the population of installations a given attack threatens and identifying the layer at which a remediation would have to be applied.Novelty separated attacks treated as new contributions of this work from those reproduced from prior literature, so that the evaluation reads simultaneously as an independent re-validation of known weaknesses and as a presentation of new ones.

## 5.2 Results

The following table summarises the set of attacks evaluated in the EmuOCPP testbed under OCPP 2.1, indicating whether each attack is considered novel in this work, whether it targets protocol/design issues or implementation behaviour, and whether it was successfully demonstrated in our environment.

| ID | Name | New | Type | OCPP Ver. | Attack Success |
|---|---|---|---|---|---|
| D2 | SEND Flood | ✓ | Des | 2.1 | ✓ |
| D1 | CSMS (D)DoS | × | Des | 2.1 | ✓ |
| M1 | MitM SP1 | × | Des | 2.1 | × |
| M2 | MitM SP2 Upgrade | × | Des | 2.1 | × |
| M3 | MitM SP3 Upgrade | × | Des | 2.1 | × |
| M4 | MitM SP Downgrade | × | Imp | 2.1 | × |
| I1 | CS Impersonation | × | Des, Imp | 2.1 | × |
| I2 | CS Impersonation | × | Des, Imp | 2.1 | × |

Table 5.1: *Attack evaluation summary for EmuOCPP under OCPP 2.1.*

The results show that D2 (SEND Flood) and D1 (CSMS (D)DoS) were successfully reproduced in the OCPP 2.1 setup. Notably, D2 is a novel attack identified in this work. All remaining attacks listed in the table were not reproducible in OCPP 2.1, as the weak-

nesses they previously relied on were mitigated in the latest version; these attacks were, however, practically achievable in OCPP 2.0.1 and earlier versions. The **Type** column distinguishes attacks stemming primarily from protocol/design choices (Des) versus those requiring specific implementation behaviour (Imp), and **New** flags the attacks treated as novel within the scope of this work.

## 5.3 Analysis and/or Discussion

This section analyzes impersonation attacks in our emulator with emphasis on the contrast between OCPP 2.0.1 and OCPP 2.1, and on why the attacks were successful in the former but mitigated in the latter. The key finding is that mitigation in the updated environment is driven by **credential binding**: the charging station identity is bound to cryptographic client credentials (SP3 mTLS + certificate subject checks), not only to a static CP ID string.

### 5.3.1 SEND Flood DoS Attack (D2) in OCPP 2.1

#### 5.3.1.1 Vulnerability identification from the specification

We identified the SEND Flood DoS vulnerability by thoroughly analysing the OCPP 2.1 specification, focusing on transport behaviour introduced in the latest version. OCPP 2.1 adds new message types beyond the traditional request–response pattern. In particular, the **SEND** message type (MessageTypeId = 6) is defined as an *unconfirmed* message that does not expect a response, i.e., messages not ending in "Request" or "Response". SEND is intended for frequent periodic data (e.g., monitor values) and, unlike CALL/CALLRESULT, it explicitly permits transmission without waiting for any acknowledgement.

The key specification statement motivating our attack is that *"since SEND messages use the same websocket connection as CALL/CALLRESULT messages, the frequent sending of large SEND messages may cause a delay for other messages."* This creates a practical opportunity: an authenticated endpoint can send SEND messages at high rate and high volume, consuming shared connection bandwidth and server-side processing time, thereby delaying (or starving) critical control messages on the same connection.

#### 5.3.1.2 Attack intuition and threat model

The SEND mechanism changes the sender constraints compared to CALL/CALLRESULT:

- The receiver **shall not** respond to SEND with CALLRESULT or CALLERROR.

- The sender **may** transmit SEND at any time, without waiting for prior responses.
- SEND shares the same WebSocket transport as all other OCPP messages.

Therefore, the attacker does not need to exploit cryptography; instead, they exploit the *asymmetry* in protocol workload: the sender can emit many messages quickly, while the CSMS still must receive, parse, route, and (at minimum) enqueue or drop each message. This causes resource contention and increases latency for higher-priority messages.

Our threat model assumes an attacker can operate a malicious charging station (or compromise one) that is able to establish a valid OCPP 2.1 session to the CSMS. The attack is thus performed *after* normal authentication, making it relevant even for TLS-based deployments.

#### 5.3.1.3 Implementation in EmuOCPP (`send_dos.py`)

We implemented the attack as a scenario script in EmuOCPP. The script starts a single malicious charging station and, after the normal boot process, transmits a burst of SEND frames:

- **Flood size:** `N_SEND_MESSAGES = 10,000`.
- **Flood duration:** spread over `TARGET_DURATION_SECONDS = 10.0` seconds via a small sleep interval between transmissions.
- **Message format:** SEND frames follow the OCPP 2.1 syntax `[MessageTypeId, MessageId, Action, Payload]` with `MessageTypeId = 6`.
- **Action used:** `NotifyPeriodicEventStream`, chosen to mirror the specification's examples of frequent periodic data.
- **Payload structure:** a small but varying structure (timestamp, value jitter, and a sequence number) to avoid trivial deduplication and to force parsing/handling at the CSMS.

From the attackers perspective, the crucial step is calling an emulator helper that transmits an *unconfirmed* message, i.e., `send_unconfirmed(action, payload)`. Because no response is expected, the attacker can maintain continuous transmission pressure without being throttled by the request–response discipline.

#### 5.3.1.4 Expected impact and interpretation

The SEND flood is designed to introduce **delay and processing contention** on the CSMS:

- **Transport contention:** large or frequent SEND traffic occupies the same WebSocket connection used for operational control, increasing queueing delay as shown in Figure 5.7.




Figure 5.7: *The server continuously receiving and printing SEND messages*

- **Server workload:** even if SEND is ultimately dropped by policy, the CSMS must still accept and inspect frames to decide routing and enforcement.

- **Protocol-level asymmetry:** the absence of mandatory responses removes backpressure that would otherwise limit attacker throughput.

In our experiments, this behaviour constitutes a denial-of-service condition in which normal OCPP message processing becomes delayed or unreliable, especially if multiple malicious stations perform the same pattern concurrently. The server experiences extreme load as you can see in Figure 5.8.

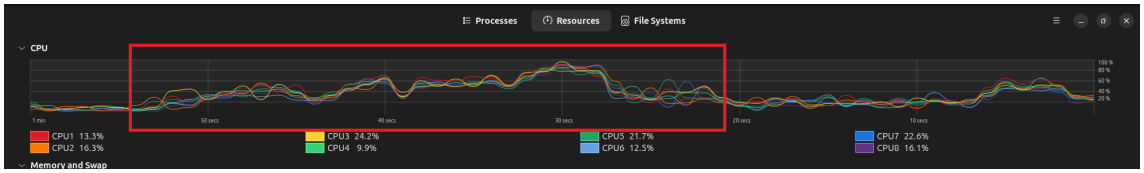



Figure 5.8: *The abnormal CPU spike observed during the attack phase*

#### 5.3.1.5 Pseudocode and explanation

Algorithm 4 presents the SEND Flood DoS procedure implemented by `send_dos.py`. The algorithm highlights the main elements: (i) connect as a legitimate OCPP 2.1 charging station, (ii) send a high volume of MessageTypeId 6 frames without waiting for responses, and (iii) maintain a target emission rate to sustain load over a chosen window.

**Algorithm 4:** SEND Flood DoS Attack (D2) using OCPP 2.1 SEND

**Input:** CSMS endpoint; authenticated CP credentials/certificates; flood size $N$; duration $T$; action name $A$ (e.g., `NotifyPeriodicEventStream`)

**Output:** Increased CSMS latency / degraded message handling due to SEND congestion

**Procedure** *SEND_Flood_DoS()*:

- Establish OCPP 2.1 WebSocket session to CSMS (normal boot/authentication)
- Compute inter-message delay $\delta \leftarrow \frac{T}{N}$
- **for** $i \leftarrow 1$ **to** $N$ **do**
  - Construct payload $P_i$ with timestamp, value jitter, and sequence number $i$
  - Send SEND frame: $[6, \text{MessageId}_i, A, P_i]$ (unconfirmed; no response expected)
  - Sleep for $\delta$ to sustain the flood over duration $T$
- **end**
- Terminate or wait (server may close connection due to overload/mitigation)

**End**

**Algorithm explanation:** The crucial property exploited is that SEND frames do not require a response and may be transmitted at any time. This removes request–response backpressure and allows the attacker to dominate the shared WebSocket connection with high-rate traffic. Because CALL/CALLRESULT messages share the same channel, their delivery and processing is delayed as SEND traffic increases, matching the performance-risk noted in the OCPP 2.1 specification.

#### 5.3.1.6 SEND DoS Mitigation (Per-CP rate limiting + alerting + containment)

To mitigate the SEND Flood DoS in EmuOCPP, we implemented a **defense-in-depth** mechanism that combines (i) **per-connection/per-CP rate limiting** at the OCPP message parsing layer, (ii) **explicit alert generation** when abuse is detected, and (iii) **containment actions** at the CSMS connection-management layer (disconnect and temporary lockout). The goal is not to "prevent SEND" (SEND is legitimate in OCPP 2.1), but to prevent a single authenticated endpoint from monopolizing the shared WebSocket channel and degrading availability of other OCPP messages.

**Mitigation design rationale.** The key issue exploited by the attacker is the absence of protocol backpressure for SEND (MessageTypeId $= 6$): because no response is expected, an attacker can transmit messages at a rate that exceeds the CSMS processing capacity.

Therefore, the CSMS must impose **local backpressure** by enforcing a maximum acceptable SEND rate and by dropping excess messages early, before costly routing or business logic.

**(1) Rate limiting at the message-handling layer.** We applied rate limiting where it is most effective: **immediately after decoding an incoming OCPP frame and identifying its MessageTypeId**. For each WebSocket session (i.e., per connected CP instance), the implementation tracks SEND volume in a sliding time window (1 second). If the number of SEND messages exceeds a configured threshold (e.g., 50/s), the CSMS:

- logs a warning such as `SEND rate exceeded ... dropping this SEND`,
- **drops** the SEND frame (no further parsing/routing),
- triggers an optional flood callback for higher-level containment.

This approach directly addresses the protocol asymmetry: the attacker can still transmit, but the CSMS does not spend unbounded work per message.

**(2) Alert generation (observable security signal).** Once flooding is detected, the CSMS generates an explicit alert message (log-level `ERROR`) indicating:

- the affected CP ID (e.g., `E2507-8420-1274`),
- the measured SEND rate (e.g., `count=51 in 1.0s`),
- the configured limit (e.g., `limit=50/s`),
- the cumulative dropped SEND count.

This alert provides forensic evidence that the degradation is due to malicious (or misconfigured) SEND traffic, rather than general network instability.

**(3) Containment: disconnect + cooldown lock.** In addition to dropping excess SEND frames, EmuOCPP can actively contain the source by:

- closing the abusive session (disconnecting the CP),
- temporarily **locking the CP ID** during a cooldown window (rejecting repeated reconnect attempts for that CP ID).

This containment prevents a single endpoint from repeatedly reconnecting and re-initiating a flood, and it helps stabilize the CSMS under sustained adversarial load. The mitigation is therefore not only “drop messages”, but also “enforce availability policy”.

**Configuration knobs (tunable thresholds).** The mitigation is configurable through the CSMS security configuration, including:

- `send_max_per_second`: maximum allowed SEND messages per second (per CP session),
- `send_dos_cooldown_seconds`: lockout duration after flood detection.

These parameters allow tuning based on deployment characteristics (expected reporting frequency, CP fleet size, server capacity).

**Experimental evidence.** Figure 5.9 shows an excerpt of CSMS logs during a SEND flood. The highlighted lines demonstrate the full mitigation chain: the CSMS receives several MessageTypeId 6 frames, then detects that the sender exceeded the configured rate, begins dropping SEND frames, raises an `ALERT` for SEND DoS detection, and finally disconnects the abusive session. Importantly, the presence of subsequent normal control traffic (e.g., Heartbeats) indicates that the CSMS remains responsive and recovers after isolating the abusive sender.

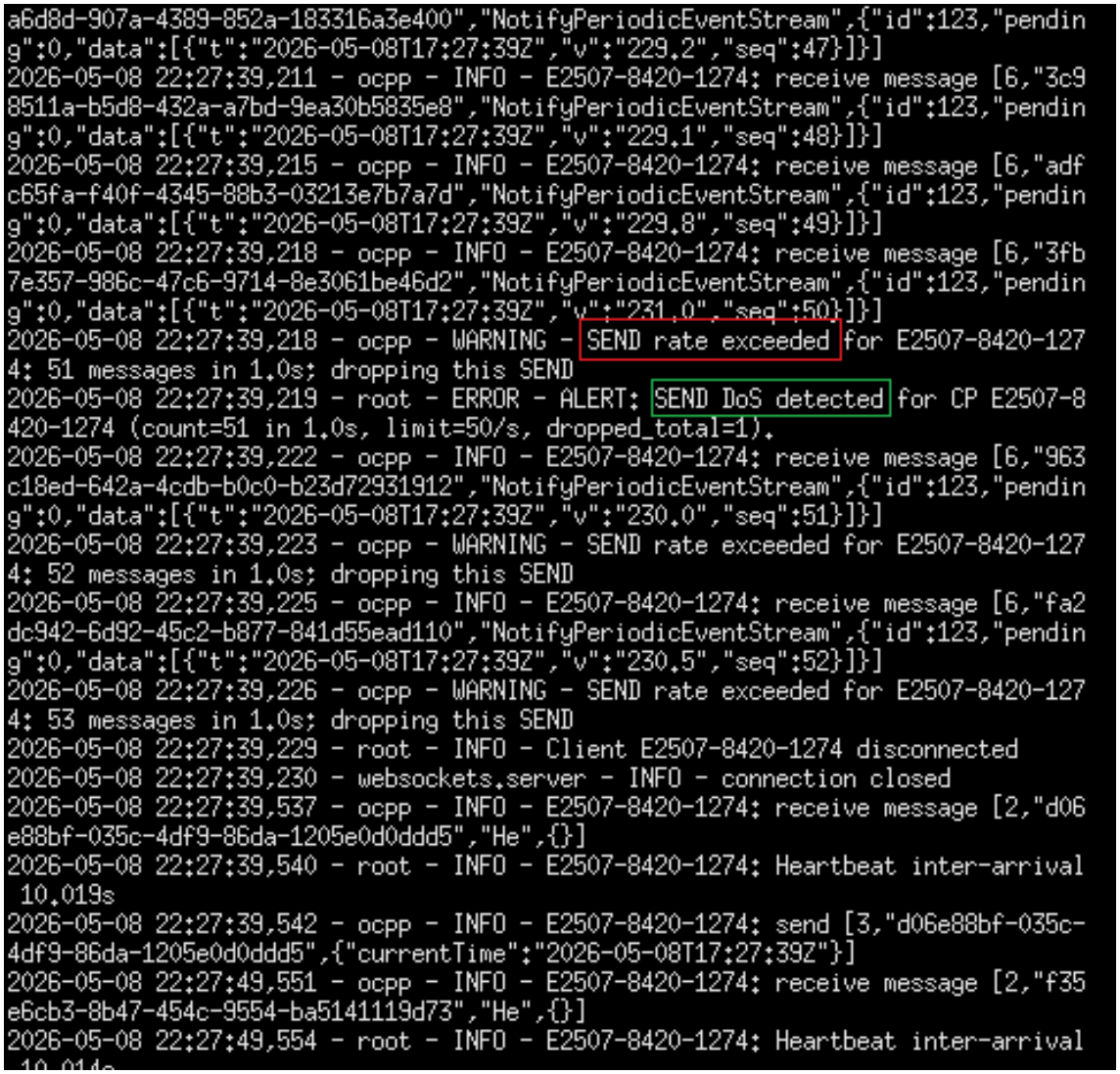



Figure 5.9: *CSMS mitigation logs during a SEND flood: after receiving multiple MessageTypeId* $= 6$ *frames, the CSMS detects that the sender exceeds the configured per-CP SEND rate (e.g., 51 messages in 1.0s with limit 50/s), begins dropping SEND frames, raises an explicit SEND DoS alert, and disconnects the abusive session.*

**Discussion:** This mitigation directly targets the OCPP 2.1 SEND asymmetry by reintroducing effective backpressure at the receiver. Rather than relying on transport-layer protections alone (TLS), the CSMS enforces application-level fairness on the shared WebSocket channel. While an attacker can still send traffic, the CSMS limits the work it performs per attacker and preserves availability for higher priority operations.

### 5.3.2 Man In The Middle Attacks (M1-M4) in OCPP 2.1

In earlier OCPP deployments (and in our emulator when running OCPP 2.0.1 and below), the MitM chain M1-M4 is primarily enabled by the availability of a weak connection mode that permits interception and traffic manipulation without strong server authentication and without a cryptographically protected channel. In practice, this was achievable by steering the charging station onto Security Profile 1 (SP1), where communication can occur without TLS and where the attacker can transparently observe, modify, and replay messages. Once positioned on-path, the attacker can then attempt upgrade and downgrade manoeuvres (M2-M4) by interfering with the stations security configuration and certificate flows.

#### 5.3.2.1 Why the attacker cannot gain MitM position in OCPP 2.1

When executing M1-M4 against the OCPP 2.1 stack, the attacker terminal never successfully establishes a stable MitM position and the client exhibits repeated connect/disconnect cycles (e.g., "Connected successfully with security profile 3 ... Connection was forcefully closed by the server ... reconnects"), as shown in Figure 5.10 and Figure 5.11. This behaviour is consistent with the fact that, in OCPP 2.1, a valid deployment must use TLS-based profiles and therefore the attacker cannot simply insert themselves as an invisible proxy on plaintext traffic. Any attempt to intercept or alter the connection either fails at the transport security layer (TLS handshake and verification) or is detected by the endpoint and results in the connection being closed.

```
Loading...
Connected successfully with security profile 3.
[E2507-8420-1274] Connected to server
[E2507-8420-1274] Connection was forcefully closed by the server
Failed to connect with security profile 3: Connection lost with server
Reconnecting in 5 seconds...
Failed to connect with security profile 3:
Reconnecting in 5 seconds...
Connected successfully with security profile 3.
[E2507-8420-1274] Connected to server
```

Figure 5.10: *Client reconnection behaviour under OCPP 2.1 when executing M1-M4, the connection repeatedly establishes under Security Profile 3 and is subsequently closed, preventing the attacker from maintaining a stable MitM position.*

Crucially, OCPP 2.1 defines only three security profiles, and while it describes an Unsecured Transport with Basic Authentication profile, it explicitly states that operating with-

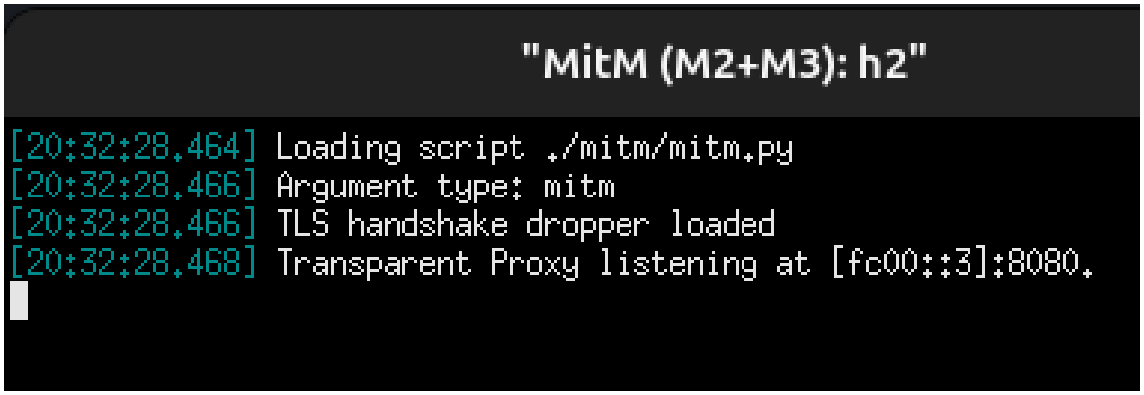



Figure 5.11: *Attacker terminal output when attempting the M1-M4 MitM chain under OCPP 2.1: the attacker does not gain a stable MitM position and no meaningful interception occurs, indicating that the attack path is mitigated.*

out implementing security is not considered a valid OCPP 2.1 implementation, as shown in Figure 5.12. The specification emphasizes that SP1 does not authenticate the CSMS and does not establish a secure channel, and consequently it should be limited to trusted networks (e.g., protected by VPN). For real-world field operation, OCPP 2.1 strongly recommends TLS-based profiles (SP2 or SP3). In other words, the security posture expected by OCPP 2.1 removes the primary precondition that made M1 feasible: a deployment in which an attacker can intercept unencrypted station-to-CSMS traffic and successfully impersonate one endpoint without being challenged by certificate validation.

The Unsecured Transport with Basic Authentication Profile does not include authentication for the CSMS, or measures to set up a secure communication channel. Therefore, it should only be used in trusted networks, for instance in networks where there is a VPN between the CSMS and the Charging Station. For field operation it is highly recommended to use a security profile with TLS.

In some cases (e.g. lab installations, test setups, etc.) one might prefer to use OCPP 2.1 without implementing security. While this is possible, it is NOT considered a valid OCPP 2.1 implementation.

When the Charging Station does not have the correct date and time set, it cannot validate the server certificate. A solution for this might be to either use NTP, mobile network to set time automatically, or have an installer tool that sets the time before the first connection.

Figure 5.12: *Excerpt from the OCPP 2.1 specification highlighting that operating without implementing security (i.e., relying on unsecured transport / SP1-like operation) is not considered a valid OCPP 2.1 implementation, and recommending TLS-based security profiles for field operation.*

#### 5.3.2.2 How certificate management breaks the M1–M4 chain

OCPP 2.1's practical resistance to M1-M4 is tightly linked to certificate-based authentication and trust management:

- **Transport protection (SP2/SP3):** In TLS-based profiles, the WebSocket connection is established over `wss://`. The Charging Station validates the CSMS certificate against a trusted root store, and (in SP3) the CSMS validates the Charging Station's client certificate. This makes passive interception ineffective (traffic is encrypted) and active modification infeasible without breaking TLS.

- **Trust anchor control:** The attack scripts for older versions relied on the attacker being able to position themselves on-path and then influence certificate installation/selection (e.g., by injecting a malicious root certificate or manipulating certificate enrolment messages). In OCPP 2.1, certificate handling prevents this kind of transparent substitution: the Charging Station and CSMS must already share an authenticated trust basis (trusted roots and correct certificate chains) for the TLS session to complete. If the attacker presents a forged certificate chain, the TLS handshake fails unless the attacker's root is already trusted.

- **Safe failure mode:** The observed client behaviour (connection closures followed by retries) reflects a fail-secure mode: when certificate validation or secure channel establishment fails, the connection is terminated rather than silently falling back to an insecure mode. This prevents an attacker from forcing the system into a weaker state through disruption.

#### 5.3.2.3 Implications for each MitM phase (M1–M4)

The MitM attack chain M1–M4, which was previously achievable in OCPP 2.0.1 and earlier versions, is summarized in Algorithm 5. In OCPP 2.1, the prerequisites required by this algorithm are no longer satisfiable in a valid deployment, primarily due to the enforced use of TLS-based security profiles and certificate-based authentication.

- **M1 (MitM on SP1):** M1 requires a plaintext or weakly authenticated transport where the attacker can read and alter messages after gaining on-path position. Under OCPP 2.1's expected deployment, the Charging Station uses `wss://` with certificate validation (SP2/SP3). Even if the attacker becomes on-path at the IP layer, they cannot decrypt or modify traffic, and they cannot impersonate the CSMS without presenting a certificate that chains to a trusted root. Consequently, Phase 2 of Algorithm 5 (establishing a usable MitM for application-layer manipulation) becomes ineffective, because the transport layer prevents transparent interception.

- **M2/M3 (SP2/SP3 upgrade manipulation):** These phases assume the attacker can remain in the middle while certificate changes occur and can substitute certificates or interfere with enrolment (Phase 3 in Algorithm 5). In OCPP 2.1, the secure channel and certificate validation prevent the attacker from transparently injecting

themselves into the certificate lifecycle unless they already control a trusted CA or have compromised endpoint trust stores. As a result, attempts to inject attacker-controlled trust anchors or leaf certificates fail at the TLS handshake and lead to immediate connection termination rather than silent acceptance.

- **M4 (downgrade back to SP1):** In pre-2.1 scenarios, the attacker could attempt to trigger fallback by selectively breaking secure connections and relying on permissive downgrade behaviour to restore visibility on SP1 (Phase 4 in Algorithm 5). OCPP 2.1's intended security posture is that operating without implementing security is not considered a valid implementation, and secure endpoints are expected to fail closed rather than downgrade to an unauthenticated plaintext mode. This removes downgrade as a reliable path to re-enter a weak transport state and maintain attacker visibility.

**Algorithm 5:** Full MitM + Downgrade Attack (M1–M4)

**Input:** CP host $h_1$; CSMS host $h_3$; attacker host $h_2$; blocking time $\Delta t$ (e.g., 35s)

**Output:** CP downgraded from SP3/SP2 to SP1 under attacker-controlled MitM

**Initialize:** Build IPMininet topology with CP ($h_1$), attacker ($h_2$), CSMS ($h_3$), CSO ($h_4$), and configure routes between $h_1$ and $h_3$ via the backbone; start network with IPv6 addressing

**Procedure** *Full_MitM_Downgrade_Attack()*:

**1: Baseline Infrastructure (Legitimate Setup)**

Launch CSMS on $h_3$ listening on ports 9000–9007 (SP0–SP3)

After startup delay, launch CSO on $h_4$ pointing to $h_3$

After startup delay, launch CP on $h_1$ pointing to $h_3$

Wait until CP completes `BootNotification` and is authenticated

**2: M1 – Establish MitM Position**

On attacker $h_2$, prepare iptables NAT and forwarding rules

Launch NDP spoofing from $h_2$ to poison $h_1$'s neighbor cache

Launch router flooding from $h_2$ to accelerate cache takeover

Wait until all CP traffic from $h_1$ to $h_3$ is routed via $h_2$; then stop flooding while maintaining the MitM path

**3: M2 + M3 – Certificate Attacks**

Force CP disconnection by disabling forwarding on $h_2$ for $\Delta t$ so the active session times out at the CSMS

Re-enable forwarding so the CP begins reconnect attempts through $h_2$

On $h_2$, deploy transparent proxy: apply `routes.mn` redirection and launch `mitmdump` with attacker root CA and attack certificates

When CP reconnects via $h_2$:

- Intercept `InstallCertificate` to inject attacker root CA (M2)
- Intercept `SignCertificate` to inject attacker-signed leaf cert (M3)
- Capture the private key material when `combined.pem` is generated

Wait until CP operates on SP2/SP3 using attacker-controlled certificates while still routed through $h_2$

**End**

**Algorithm 5:** *

Algorithm 5 (continued): Full MitM + Downgrade Attack (M4)

**4: M4 – Enforced Downgrade to SP1**

On attacker $h_2$, remove transparent proxy routing (`rem_routes.mn`) but keep the MitM data path in place

Install firewall rules on $h_2$:

- Drop traffic to and from the SP3 port (e.g., TCP 9007)
- Drop traffic to and from the SP2 port (e.g., TCP 9006)
- Leave the SP1 port (e.g., TCP 9005) open for downgrade

Wait for CP reconnection attempts:

- CP attempts SP3 $\rightarrow$ timeout
- CP attempts SP2 $\rightarrow$ timeout
- CP falls back to SP1 and successfully authenticates

Mark the downgrade as successful and continue monitoring CP traffic on weaker SP1 while attacker retains MitM visibility

#### 5.3.2.4 Conclusion

The key observation from our experiments is that the MitM chain M1-M4 is not merely "harder" in OCPP 2.1; rather, its enabling condition, operational reliance on SP1-style unsecured transport, is no longer compatible with a valid OCPP 2.1 deployment. In versions up to OCPP 2.0.1, mixed deployments and permissive fallback made it realistic to end up on weak transport settings under disruption, enabling MitM positioning and message manipulation. In contrast, OCPP 2.1's emphasis on TLS-based communication and certificate validation causes MitM attempts to collapse at connection establishment: interception leads to handshake/validation failure and the endpoints close the connection and retry, rather than continuing insecurely.

Overall, the experimental outcome (the attacker never stabilizing a MitM position; the client repeatedly reconnecting under Security Profile 3) aligns with the OCPP 2.1 security-profile requirements and the stronger role of certificate management in enforcing authenticated, encrypted channels.

### 5.3.3 Impersonation Attacks (I1 and I2)

This section analyzes impersonation attacks in our emulator with emphasis on the contrast between OCPP 2.0.1 and OCPP 2.1, and on why the attacks were successful in the former but mitigated in the latter. The key finding is that mitigation in the updated environment is driven by **credential binding**: the charging station identity is bound to cryptographic client credentials (SP3 mTLS + certificate subject checks), not only to a static CP ID

string.

#### 5.3.3.1 Impersonation Attacks (I1 and I2)

Both I1 and I2 target the same core weakness: CP identity ambiguity when the CSMS relies primarily on static/public station IDs and weak duplicate ID handling. The two attacks differ in timing:

- **I1:** attacker impersonates while victim is still connected.
- **I2:** attacker waits for victim offline window, then reconnects first.

In our experiments, we tested the attacks in two regimes:

1. **OCPP 2.0.1 path (legacy behavior):** attack feasibility under weaker identity binding.
2. **OCPP 2.1 path (updated behavior):** strict SP3 enforcement with credential binding.

**Note** In our emulator's OCPP 2.1 setup, practical operation is TLS only for CP connectivity profiles used in testing (SP2/SP3), and **SP1 is not part of in the OCPP v2.1**. This is relevant because SP1 style password only assumptions are exactly what make impersonation easier in legacy/weak deployments.

#### 5.3.3.2 I1 — Impersonation with victim online

**How we tested I1** The victim CP establishes a normal session. The attacker then attempts a second session with the same CP ID. We observed behavior under duplicate-ID policies and security profiles.

**Why I1 was successful in 2.0.1 path** In the legacy path, station identity could still be effectively treated as a static CP ID + credentials combination in practice, so a second endpoint claiming the same ID could be accepted (or could replace victim depending on policy):

- `allow_multiple_serial_numbers = 1`: both sessions may coexist.
- `allow_multiple_serial_numbers = 2`: latest session may replace victim.

Thus, the attacker could gain a valid session under victim identity.

**Why I1 is mitigated in 2.1 path** In the updated 2.1 path with strict SP3 enforcement:

- client certificate is required,
- certificate chain is verified,
- certificate identity (e.g., CN) is bound to CP ID.

An attacker that only knows CP serial/ID (and even password) but *does not possess the victim certificate/private key* cannot complete authentication/binding. Therefore, the impersonation session is rejected.

**Pseudocode**

**Algorithm 6:** Charger Impersonation Attack (I1 – Victim Online)

**Input:** CP identity $(vendor, model, cp_id)$; attacker knowledge of CP credentials; CSMS policy `allow_multiple_serial_numbers`; selected SP (2.0.1 or 2.1)

**Output:** Attacker obtains a concurrent or takeover session under the victim CP ID (if not mitigated)

**Initialize:** Configure CSMS duplicate-ID policy and select OCPP version + SP;

**Procedure** *Impersonation_I1()***:**

```
Launch legitimate CP client using true identity (vendor, model, cp_id);
Wait until CSMS acknowledges BootNotification;
Launch attacker CP client using same identity (vendor, model, cp_id);
if SP == 3 and CSMS enforces client-cert binding to cp_id (e.g., CN/SAN ==
  cp_id) then
    Attacker fails mutual TLS or binding check (no valid client certificate);
    Legitimate session remains active; attack fails;
else
    if policy == 0 then
        CSMS rejects attacker as duplicate ID; attack fails;
    else if policy == 1 then
        CSMS keeps both sessions connected; attacker operates in parallel;
    else if policy == 2 then
        CSMS drops victim connection and keeps attacker session ("last wins");
    end
end
if attacker session accepted then
    Record session outcome (parallel vs takeover) and CSMS behavior for
      analysis;
end
```

**End**

Algorithm 6 summarizes I1 as a parallel-session impersonation attempt in which the adversary reuses the victims identity tuple to establish a second session while the legitimate

charging station remains online. The procedure first confirms the victim is fully registered by waiting for an accepted `BootNotification`. It then launches the attacker with the same identity. In the 2.0.1 like path (weak binding), the outcome depends on the CSMS duplicate ID policy (reject, coexist, or replace). In the 2.1 hardened path, strict SP3 credential binding prevents acceptance unless the attacker also possesses the victims client certificate and private key.

#### 5.3.3.3 Experimental Validation of I1 Mitigation in OCPP 2.1 Under Policy 1

The following screenshots demonstrate that impersonation attack I1 fails in the hardened OCPP 2.1 SP3 environment even under Policy 1, where the CSMS normally allows multiple simultaneous connections for the same CP ID. Although the attacker attempts to establish a parallel session using the victim charging station identity, the attack is rejected because the attacker does not possess the required client certificate and private key bound to the CP identity.

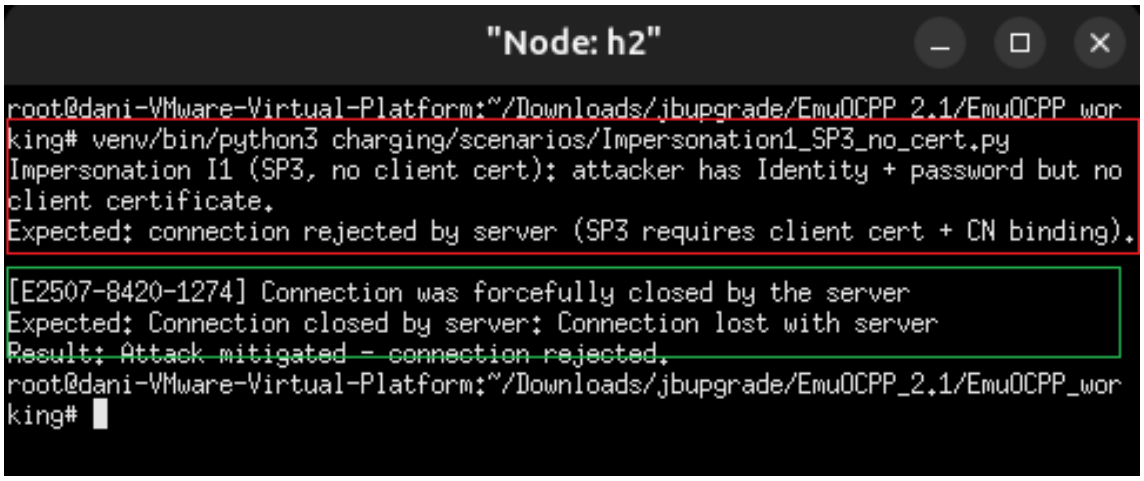



Figure 5.13: Attacker terminal output during impersonation attack I1 under OCPP 2.1 SP3 with Policy 1 enabled. The attacker attempts to connect using the same CP ID as the legitimate charging station while the victim session is already active. Although Policy 1 permits simultaneous sessions, the attacker connection is rejected because no valid client certificate is provided for SP3 authentication.

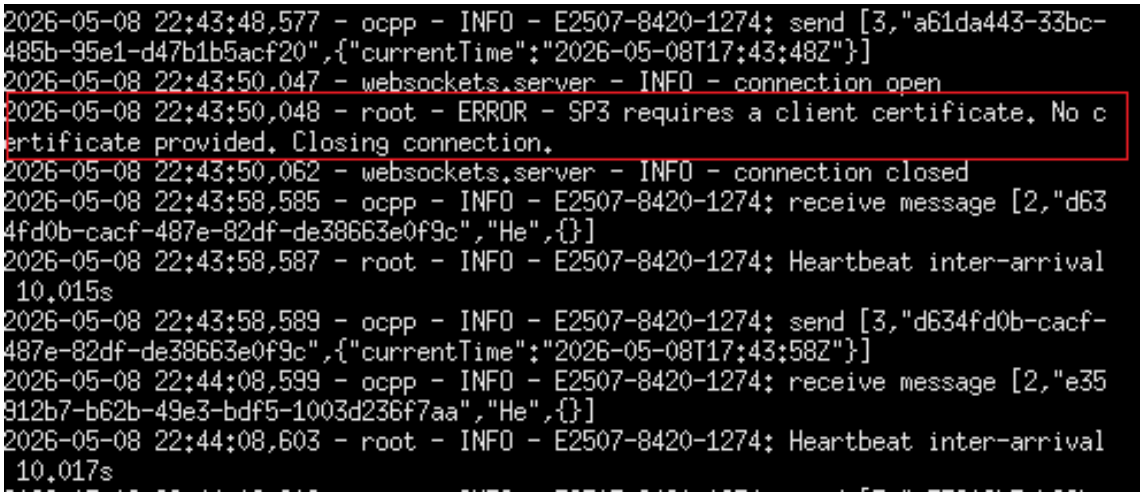



Figure 5.14: CSMS logs during impersonation attack I1 under OCPP 2.1 SP3. The highlighted entries show that the server enforces mutual TLS authentication and rejects the attacker connection with the error message "SP3 requires a client certificate. No client certificate provided." The logs further confirm that the legitimate charging station session remains active while the attacker connection is denied.

#### 5.3.3.4 I2 — Impersonation in offline/reconnect window

**How we tested I2** Victim first connects normally, then is disconnected (reboot/service-restart window). During this offline window, attacker attempts first reconnect with victim CP ID. After attacker attempt, victim tries to reconnect again.

**Why I2 was successful in 2.0.1 path** Even with "one active session per ID" policies, I2 can still succeed if attacker reconnects *first* after victim drops:

- victim offline opens race window,
- attacker claims same ID first,
- CSMS may register attacker as legitimate current holder of that ID,
- victim reconnect then appears as duplicate and is rejected.

Hence, I2 can bypass the apparent protection of duplicate reject policy by exploiting timing.

**Why I2 is mitigated in 2.1 path** Under strict 2.1 SP3 credential binding, winning the race is not enough:

- attacker still must present correct client cert/key for that CP identity,
- without victim certificate, attacker cannot establish authenticated bound session,
- victim reconnect remains the only valid identity holder.

Therefore, I2 fails in our updated 2.1 hardened path unless attacker also compromises victim cryptographic material.

**Pseudocode**

**Algorithm 7:** Charger Impersonation Attack (I2 – Offline Window Takeover)

**Input:** CP identity $(vendor, model, cp_id)$; attacker knowledge of CP credentials; CSMS policy `allow_multiple_serial_numbers`; selected SP (2.0.1 or 2.1)
**Output:** Attacker captures the CP ID during a victim disconnect window (if not mitigated)
**Initialize:** Configure CSMS duplicate-ID policy and select OCPP version + SP;
**Procedure** *Impersonation_I2()*:
    Launch legitimate CP client using true identity $(vendor, model, cp_id)$;
    Wait until CSMS acknowledges `BootNotification`;
    Induce or wait for victim disconnect (reboot/service restart/offline window);
    Attacker monitors for offline window (timing or observed disconnect);
    Launch attacker CP client using same identity $(vendor, model, cp_id)$ during offline window;
    **if** *SP == 3* ***and*** *CSMS enforces client-cert binding to cp_id (e.g., CN/SAN == cp_id)* **then**
        Attacker fails mutual TLS or binding check; takeover fails;
    **else**
        CSMS accepts attacker as first post-disconnect session for $cp_id$;
        Attacker occupies CP identity;
    **end**
    Victim attempts to reconnect using $(vendor, model, cp_id)$;
    **if** *attacker occupies cp_id* **then**
        CSMS rejects victim reconnect as conflicting/duplicate; I2 succeeds;
    **else**
        CSMS accepts victim reconnect; I2 mitigated;
    **end**
    Record CSMS decision and availability impact for analysis;
**End**

Algorithm 7 describes I2 as an offline window takeover. The procedure first brings the victim online and confirms registration via `BootNotification`. It then opens an offline window (e.g., reboot/service restart) and has the attacker connect first with the same CP ID. In weak-binding deployments (2.0.1 like behavior), the CSMS may accept the attacker

as the first session and later reject the victim reconnect. In the hardened 2.1 path, strict SP3 credential binding prevents the attacker from occupying the identity without the victims certificate/private key, so the victim can reconnect successfully.

#### 5.3.3.5 Evidence of I2 Impersonation Attack Mitigation in OCPP 2.1

The following screenshots demonstrate that impersonation attacks I1/I2 fail in the hardened OCPP 2.1 SP3 environment because the attacker cannot satisfy certificate-based credential binding.

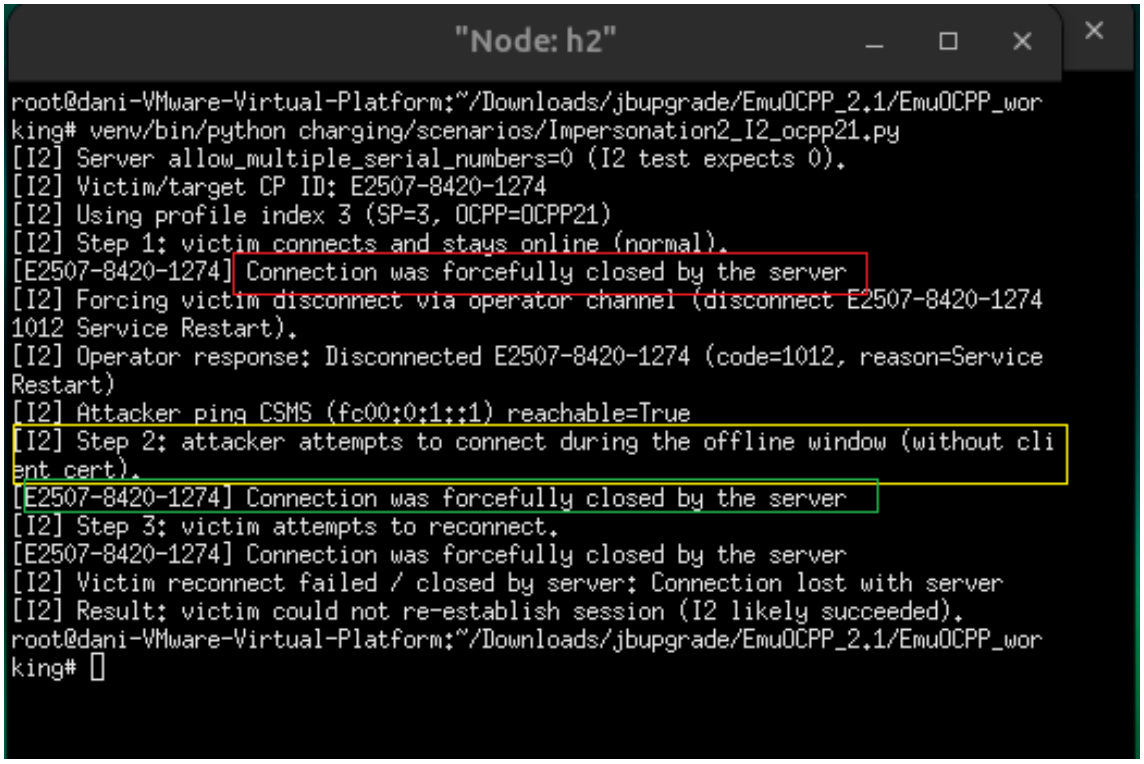



Figure 5.15: Attacker terminal output for impersonation I2 under OCPP 2.1 SP3. The highlighted lines show that the attacker first triggers an offline window by forcing the victim to disconnect (code 1012), then attempts to reconnect using the same CP ID. The CSMS rejects the takeover because the attacker does not present the bound client certificate, and the script proceeds to the victim reconnect step.

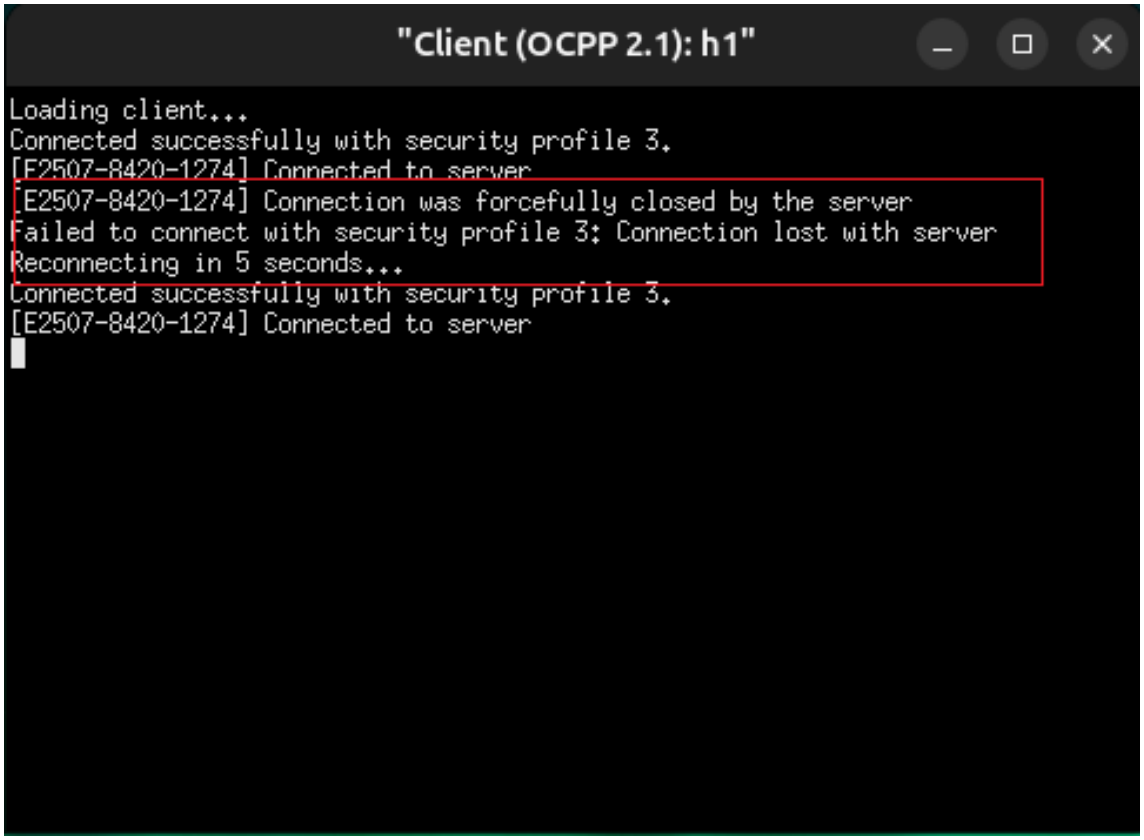



Figure 5.16: Victim client output during impersonation I2. The highlighted boxes show that the legitimate CP connects under SP3, is forcefully disconnected by the CSMS (service restart window), and then reconnects successfully. This indicates that while the attacker can create a disruption window, it cannot steal the session without the bound client certificate.

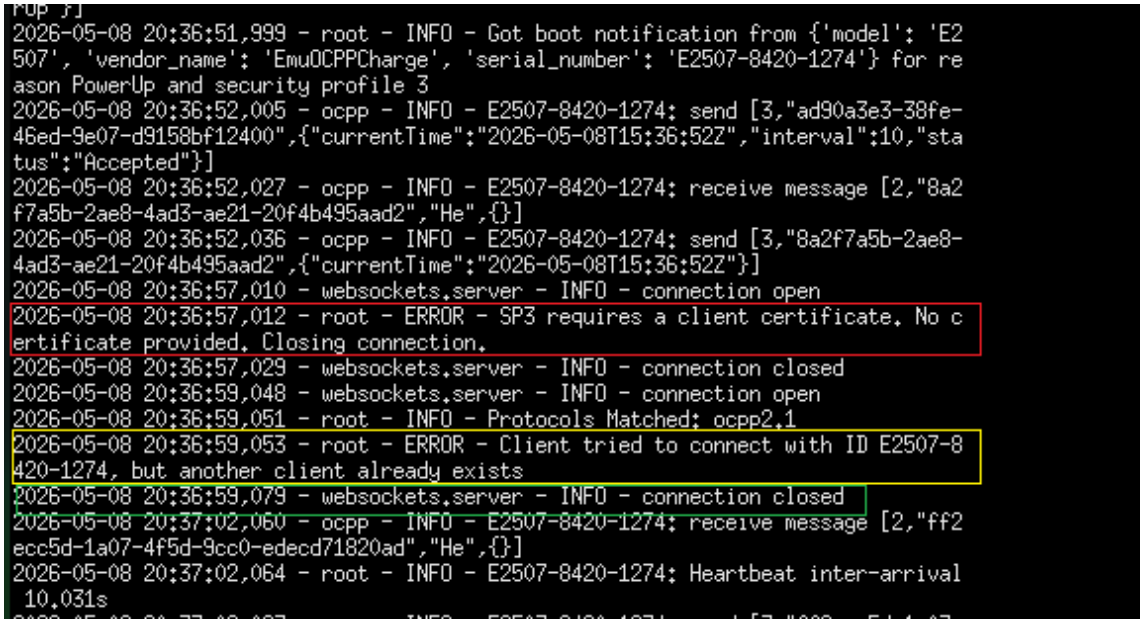



Figure 5.17: CSMS logs during impersonation I2 under OCPP 2.1 SP3. The highlighted entries show the CSMS enforcing mutual TLS and rejecting the attacker connection because no client certificate is provided. After the forced disconnect, the legitimate CP reconnects and continues operation, confirming that timing alone is insufficient to hijack the CP ID when certificate binding is enforced.

#### 5.3.3.6 Cross-version discussion: why 2.0.1 succeeded, why 2.1 did not

The difference we observed is not merely "new version number"; it is the **enforcement quality** of identity checks:

- In the 2.0.1 attack path, practical behavior allowed identity ambiguity based on CP ID/timing/policy.
- In the updated 2.1 path, identity is anchored to cryptographic credentials and explicit certificate-to-ID checks.

This explains the key result: I1/I2 were successful in the weaker 2.0.1 path, and were mitigated in the hardened 2.1 path because of credential binding.

#### 5.3.3.7 Cross-version discussion: why 2.0.1 succeeded, why 2.1 did not

The difference we observed is not merely "new version number"; it is the **enforcement quality** of identity checks:

- In the 2.0.1 attack path, practical behavior allowed identity ambiguity based on CP ID/timing/policy.
- In the updated 2.1 path, identity is anchored to cryptographic credentials and explicit certificate-to-ID checks.

This explains the key result:

**I1/I2 were successful in the weaker 2.0.1 path, and were mitigated in the hardened 2.1 path because of credential binding.**

#### 5.3.3.8 Residual risk and scope

The mitigation blocks ID only and timing only impersonation. Remaining risk shifts to *credential compromise*:

- attacker steals victim certificate/private key,
- attacker abuses misconfigured trust/certificate validation.

Thus, secure key storage and strict PKI operations remain necessary defense layers beyond protocol flow hardening.

# Chapter 6

# Conclusions and Future Work

This research set out to address a critical gap in the empirical evaluation of OCPP 2.1, the latest iteration of the de facto communication standard for EV charging infrastructure. While prior work has extensively examined OCPP 1.6 and 2.0.1, no systematic, emulation-based study had assessed whether the security enhancements introduced in OCPP 2.1 effectively mitigate previously documented vulnerabilities or inadvertently expose new attack surfaces. By extending the EmuOCPP framework to support OCPP 2.1 and replicating, comparing, and extending prior attack scenarios, this study provides validated, evidence-based insights into the security posture of the protocol's most recent release.

## 6.1 Summary of Findings

The experimental results confirm that the Man-in-the-Middle (M1–M4) and impersonation (I1, I2) attacks originally demonstrated against OCPP 2.0.1 are reproducible under realistic adversarial conditions, consistent with previously published findings. Packet-level inspection through the developed Wireshark dissector further validated that these vulnerabilities exist at a fundamental design level rather than as artifacts of specific implementations.

When the same attacks were executed against the upgraded OCPP 2.1 environment, the outcomes diverged sharply. The strengthened certificate management workflow, the correlation of `SignCertificateRequest` with `CertificateSigned` via `requestId`, the controlled Security Profile transitions introduced in use case A05, and the hardened provisioning fields in B09 collectively break the chain of preconditions on which M1–M4 depend. Likewise, the impersonation attacks I1 and I2 were neutralised under Policy 1, confirming that the protocol-flow hardening introduced in 2.1 closes the identifier-only and timing-only impersonation vectors observed in 2.0.1.

At the same time, the upgrade revealed a new attack surface. The `SEND` message type,

introduced in OCPP 2.1 to support fire-and-forget asynchronous notifications, lacks the back-pressure and acknowledgement semantics of the existing `CALL`/`CALLRESULT` pattern. This research demonstrated a SEND-flood Denial-of-Service attack (D2) that exploits this asymmetry, and proposed a mitigation strategy combining per-Charging-Point rate limiting, alerting, and containment. These findings underscore that while OCPP 2.1 substantially raises the bar against the most prominent vulnerabilities of its predecessor, the addition of new message semantics introduces operational risks that must be addressed at the CSMS layer.

## 6.2 Research Contributions

This work makes three principal contributions to the field of EV charging infrastructure security:

- **Specification-level comparative analysis of OCPP 2.0.1 and OCPP 2.1.** A structured review of the protocol's security profiles, certificate lifecycle, ISO 15118 integration, provisioning workflows, monitoring constructs, and Device Model hierarchy was performed, producing a consolidated comparison that documents how OCPP 2.1 evolves the security and architectural foundations of 2.0.1 without altering its underlying transport or cryptographic layers.

- **Extension of the EmuOCPP framework to OCPP 2.1.** The base framework was upgraded with a compatibility shim that preserves backward behaviour while enabling the new 2.1 constructs. Support was added for the SEND message type, CALLRESULTERROR asynchronous failure reporting, and the use cases A02, A03, A05, B09, F06, and M01, together with the certificate-correlation and network-profile fields introduced in the 2.1 specification. The resulting environment provides a reproducible, extensible testbed for OCPP 2.1 security research.

- **Empirical security evaluation across protocol versions.** The MitM (M1–M4), impersonation (I1, I2), and the newly identified SEND-flood DoS (D2) attacks were systematically executed against both OCPP 2.0.1 and OCPP 2.1 deployments. The results provide the first emulation-based evidence of which prior vulnerabilities are mitigated by 2.1, which residual risks persist (notably credential compromise and PKI misconfiguration), and which new vulnerabilities the 2.1 message model introduces.

## 6.3 Limitations

Several limitations bound the scope of this study. The evaluation focuses on the subset of OCPP 2.1 use cases most relevant to the attack surfaces inherited from 2.0.1, and does not yet exercise the full set of new functional blocks introduced in 2.1. The mitigation proposed for the SEND-flood attack was validated under emulation rather than on production CSMS deployments, and the residual risk surface around credential compromise and certificate-trust misconfiguration was discussed analytically rather than tested empirically. Finally, the experiments were conducted in a virtualised network topology; while this enables reproducibility, it does not capture the timing characteristics or hardware-level idiosyncrasies of physical Charging Stations.

## 6.4 Future Work

Several directions extend naturally from this research. First, the EmuOCPP 2.1 framework can be expanded to cover the Vehicle-to-Grid (V2G) bidirectional power transfer functionality formalised in OCPP 2.1, including the associated authorization, metering, and scheduling flows. V2G introduces new operational states and message exchanges whose security implications, particularly around energy settlement and grid-side denial-of-service, remain unexamined.

Second, deeper integration with ISO 15118-20 is a priority. While this work covered the certificate-installation workflow (M01) and the updated authorization use cases, a comprehensive evaluation of Plug & Charge under 2.1, including contract certificate provisioning, EVCC–SECC TLS handshakes, and the end-to-end EV-to-CSMS trust chain, would close the remaining gap between protocol-layer testing and full charging-session security.

Third, the framework can be extended to support the remaining OCPP 2.1 functional blocks, including the new monitoring severity levels in N07, the Device Model hierarchy variables, and the delta monitoring types, enabling a broader survey of attack surfaces beyond those inherited from 2.0.1. Complementary directions include validating the proposed SEND-flood mitigation on real CSMS implementations, integrating machine-learning-based intrusion detection trained on the traffic generated by the framework, and porting selected scenarios to hardware-in-the-loop testbeds to assess timing and resource-exhaustion behaviour under realistic deployment conditions.

Collectively, these directions would build on the foundation established in this research and continue the work of providing the EV charging community with the systematic, evidence-based security guidance that the rapid evolution of the protocol demands.

# Appendix A

# Background

## A.1 IPMininet

IPMininet is an extension of Mininet designed to emulate realistic IP networks with full routing functionality. While Mininet provides a lightweight environment to create virtual networks composed of hosts, switches, and links, IPMininet enhances it by integrating routing daemons from FRRouting (FRR), such as OSPF, BGP, and RIP. Each virtual router in IPMininet runs these daemons within its network namespace, enabling real routing behavior just like in a physical network. The tool automatically assigns IP addresses, configures interfaces, builds routing tables, and establishes adjacencies between routers. This makes it possible to simulate complex multi-hop topologies with dynamic route propagation, latency, and bandwidth effects, all while remaining highly reproducible and isolated in software.

## A.2 CheckOCPP

An integral methodological component of this research is the use of CheckOCPP, a Wireshark-based protocol dissector designed specifically for the Open Charge Point Protocol (OCPP). It serves as a network-level validation and analysis tool that enables the passive inspection of OCPP traffic for compliance, correctness, and version consistency. CheckOCPP supports several key analytical functions that align directly with the studys objectives of evaluating and comparing OCPP security resilience:

- **Automatic Version Detection:** Distinguishes between OCPP 1.6, 2.0, and 2.0.1 traffic, enabling precise multi-version comparison and protocol evolution tracking.
- **Protocol Compliance Validation:** Verifies message schema conformance against

official OCPP JSON structures, identifying deviations or incomplete implementations.

- **Non-Compliant Packet Detection:** Flags packets that violate structural or semantic rules, highlighting areas of potential misconfiguration or security weakness.

- **IPv4/IPv6 Classification:** Distinguishes transport layers for contextual understanding of network deployment and routing behaviors.

These features allow for systematic, repeatable validation of message flows and protocol behavior [15].

## A.3 Mobility House OCPP Python Library

The Mobility House OCPP Python library is an open-source implementation of the Open Charge Point Protocol (OCPP), developed and maintained by The Mobility House to support communication between electric vehicle (EV) charging stations and central management systems. The library provides developers with a ready-made framework for building OCPP-compliant applications in Python, supporting versions OCPP 1.6, OCPP 2.0.1, and OCPP 2.1. It is designed to simplify the complex processes of message handling, schema validation, and WebSocket communication defined by the OCPP standard, allowing users to focus on the functional and analytical aspects of their systems.

Built on Pythons asynchronous `asyncio` architecture, the library enables efficient handling of multiple concurrent connections between charging points and central systems. It provides core components such as message classes (`Call`, `CallResult`, and `CallError`), payload schemas for various OCPP actions, and decorators for message routing. These abstractions allow developers to define handlers for specific messages (e.g., `BootNotification`, `StatusNotification`, `MeterValues`) without manually managing low-level communication details. The library facilitates seamless requestresponse cycles between charge points and central systems using WebSocket-based JSON messages.

The Mobility House library ensures strict adherence to OCPP message structures and communication patterns, maintaining protocol compliance while remaining flexible for customization. It can be used to simulate charging infrastructure, prototype smart charging algorithms, or build real-world central system applications. Although it does not offer a full end-to-end solution, requiring users to implement additional features such as authentication, database management, and security, it provides a stable and extensible foundation for OCPP-based research and development. Overall, the library plays a crucial role in enabling efficient, standardized, and interoperable EV charging communication systems.

## A.4 List of Abbreviations

| Abbreviation / Code | Description |
|---|---|
| OCPP | Open Charge Point Protocol: Communication protocol between EV charging stations and central systems. |
| IPMininet | Extension of Mininet to emulate realistic IP networks with routing daemons (OSPF, BGP, RIP). |
| FRR | FRRouting: Open-source routing software used in IPMininet for dynamic routing protocols. |
| CSMS | Central System Management System: Backend managing EV charging stations. |
| EVSE | Electric Vehicle Supply Equipment: Charging station hardware component. |
| EV | Electric Vehicle. |
| BootNotification | OCPP message sent by a charge point to inform the CSMS about its status and capabilities. |
| StatusNotification | OCPP message reporting real-time status of a charging point. |
| MeterValues | OCPP message sending energy metering and consumption data from a charge point. |
| CheckOCPP | Wireshark-based OCPP dissector used for protocol validation and traffic analysis. |
| IdToken | Identity abstraction used in authorization; represents a user, card, or EV identity token. |
| **OCPP 2.0/2.0.1 Use Case Codes** | |
| A0 | Security functional block baseline. |
| A02 | Certificate update initiated by CSMS. |
| A03 | Certificate update initiated by Charging Station. |
| A04 | Security Event Notification. |
| A05 | Upgrade / downgrade Security Profile ($2 \leftrightarrow 3$). |
| B09 | Setting a new NetworkConnectionProfile. |
| B10 | Migrating network profiles. |
| C07 | ISO 15118-based authorization. |
| C08 | ISO 15118-based authorization (continued flow). |
| C17–C25 | Authorization use cases for prepaid cards, integrated payment terminals, stand-alone terminals, and QR-code payments. |

Table A.1: Abbreviations and OCPP use-case identifiers used throughout the document.

# References


[1] M. H. Boussaha, A. F. M. Shahriar, M. Ma, and M. Conti. Emuocpp: Effective and scalable ocpp security and privacy testing. In *Vehicle Security and Privacy Workshop (VehicleSec '25)*, San Diego, CA, USA, 2025.

[2] G. Coppoletta, R. Gjomemo, A. Kaur, N. Valizadeh, V. N. Venkatakrishnan, and O. Rana. Ocppstorm: A comprehensive fuzzing tool for ocpp implementations. In *Vehicle Security and Privacy Workshop (VehicleSec 2024)*, 2024.

[3] C. Dalamagkas et al. Federated detection of open charge point protocol 1.6 cyberattacks. *Complex Engineering Systems*, 3(1):45–62, 2025.

[4] D. Elmo, G. Fragkos, J. Johnson, K. Rohde, S. Salinas, and J. Zhang. Disrupting ev charging sessions and gaining remote code execution with dos, mitm, and code injection exploits using ocpp 1.6. In *Proceedings of Resilience Week (RWS)*, pages 1–8, National Harbor, MD, USA, 2023.

[5] S. Hamdare, O. Kaiwartya, M. Aljaidi, M. Jugran, Y. Cao, S. Kumar, M. Mahmud, D. Brown, and J. Lloret. Cybersecurity risk analysis of electric vehicles charging stations. *Sensors*, 23(15):6716, 2023.

[6] Safa Hamdare, David J. Brown, Devki Nandan Jha, Mohammad Aljaidi, Yue Cao, Sushil Kumar, Rupak Kharel, Manish Jugran, and Omprakash Kaiwartya. Cyber defense in ocpp for ev charging security risks. *International Journal of Information Security*, 24(1):134, 2025.

[7] S. Hsaini, M. Boulmalf, N. El Kamoun, and A. El Ouali. Cybersecurity analysis of ocpp-based electric vehicle charging management systems. *Energies*, 15(20):6735, 2022.

[8] H. Jahangir, S. Lakshminarayana, and H. V. Poor. Charge manipulation attacks against smart electric vehicle charging stations and deep learning-based detection mechanisms. *IEEE Transactions on Smart Grid*, 15(5):5182–5194, 2024.

[9] M. Nasr, K. Sarieddine, C. Assi, and M. Debbabi. Chargeprint: A security analysis framework for electric vehicle management systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, pages 1–15, 2023.

[10] Open Charge Alliance. Open charge point protocol 1.6 specification, 2017.

[11] Open Charge Alliance. Open charge alliance overview. Technical report, Open Charge Alliance, 2020. Accessed: 2025-09-03.

[12] Open Charge Alliance. Our mission: Enabling interoperable ev charging, 2022. Accessed: 2025-09-03.

[13] Open Charge Alliance. Open charge point protocol 2.0.1 specification, 2024.

[14] Open Charge Alliance. Certified companies, 2025.

[15] Open Charge Alliance. Open charge point protocol 2.1 specification, 2025.

[16] M. A. Sayed, M. Debbabi, and C. Assi. Impact of cyberattacks on electric vehicle charging and power grid operation. *International Journal of Electrical Power & Energy Systems*, 137:107784, 2022.